408

操作系统

目　录

一、操作系统概述.....5
(一)操作系统的基本概念.....5
1.操作系统概念.....5
2.操作系统的功能和目标.....5
3.操作系统的特征.....6
(二)操作系统的发展历程.....7
(三)程序运行环境.....8
1.CPU 运行模式内核模式（管态）、用户模式（目态）.....8
2.中断和异常的处理 （判断中断方式可以采用硬件识别法）
.....8
3.中断和异常的处理过程.....9
4.系统调用.....10
5.程序的链接与装入.....12
6.程序运行时内存映像与地址空间.....12
(四)操作系统结构.....13
(五)操作系统引导.....15
(六)虚拟机.....16
二、进程管理.....19
(一)进程与线程.....19
1.进程与线程的基本概念.....19

2.进程/线程的状态与转换.....21
3.线程的实现.....22
4.进程与线程的组织与控制.....23
5.进程间通信共享内存，消息传递，管道，信号.....26
(二)CPU 调度与上下文切换.....33
1.调度的基本概念.....33
2.调度的目标.....33
3.调度的实现.....33
4.CPU 调度算法.....35
5.多处理机（多 CPU）调度.....39
6.上下文及其切换机制.....39
(三)同步与互斥.....42
1.同步于互斥的基本概念.....42
2.基本的实现方法软件方法、硬件方法.....43
3.互斥锁.....44
4.信号量.....44
5.条件变量（管程）.....45
6.经典同步问题生产者-消费者问题；读者-写者问题；哲学家
.....46
(四)死锁.....54
1.死锁的基本概念.....54
2.死锁预防 注意二者的区别.....55

3.死锁避免.....55
4.死锁检测和解除.....57
三、内存管理.....58
(一)内存管理基础.....58
1.内存管理的基本概念.....58
2.连续分配管理方式.....61
3.页式管理.....63
4.段式管理.....66
5.段页式管理.....69
(二)虚拟存储管理.....70
1.虚拟内存基本概念.....70
2.请求页式管理.....71
3.页框分配.....72
4.页置换算法.....74
5.页框回收.....75
6.内存映射文件(Memory-Mapped Files).....76
7.虚拟存储器性能的影响因素及改进方式.....77
四、文件管理.....78
(一)文件.....78
(二)目录.....94
1.目录的基本概念.....94
2.树形目录及其它三种结构.....94

3.目录的操作.....96
4.文件共享硬链接和软链接.....96
(三)文件系统.....99
1.文件系统的全局结构(layout).....99
2.外存空闲空间管理办法.....100
3.虚拟文件系统.....104
4.文件系统挂载(mounting).....105
五、输入输出(I/O)管理.....106
(一)l/O 管理基础 （这一节的内容自己整理的不够到位！）..106
1.设备.....106
2.I/O 控制方式.....109
3.I/O 软件层次结构 （理解不够）☆☆☆☆.....112
4.输入/输出应用程序接口 （理解不够）☆☆☆☆.....113
(二)设备独立软件（设备无关软件）.....115
1.缓冲区管理.....115
2.设备分配与回收.....119
3.假脱机技术(SPOOLing).....121
4.设备驱动程序接口（理解不够）☆☆☆☆☆.....122
(三)外存管理.....127
1.磁盘.....127
2.固态硬盘读写性能特性，磨损均衡.....133
六、PV 操作.....134

(1)生产者-消费者问题.....134
(2)理发师问题.....141
(3)读者-写者问题.....145
(4)哲学家进餐问题---需要持有多种资源才能执行.....150
(5)单纯的同步问题.....153

408—操作系统

一、操作系统概述

(一)操作系统的基本概念

1.操作系统概念

好像只有这从上至下四层结构，用户、应用程序、操作系统、硬件

2.操作系统的功能和目标

操作系统是资源管理者：	是硬件与用户之间的接口：	对计算机资源进行扩充：
处理机管理： 对处理机的管理实际是对进程的管理。所以当提到进程管理时，也是对的	命令接口（**是组织和控制作业的执行**）： ①联机命令接口 ②脱机命令接口	裸机就是只有硬件，没有任何系统软件/应用软件支撑。 扩充机&虚拟机=裸机+软件
存储器管理： 提高内存的利用率，	程序接口： 本质上就是一组系统调用，系统调用是操	

内存分配与回收、地址映射、内存保护与共享、内存扩充等	作系统给用户/应用程序提供的。一定要记住是操作系统提供的系统调用，而库函数是高级语言所写函数的汇总。库函数比如简单的数学计算的库函数与系统调用无关，库函数属于用户程序的一部分、系统调用的上层。 系统调用可能需要执行上下文的切换和一定有状态的转换（用户态转向内核态） 例如一些非阻塞系统调用 read（）直接就从缓冲区读走数据了，必须要阻塞进程，也就没有上下文切换 图形用户界面 GUI 它不是操作系统的一部分，但它的功能都是通过系统调用实现的，这才属于操作系统。 普通用户是不能直接使用系统调用，**必须通过代码间接使用。**	
文件管理： 文件存储空间的管理、目录管理、文件		

读写管理和保护。 操作系统对文件进行管理，管理的不是文件的内容，而是文件的物理结构、逻辑结构、内部结构、组织交互。所以操作系统对源程序不管理。		
设备管理： 缓冲管理、设备分配、设备处理和虚拟设备		

3.操作系统的特征

并发	共享	虚拟	异步
宏观同时运行、微观交替运行（通过分时操作来实现的）	**互斥共享：**虽然我们可以使用同一种资源、但是必须使用完之后才允许另一个进程使用该资源	时分复用：虚拟处理器（分时间片交替使用CPU，就好像自己独占CPU一样） 空分复用：虚拟存储器	以不可预知的速度前进，结果不确定

并发和并行的区别，并行只存在于多核CPU！在支持多道程序的单 CPU 电脑中，是可以实现并发执行的	同时访问：这个同时在宏观上也是同时，在微观上其实是交替使用，例如磁盘、重入代码编写的文件	虚拟设备：将一台物理设备虚拟为多个逻辑设备，每个用户进程独占一个逻辑设备，分时使用。 本质就是请求缓冲+串行执行，例如打印机，我另一个进程的数据先保存到缓冲队列里面，继续执行后面的部分。 **例如典型的 SPOOLing 技术**	但是可以通过同步机制来约束执行顺序，限制最终结果
并发和共享互为存在条件			

(二)操作系统的发展历程

手工阶段	单道批处理系统	多道批处理系统（没有任何交互能力）	分时操作系统	实时操作系统	网络操作系统和分布式操作系统
此阶段是没有操作系统的	首次实现脱机，不需要人工一直参与（有监督程序）	所有作业排成队列，可以实现并发操作，特点是“多道、并发、制约（由于不同任务可能使用共享资源，所以互相制约）”	特点就是“分时间片”，多个用户通过终端共享一台主机、关键是如何才能实现用户与主机交互？特点是“同时性、交互性、独立性就是好像自己独占主机、及时性”首次实现人机交互！！	实时操作系统关键是要在非常短的时间内做出反应（比时间片还短），例如订票、火箭发射	网络操作系统：网络中的各个计算机结合起来实现互相之间的数据通信和数据传送
用户独占全机、	问题是：当IO时，CPU空	问题是：用户响应时间长（原因是因为需要等待	分时系统的“交互性”是面向人类用户的广义	硬实时系统：火箭发射！	分布式操作系统：各个计算

CPU 等待手工操作，CPU 的利用不够充分	闲，但又不能让给其他进程，造成 CPU 资源浪费	其他进程执行完才可以，而分时操作系统响应时间短是因为分时间片了，新进程立马可以获得一个时间片来运行），这个功能依赖于中断技术。 没有人机交互功能，即不了解自己程序的运行情况、也不能控制计算机 引入多道程序设计技术失去了顺序性、封闭性	流畅操作体验； 实时系统的“响应性”是面向特定硬件/任务的确定性时限保障。 主要用于交互式作业而非批处理作业！ 分时操作系统+优先级+非抢占式=既可以改善响应时间、也避免了某些进程由于优先级迟迟得不到运行、	软实时系统：订票系统（可以允许有一点点延迟） 交互性不如分时性操作系统，因为实时操作系统可能因为优先级的问题导致某个操作延迟了执行，流畅度不太行，但是分时操作系统几乎都是同等级使用时间片，流畅度一样例如	机同等地位，分布性、并行性合力完成同一个任务

		由于需要中断，所以相对于单道批处理系统是开销更大的 CPU、IO 设备利用率都提高了	分时操作系统要求的是优秀的响应时间，而不是优秀的处理完成的时间。实时操作系统需要优秀的处理时间。	LINUX 的鼠标，无论什么时候，响应速度都是那么快，不会因为有更高优先级的事情，导致鼠标延迟反应。**实时系统要求的是处理时间，所以需要用抢占式的优先级高者优先！**	

2018 年考察的多任务操作系统可等同于具有交互功能的多道批处理操作系统。

同时自己也混淆了多道批处理操作系统，它的技术本质与硬件无关，无论是单核 CPU 和多核 CPU 都可以是多道批处理操作系统，所以也就拥有了并行性和并发性，同时多个任务必须互斥的访问共享资源。

同时多道批处理操作系统不依赖于虚拟存储管理技术，早期的多道批处理操作系统直接就把需要执行的所有进程的数据全都引入内存了。

操作系统的基本类型：批处理、分时和实时

一些新的理解：
多道批处理系统没有时间片，只是依靠自然断点来实现不同的作业的切换，自然断点指的就是 CPU 执行过程转为 IO 过程把 CPU 让出来了，下一个程序就可以执行了，这个执行可能会长达几个小时，所以没有分时系统那样交互的功能；

分时系统主要用于交互型作业；

通过加入优先级，使得交互的速度更快；但是不能引入抢夺式算法、否则导致一些程序迟迟不能执行

多任务操作系统跟现代的操作系统一样，把一个程序分成多个任务，可以交给不同的处理机并行执行，可以把任务交给同一个处理机来并发执行

以往的旧的多道批处理操作系统是把所有的进程数据全部调入主存，所以不需要虚拟存储管理；现代的可以利用虚拟存储系统；
YYK
2026/10/06 17:56

(三)程序运行环境

1.CPU 运行模式内核模式（管态）、用户模式（目态）

特权指令：IO 指令、开关中断指令、内存清零、修改程序状态寄存器 PSW、存取用于内存保护的寄存器（例如页表基址寄存器）、由内核态转为用户态也为特权指令（但是，用户态转为内核态其实是用户态触发了一个系统调用（执行访管指令或叫陷入指令或者叫访管中断，它属于内中断！！！），然后 CPU 内部硬件自动转换到内核态，所以是由硬件的一个小开关完成 CPU 模式的转换）

非特权指令：仅限于访问用户的地址空间、比如一些存数指令（存到内存里面）、寄存器清零指令（这个寄存器指的是通用寄存器）、程序驱动程序必须在内核态执行！（涉及到对硬件资源的控制）

稍微靠上一点点	进程管理	但这都属于操作系统
	存储器管理	
	设备管理	
最底层	时钟管理（计时+时间片轮转）	
	中断机制（一小部分属于内核是指中断处理的通用框架（保存现场/调度）属于内核核心，代码极简。）	
	设别驱动	

	对系统控制的数据结构处理（PCB\TCB\消息队列\内存分配表等）	

2.中断和异常的处理 （判断中断方式可以采用硬件识别法）

1.定义

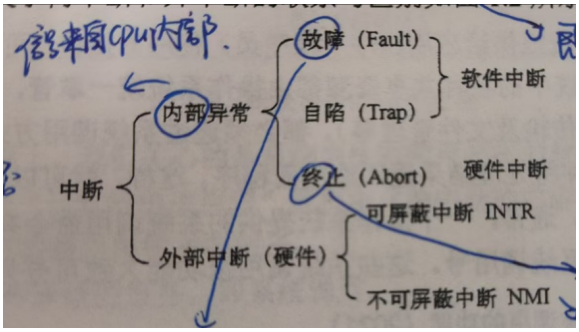
408 中的中断都是外中断，由于外部导致的中断，与指令本身无关，只发生在指令的中断周期。

（例如时钟中断 CPU 时间片到）

异常也叫内中断，由于执行指令时导致的，例如（例如操作码非法、除以 0、缺页、地址溢出、运算溢出、专门的陷入指令（例如应用程序请求的用户态转为内核态）），异常是不能被屏蔽的

处理异常，运行用户态的 CPU 需要进入到内核态，这个过程是硬件自动完成的
YYK
2026/10/07 16:22

2.中断的分类



Interrupt Request 普通的中断（也是可屏蔽）

Non-Maskable Interrupt 不可屏蔽中断，不可掩盖中断，不可 mask 的中断
YYK
2026/10/07 16:30

故障就是指令执行过程中跟软件有关的错误。

自陷是程序员自己设计好的“异常”，例如系统调用、条件陷阱指令等，应用程序用户态和操作系统内核态的转换也称为软中断。

终止一般就是内核程序无法修复的错误，强制退出程序、或者控制器出错、存储器校验错，一般都是硬件出了问题。

可屏蔽中断是通过 INTR 线传送到 CPU 的中断请求。

不可屏蔽中断比如断电

3.中断和异常的处理过程

（1）在该指令 i 执行过程中发现异常，执行异常处理程序，处理异常后返回第 i 条指令。

在该指令 i 执行完成后发现中断，执行中断处理程序处理异常后返回第 i+1 条指令。

YYK
2026/10/22 20:04

另外：如果发现异常和中断事件时致命错误，则终止该程序。异常和中断处理程序由操作系统完成。

（2）响应中断就是在执行周期结束（一条指令执行结束），而中断处理程序是在中断周期结束

4.中断处理和子程序调用的区别

①中断处理程序与当前执行的程序无关，而子程序与主程序都属于同一个程序的一部分，属于主从关系。

②中断都是随机的、但子程序调用是通过 CALL 指令实现的，程序员提前设计好的

③调用子程序完全就是软件处理过程，但是中断处理程序必须要结合硬件实现（寄存器、电路等）

④中断处理程序的入口地址是硬件向量法产生向量地址、再有向量地址找到入口，但是子程序的地址是 CALL 指令的地址码给出的。

⑤两者都需要保护 PC 计数器、通用数据寄存器、通用地址寄存器的内容，中断程序是由中断隐指令帮忙保存，而调用子程序是 CALL 指令完成，压入栈中了。子程序不改变程序的状态，所以 PSW 的值不需要压入堆栈，但是系统调用时是需要压入堆栈的。

⑥响应中断时，会有处理中断的优先级，而调用子程序没有这种操作。

⑦系统调用的被调用过程一定是在内核态、子程序调用的过程可能是用户态也可能是内核态（比如执行中断处理程序时，有需要调用一个小的子程序返回一定的参数，这很正常）

中断处理比系统调用更重要！！！！！

中断的流程

① CPU 检测到中断信号然后进行中断判优	
②检查 EFLAGS 寄存器的 IF 标志	目的是检查是否可以响应可屏蔽中断，若可响应才继续执行
③硬件在中断向量表中找到该中断信号对应的中断向量	中断向量保存的是处理该中断信号的处理程序地址，中断向量表本质就是一个数组，该数组由操作系统初始化 中断向量基址寄存器+中断信号偏移量=中断向量的地址
④触发了一种特权级校验	如果当前是用户态，则允许特权转化成内核态（修改某个特权寄存器的值即可），并且切换了栈指针，即从用户栈切换到内核栈！！！

⑤保护上下文（此时已经进入内核态）	硬件压入内核栈区：硬件保存断点（包括 PC 和 PSW） 软件（操作系统）压入内核栈：保存中断屏蔽字（一组二进制编码，每一位对应一种中断类型，关闭该类型的中断，防止同类型优先级的中断嵌套，类型中断而非全局中断！！！保存下来方便恢复上下文环境，恢复到前文的允许中断情况）、通用数据寄存器和通用地址寄存器
⑥执行中断处理程序	
⑦恢复上下文	从内核中断栈弹出通用寄存器、用户栈指针等软件手动保存的上下文； 执行 IRET 指令，硬件自动弹出 EIP（x86 系统中的下一条指令地址，也叫 PC）、CS（用户态代码段）、EFLAGS（含中断屏蔽位），恢复用户态执行流； CPU 继续执行用户程序中被中断的下一条指令

执行系统调用之前，需要执行命令解释程序。是在用户态输入我想执行的命令（在 cmd 这个程序里），相当于命令解释程序 (Shell) 理解了你输入的 clear 是想运行一个叫 clear 的程序。它找到这个程序在哪，然后让操作系统执行这个 clear 程序。程序 clear 自己内部知道怎么调用操作系统的接口（或发送特定指令）来清屏。所以这

就是命令解释程序。程序员在代码中直接调用系统调用（如 read）时，根本不会涉及外部的命令解释程序（Shell）。

4.系统调用

系统调用就是操作系统给程序员调用资源的一个接口，凡是涉及到程序员需要调用相应资源，都需要通过系统调用实现，记得是资源资源资源。记得一定要从程序员出发的操作！！！例如页面置换根本就不是程序员出发的，它根本不知道这个事情，反而创建新进程是程序员触发的系统调用（2021 真题）

系统调用的一些功能

启动和关闭设备（这属于资源吧）	牵涉到用户对资源的调用
文件控制	读写、创建以及删除
进程控制	创建、撤销（用户可以主动关闭 APP 吧？）、阻塞（表面与用户无关，但实际上程序员可以 sleep()系统调用或者 wait 进程）、唤醒（也可以由程序员 kill 唤醒一个进程）

系统调用处理时的 CPU 状态变化

①用户程序将**系统调用号和所需参数**压入用户栈（也叫传递参数）	便于后续使用，通过系统调用号和调用内部的参数来找到调用子程序和执行系统调用程序

②用户程序执行“陷入指令 trap（而设置断点指令属于陷入指令的范畴，所以为用户态）”	陷入指令修改了特权寄存器的值，使得 CPU 由用户态转为内核态
③硬件和操作系统内核程序合力保护现场，压入堆栈。 硬件保存 PC 的值、PSW（设置断点指令和保存 PC 的值操作时间、意义完全不同） 操作系统保存通用数据寄存器和通用地址寄存器（手动）	PC（由中断隐指令硬件保存，2015 真题） PSW 状态寄存器（由中断隐指令硬件保存，便于程序返回继续执行，比如 if（xxxx==0）,如果 PSW 寄存器的数值丢了，就运行混乱了） 通用寄存器数据（保存的一些中间数据结果，由操作系统保存） 通用地址寄存器的值（程序运行时的临时数据、内存地址或函数参数，通用地址寄存器操作系统进入内核后手动保存）
④分析系统调用类型、转入系统调用子程序	本质是查询系统调用表（本质上就是号码编号映射到对应子程序的地址，数组），找到对应入口地址
⑤子程序结束后，恢复现场环境，返回被中断的进程或者新进程	CPU 由内核态转为用户态

在内核态的时候是没有进程在执行的，执行的只有系统调用主程序和子程序。

另外两个会发生用户态到内核态：

用户程序产生了一个错误状态：例如缺页、内存溢出等

用户程序企图执行一条特权指令

补充：王道课后题我做错的地方

（1）在通用操作系统执行程序的时候，不需要向操作系统预定时间：想一想执行一个自己写的 helloworld，还需要自己指明运行时间，就有点鬼畜；让操作系统根据优先级和时间片执行就好了

（2）编译器并不是操作系统提供的，而是人为编写的，把高级代码转化为汇编或者机器代码，即把一行行的高级语言编译为硬件执行层面的逻辑；

（3）输入输出指令，涉及到对外设硬件的操作，自然需要到内核态执行该指令

（4）存数指令和寄存器清零指令可以在用户态下执行：

存数指令：操作的是受控的、虚拟化的内存。硬件和操作系统共同保证了你的访问被限制在自己的地址空间内。

寄存器清零指令：操作的是当前执行流的私有上下文，不影响系统全局状态。

YYK
2026/10/08 16:45

（5）设备驱动程序，涉及到对设备的操作，自然需要在内核态下执行；操作系统初始化程序，在开机时没有任何用户程序，自然是内核态执行的；

（6）时钟中断属于外部中断

（7）一般函数调用的过程中，只需要保存 PC 的值，不需要保存 PSW 的值

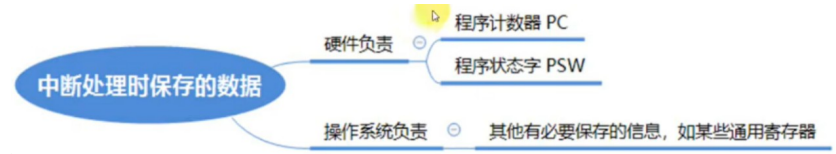
调用者通常不依赖跨函数调用的条件码状态 我可以这么认为，调用子程序之后，PSW 的值改变了也对主函数没有影响

YYK
2026/10/08 16:47

（8）压栈指令，也可能在用户态执行，例如一般函数调用的压入 PC 的值；数据传送指令也可以在用户态执行

（9）关中断指令和开中断指令必须在内核态执行

（10）中断处理需要保存的内容



操作系统还负责保存屏蔽字；
初始化中断向量表；

YYK
2026/10/08 17:00

（11）软中断的定义就是“自愿中断，即自愿进入 trap 指令，用户态转为内核态”

（12）存储保护错的本质就是用户程序执行指令过程中想要强行访问主存，触发了保护，

所以属于异常；

5.程序的链接与装入

6.程序运行时内存映像与地址空间

(四)操作系统结构

分层，模块化，宏内核，微内核，外核

分层就是分了 0、1、2、3.....层，硬件在第 0 层，应用程序在最高一层。

优点是：便于调试和验证，不同层之间不会干扰、易于扩充和维护

缺点是：如何定义各层的界限？层数太多穿层、效率太低如何定义各层之间通信？

模块-接口化就是把操作系统分为进程管理、文件管理、存储器管理等大模块，每个模块内部有小的模块，模块之间需要定义接口

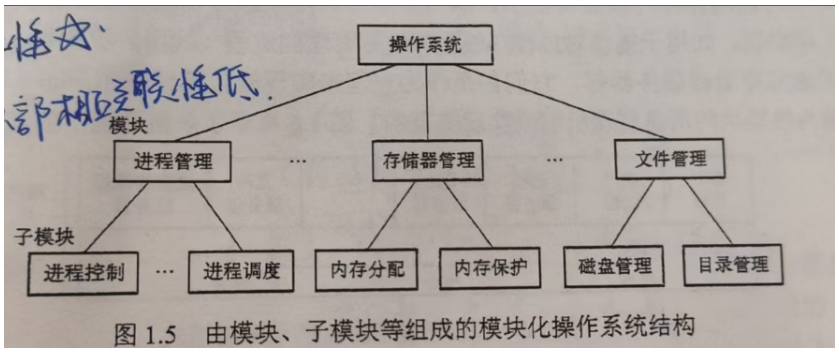


图 1.5　由模块、子模块等组成的模块化操作系统结构

模块大小影响操作系统效率，模块太大，内部太复杂；模块太小，联系太多，系统太乱。

模块独立性影响操作系统效率，内聚性越高，模块内部联系越紧密，模块之间的独立性越好；耦合性越高，模块之间的联系越紧密，独立性越差！

优点是：操作系统的设计偏向于正确的道路了，加速了开发过程

缺点是：接口太难设计

宏内核（单内核、大内核）就是把主要功能模块全都集中在一起

优点是：模块之间沟通顺畅，性能极其强悍

所有模块都在一起，交流特别方便，自然性能强悍

YYK
2026/10/08 17:06

缺点是：随着发展，宏内核臃肿不堪

微内核就是在宏内核基础上把一些非核心功能移到用户空间，目前主流操作系统都采用宏内核 + 微内核的混合内核

基本概念：

①最重要的功能保留在内核，不重要的功能被划分为多个服务程序放在用户态执行，服务程序之间通过微内核进行通信，降低设计的复杂性

②操作系统=微内核+多个服务器，每个服务器负责一大类的功能（例如进程管理、内存管理等），微内核与硬件有关、包含一些基本功能、负责客户与服务器之间的通信。

③可靠性高：如果服务器崩溃了，重启就行，与微内核不干扰。但是在宏内核中某个功能崩溃会导致整个系统崩溃。

微内核的基本功能：

①进程、线程管理（进程切换调度属于微内核；如何对进程进行分类、优先级的确认方式属于策略问题，放入服务器当中）

②低级存储器管理（只与硬件有关的部分，例如地址转换过程。而采用何种页面置换算法、如何进行内存分配与回收应该放在存储器管理服务器中）

③中断和陷入处理（微内核负责识别、服务器负责处理）

微内核的特点：2023 真题

①扩展、灵活

②可靠性和安全性

③可移植性：其他各种服务器的代码与硬件无关，所以只需要针对不同硬件对微内核进行修改

④分布式计算：微内核和服务器之间采用消息传递机制、更好的支持了分布式系统和网络系统

微内核的缺点：

①在内核态和用户态之间切换、开销较大

②宏内核适用于桌面操作系统、微内核适用于可靠性的工业航空航天军事等领域

外核就是给每个用户分配物理上的空间，互不冲突，记录给每个虚拟机的有关资源，只需要保证虚拟机之间不冲突即可。

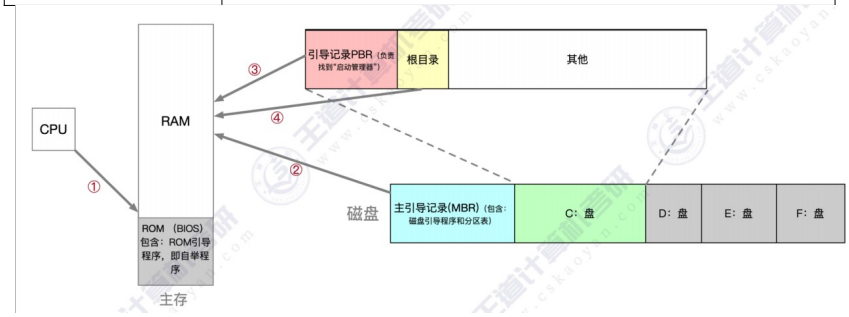
(五)操作系统引导

①激活 CPU，读取固化在 ROM/BIOS 芯片中的自举程序	这时候是把指令寄存器的值设置为自举程序 BIOS 的第一条指令，然后就执行 BIOS
②硬件自检	BIOS 在内存最开始的空间中构建临时的中断向量表（仅包含 BIOS 自带的中断处理程序，因为第三步需要使用到中断机制），然后通电自检（需要用到中断功能），若无问题，则屏幕显示各种硬件信息
③ BIOS 通过 CMOS 设置 / UEFI 固件确定启动设备顺序（如优先从第一个固态硬盘、第二个固态硬盘、U 盘或网络启动）	BIOS 按启动顺序尝试读取设备的 MBR，若读取成功且 MBR 末尾有有效标志（0xAA55），则跳转至 MBR 代码执行。
④加载主引导记录 MBR	MBR 包含引导程序、活动分区表、MBR 结束标志；
⑤扫描硬盘分区表，并加载硬盘活动分区	MBR 中的引导代码扫描分区表找到装有操作系统活动分区条目后，读取该分区的第一个扇区的引导记录（PBR） 到内存
⑥加载分区引导记录 PBR	PBR 包含分区特定的引导代码：识别并加载该分区的文件系统驱动（如 FAT32/NTFS 驱动）； 文件系统元数据：如簇大小、根目录位置等； 启动管理器位置指针：指向该分区内的启动管理器文件

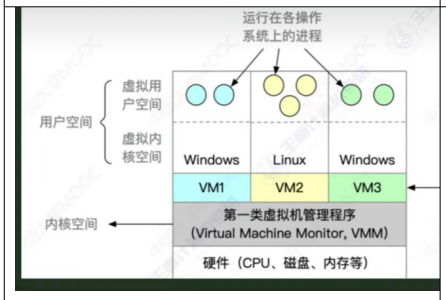
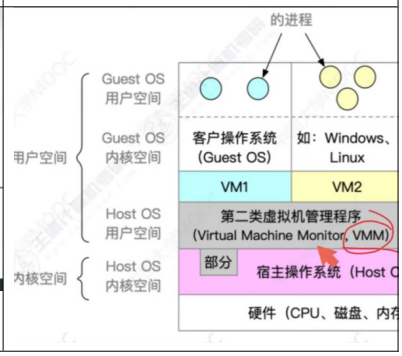
⑦加载启动管理器	按照分区引导记录扫描活动分区，并找到启动管理器
⑧加载操作系统	将操作系统初始化程序加载到内存。 替换 BIOS 的实模式中断向量，建立操作系统管理的中断处理机制（如页故障、系统调用、外设中断） 启用分页机制；建立内核页表；初始化虚拟内存系统。创建内核线程（如 kthreadd）；初始化进程调度器（；启动 init 进程（PID=1，用户空间的第一个进程）。 启动管理器加载内核后，内核初始化文件系统时，内核通过文件系统驱动（如 ext4/NTFS 驱动）读取 inode 区元数据（如 inode 数量、块组分配表）。 位示图，根文件系统挂载完成后，内存管理与文件系统协同初始化时，内核读取位示图（记录磁盘块使用状态），用于后续文件读写时的块分配

建立中断机制、启用分页（基础内存环境准备）；
初始化进程调度、创建内核线程（进程管理准备）；
（磁盘交互准备）。

YYK
2026/07/18 17:07



(六)虚拟机

第一类虚拟机	第二类虚拟机
通过 VMM 虚拟机管理程序对各类操作系统进行管理，VMM 直接运行在硬件上面，也叫裸金属架构，能够直接控制和分配硬件资源，资源分配方式类似于外核	在宿主操作系统上下载 VMWare，分配的是虚拟内存
性能更好	性能差，需要又有宿主操作系统中介
可支持虚拟机的数量，更多	少，因为宿主操作系统也需要资源
可迁移性差	导出 iso 文件就可以迁移
运行模式，运行在最高阶层，可以执行最高特权的指令	部分运行在用户态，部分运行在内核态（注意看下面的图片，部分 VMM 越界到宿主操作系统），有些特权指令会被 VMM 截获
	

两类虚拟机管理程序（VMM）的对比

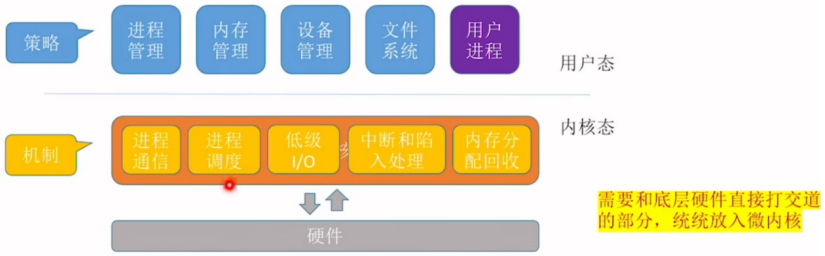
	第一类VMM	第二类VMM
对物理资源的控制权	直接运行在硬件之上，能直接控制和分配物理资源	运行在Host OS之上，依赖于Host OS为其分配物理资源
资源分配方式	在安装Guest OS时，VMM要在原本的硬盘上自行分配存储空间，类似于"外核"的分配方式，分配未经抽象的物理硬件	GuestOS 拥有自己的虚拟磁盘，该盘实际上是 Host OS 文件系统中的一个大文件。GuestOS分配到的内存是虚拟内存
性能	性能更好	性能更差，需要HostOS作为"中介"
可支持的虚拟机数量	更多，不需要和 Host OS 竞争资源，相同的硬件资源可以支持更多的虚拟机	更少，Host OS 本身需要使用物理资源，Host OS 上运行的其他进程也需要物理资源
虚拟机的可迁移性	更差	更好，只需导出虚拟机镜像文件即可迁移到另一台 HostOS 上，商业化应用更广泛
运行模式	第一类VMM运行在最高特权级（Ring 0），可以执行最高特权的指令。	第二类VMM部分运行在用户态、部分运行在内核态。GuestOS 发出的系统调用会被 VMM 截获，并转化为 VMM 对 HostOS 的系统调用

补充：错误的地方

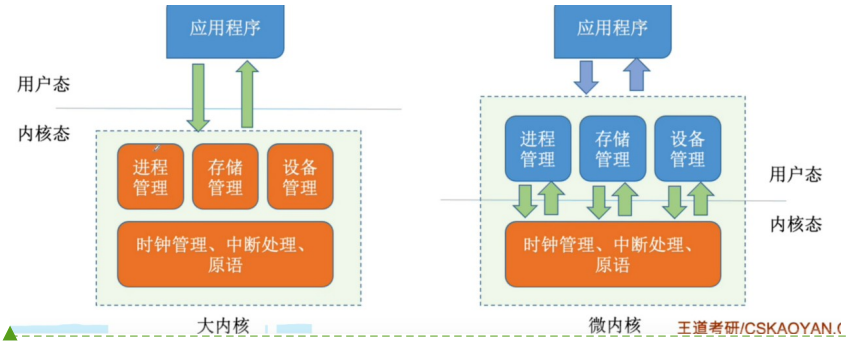
（1）分层式架构结构更清晰、看到分层式一定就是便于调试和单向依赖

（2）模块化更难调试和验证

（3）微内核适合放入那些模块



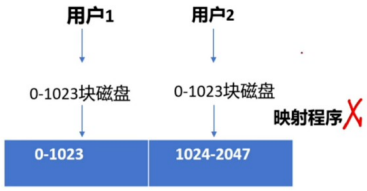
（4）微内核和宏内核之间的区别



微内核比单一内核更稳定、微内核添加系统服务功能不需要对内核态的成分进行修改
YYK
2026/10/09 17:44

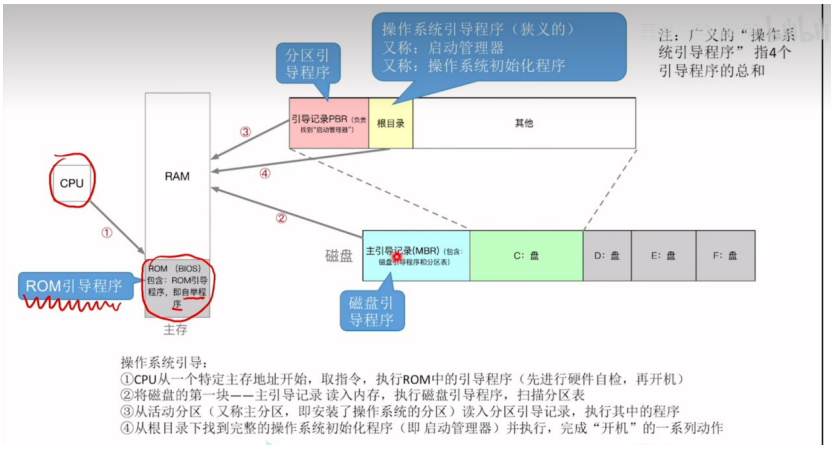
（5）外核的功能

外核：为虚拟机分配资源，并检查这些资源使用的安全性，以确保没有机器会使用他人的资源。



外核替代了映射程序的作用，主要是分配资源，二不负责进程调度
YYK
2026/10/09 17:46

（6）开机过程



区分操作系统引导程序和 ROM 引导程序的区别，操作系统引导程序
YYK
2026/10/09 17:48

计算机的启动过程：
①首先给 CPU 通电，得到 BIOS 的首地址 FFFF0H ，然后跳转 JMP 到这里
②硬件自检
③执行 ROM 引导程序
④读入磁盘引导程序
⑤读入引导程序 PBR
⑥读入该分区盘的根目录（含操作系统初始化程序）
YYK
2026/10/09 17:51

二、进程管理

(一)进程与线程

1.进程与线程的基本概念

（1）进程的概念：

①进程是一个正在执行程序的实例

②进程是一个程序的代码和数据从磁盘加载到内存，在 CPU 上的执行过程

③进程是一个具有独立功能的程序在一个数据集合上的运行过程。

④进程是系统进行资源（包括 CPU、存储器和其他设备服务于某个进程的时间）分配和调度的一个独立、基本单位！

⑤进程映像=程序段+代码段+PCB

⑥一个进程在其生命周期内不止可以运行一个程序，可以运行多个程序，这个程序不止限于微信等大程序，连脚本、共享库这些都叫程序。但是只能顺序的执行多个程序，不能同时执行多个程序

（2）进程的特征：

动态性、并发性、独立性、异步性

（3）进程的组成：

最重要的就是 PCB，它是进程存在的唯一标志，而不是 PID！

表 2.1 PCB 通常包含的内容

进程描述信息	进程控制和管理信息	资源分配清单	处理机相关信息
进程标识符（PID）	进程当前状态	代码段指针	通用寄存器值
用户标识符（UID）	进程优先级	数据段指针	地址寄存器值
	代码运行入口地址	堆栈段指针	控制寄存器值
	程序的外存地址	文件描述符	标志寄存器值
	进入内存时间	键盘	状态字
	CPU 占用时间	鼠标	
	信号量使用		

四大标志：

① PCB 的身份证号 PID

②运行情况

a.调度上处理机，要查询进程当前状态和优先级

b.进入内存时间用于某个 CPU 调度算法的参数

c.CPU 占用时间可以用于时间片切换

d.代码运行入口地址可以用于找到代码的第一行，发生中断保存上下文时，会把断点地址保存在这里面，方便下一次运行时找到入口

e.程序的外存地址是“找到程序并加载到内存”的依据。

③资源分配情况

a.代码段指针是指向内存代码的起始地址，并不会随着代码运行发生改变

b.数据段指针和堆栈段指针同理（指针只用于界定范围，并不用于地址计算）

C.文件描述符保存打开的文件的 fd，可以同时保存多个 fd

④上下文情况

a.通用寄存器值：保存进程运行时的数据和中间结果等，恢复运行时可继续使用。

b.地址寄存器值：存放内存地址信息，如基址、变址寄存器等，专门用于内存访问和地址计算，与上面数据段指针和堆栈段指针的作用要区分开来。

C.控制寄存器值：记录控制相关状态，如模式位（用户态/内核态）等，决定指令执行权限。

D.标志寄存器值：存储运算结果标志（如溢出、零标志等）。

程序段：多个进程可共用程序段

数据段：原始数据或者进程产生的加工数据

例如只读数据段、可读写数据段

（4）PCB 的组织方式

链表链接，按照不同状态甚至不同原因阻塞的，分开链接

索引表，索引指针指向 PCB

（5）线程的基本概念

①引入线程是为了减少程序在并发执行时所付出的时空开销

解释一下：假设微信 exe 是一个程序文件，打开微信就是一个巨大的进程，而发送线程、获取信息线程、监听网络线程、检测鼠标点击这都是一个个线程，他们在宏观上是并行的，微观上是串行的，所以线程是独立调度的基本单位，它是一个基本的 CPU 执行单元，程序执行流的最小单元。

②线程不拥有系统资源，只拥有一些基本的必要资源

③一个线程可以创建和撤销另一个线程

（6）线程与进程的比较

①进程调度开销大、同进程内的线程调度只需要保存和恢复线程的栈指针、寄存器值等少量上下文，无需切换页表和内存映射。

②线程的并发性会提高系统资源利用率和系统吞吐量（本质都是通过轻量切换过程，减少了浪费的等待时间、使得 CPU 利用更充分、更多的任务被执行完成）

③线程不拥有系统资源，同一进程的所有线程拥有相同的地址空间

④同一进程的各个线程是由于相互合作或者提高并发性才被创建的

⑤引入线程，则进程可以被同时运行在多个 CPU 上

线程的属性

①线程=PID+TCB（记录寄存器、栈等现场信息）

②同一进程中的线程共享该进程的资源

③同一个服务程序被不同的用户（这里的用户指的是需求，被不同的功能需求）调用时，操作系统将他们创建成不同的线程：本质是服务程序通过线程的轻量并发能力，高效、低成本地同时处理多个用户的独立请求，这是服务器高并发设计的常见策略。

例如微信内部同时申请上传一张图片、下载一个文件，选择的是创建两个线程，而不是重新打开两个微信进程。

2.进程/线程的状态与转换

创建态：

先创建 PCB、写入 PCB 中的信息，然后分配资源（如果所需资源不够，则停留在创建态），分配资源结束后才插入到就绪队列。

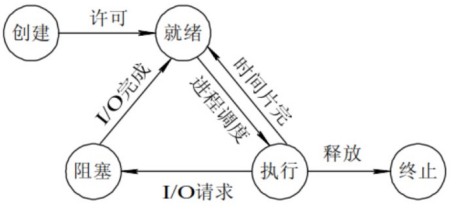
阻塞态：

这个就绪队列的数据结构可以有多种实现方式：
普通队列、多级队列、链表+哈希表等
YYK
2026/07/19 13:27

进程因为要等待 IO 完成或者某个资源，而进入阻塞态。中断时，进程是进入了阻塞态。读磁盘时是一定被阻塞的，因为要让 CPU 充分利用，把 CPU 让出去给其他人，不能迷糊，不要认为磁盘是共享设备，可以并发读取，根本原因在于你用着磁盘，CPU 就空闲了！

终止态：

执行了 exit 系统调用，操作系统结束该进程并且回收资源（包括 PCB、资源、CPU）。



其中就绪态到运行态：可剥夺操作系统，有更高优先级的进程进入，就会被剥夺 CPU

阻塞态→就绪态：当 IO 结束或者中断结束，由中断处理程序将进程的状态调整为就绪态

运行态→阻塞态是主动行为，阻塞态→就绪态是被动行为

进程因状态转换（如完成打印任务，进入下一执行阶段）而主动放弃资源的结果，**请注意是进程状态转换导致释放打印机，而不是因为释放了打印机而导致自身状态转换，这个因果关系一定要弄清楚！**

阻塞进程：用户态到内核态，在内核态将该进程阻塞，然后还要处理其他事情，再返回用户态调度新的进程
唤醒进程：先从用户态到内核态，将该进程的状态调整为就绪态，然后处理其他事情，再返回用户态继续执行原进程。
执行 CPU 调度，之后还要进行上下文切换，然后还要检查各类事情，再返回用户态。
唯独：执行系统调用完成之后立马从内核态到用户态！！！！！（2023 真题）

3.线程的实现

内核支持的线程，线程库支持的线程

线程的实现方式：

（1）用户级线程

“从用户视角能看到的线程”，切换撤销等都在用户空间进行，内核意识不到线

程的存在，应用程序控制的线程

②调度仍然以进程为基本单位，因为操作系统根本看不到线程，所以对于线程来说是调度不公平的，假设有两个进程，第一个进程有 100 个线程、第二个进程有 1 个线程，那么第一个进程中的线程所获得 CPU 的时间是第二个进程中的线程的 0.01 倍。

③用户级线程的切换不需要转到内核空间、开销小、用户级线程与操作系统无关即可以运行在任何操作系统上面，对线程管理的代码是应用程序而非操作系统。

④进程内部的某个线程被阻塞，内部其他线程也被阻塞；进程每次只能获得一个 CPU，所以导致内部的线程无法并行执行。

⑤线程库是为程序员提供创建和管理线程的 API 接口

⑥在线程同步时，由于线程是由程序控制的，在某些线程需要在其他线程执行完之后才能执行，这过程只是在用户级线程之间切换。但是线程的系统调用、IO 处理、异常处理这都涉及到内核

（2）内核级线程

内核级线程是在操作系统的帮助下实现的；内核感知到线程的存在。

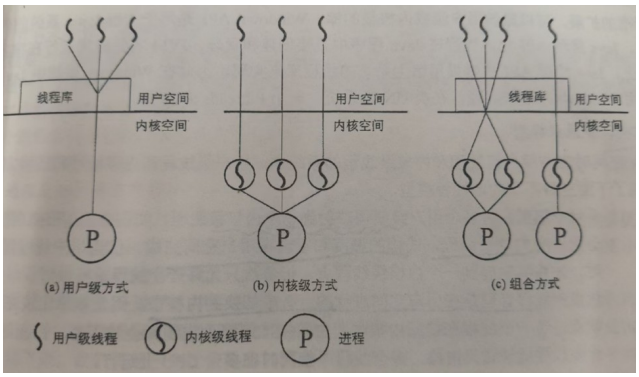
①优点是：同一个进程内的不同线程可以并行执行（即一个进程可以同时获得多个 CPU）；内核级线程本身切换也快、开销小；

②同一进程中的频繁线程切换需要从用户态转到内核态，开销大（但用户级线程是在用户空间内发生转换的，就没有这样的缺点）；因为此时线程管理是由操作系统完成的

（3）组合方式

为了解决用户级线程“进程内部一个线程阻塞导致其他线程也被阻塞”&内核级线程“同一进程内部线程的切换需要用户态转到内核态”这两大缺点，引进来了组合方式！

内核级线程和用户级线程的关系要梳理清楚：在组合方式中，内核级线程和用户级线程并不一定代表着同一种任务，通常情况下把用户级线程当做具体的任务了，而把内核级线程抽象成一个调度的单位，例如内核级线程 A\B 共同绑定着用户级的线程任务 CDEF（不是固定 C 归 A 管、D 归 A 管，E 归 B 管、F 归 B 管，而是线程库会根据情况动态分配），这样 CDEF 内部可以轻松在用户级别切换，而通过 AB（在多 CPU 系统中）也可以实现 CDEF 的并行执行



多线程模型：

（1）多对一模型

即内核区的一个核心线程映射到用户区的多个核心线程。线程的调度和管理在用户空间完成，只有在该用户线程需要访问内核区时，才将该用户线程映射到内核线程。

优点和缺点同“用户级线程”

（2）一对一模型

每个用户级线程映射到一个内核级线程，每个进程拥有相同数量的用户级线程和内核级线程，每创建一个用户级线程，就需要创建一个内核级线程，优缺点同“内核级线程”

（3）多对多模型

同组合线程

并发性较好的多线程模型：一对一模型和多对多模型，充分利用多核 CPU（真正的并行执行）

4.进程与线程的组织与控制

进程控制：（主要包含进程创建、进程终止、进程进程阻塞和唤醒）

进程控制的程序段成为原语，是操作系统内核中实现特定基本功能的、不可分割的程序片段（它只是一些程序代码，属于内核的程序代码，由于需要经常对进程进行控制，所以这些**原语常驻内存**）、原语执行过程中不允许被中断。

（1）进程创建

父进程与子进程：子进程继承父进程的资源、父进程要回收子进程的资源。但是父进程和子进程并不使用同一片地址空间，子进程只是把父进程的内容复制一份，重新开辟了一片新的空间（2020 真题）

孤儿进程与僵尸进程：当父进程提前终止，子进程就成为孤儿进程，通常会被 init 进程收养，所以父进程终止也可能不会终止子进程（2024 真题）；而子进程先终止，父进程未及时回收其资源，子进程就会变成僵尸进程。

导致进程创建的操作：

终端用户登录系统	每个用户登录后都是一个独立的交互主体，需要独立的资源（如内存空间、环境变量）和执行流，进程是系统管理这种“独立性”的基本单位。相当于开机登录系统就是一个巨大的进程
作业调度	批处理系统中，用户提交的“作业”（如一个包含多个程序的计算任务）需要被拆分为可执行的进程，由调度器按规则分配 CPU 资源，因此会触发进程创建。
系统提供服务	打印文件（创建新进程是为了“并发处理多个请求”）、访问数据库（客户端进程发起连接请求，服务器会创建一个新进程 / 线程专门处理该连接） 而分配设备并不会造成创建新进程！
应用程序请求	无需多言

申请 PCB（PCB 数量有限，若不足则创建失败）→分配资源（若资源不足则停留在创建态）→初始化 PCB（标志信息、CPU 状态信息、控制信息、进程优先级）到这一步才算创建完成（2021 真题）→插入就绪队列

（2）进程终止

正常结束、异常结束（执行过程中出错）、外界干预（操作员主动关闭、父进程被请求子进程关闭、父进程被撤销）

进程终止时的操作：

①根据 PID 进程号找到 PCB 集合链表中找到 PCB，确认其状态（运行态、就绪态、阻塞态等）和相关资源信息

②若进程存在子进程，必须先终止所有子进程（或转移其“父进程”身份给其他进程，如 init 进程），但并不是所有系统都要求子进程随着父进程一起死亡

③如果该进程还持有 CPU，立即终止执行，将 CPU 给其他进程

④释放资源：2024 真题

用户空间内存（如堆、栈）、页表等

若进程使用了共享内存，需检查引用计数，仅当计数为 0 时才彻底释放共享内存。

关闭进程打开的所有文件描述符（包括普通文件、管道、设备文件等），若文件引用计数为 0 则释放文件对应的内核数据结构（如 inode）。

- 回收进程持有的信号量（同步互斥临界资源等）、消息队列（若进程是消息队列的 “创建者”，需检查队列的引用计数（当前关联的进程数）：
  - 若计数为 0，删除消息队列，释放其占用的内核存储空间（包括消息链表、队列描述符等）。
  - 若计数 >0，仅解除当前进程与队列的关联，队列继续存在。

）、定时器以及如时间片、寄存器上下文等。

回收该进程所持有的设备

⑤将 PCB 从各个阻塞、就绪队列中删除

⑥回收 PCB

（3）进程阻塞和唤醒

等待队列的是意思是等待资源，而不是等待 CPU，所以等待队列=阻塞队列

阻塞过程非常简单：

根据 PID 找到 PCB→检查状态且保护上下文→让出 CPU、插入阻塞（等待）队列

唤醒过程：

根据 PID 找到 PCB→从阻塞队列移出，设置为就绪态→插入就绪队列

阻塞原语和唤醒原语必须成对出现，即如果该进程主动调用阻塞原语，另外还需要插入一条唤醒原语到另一个进程，以便后续唤醒，而不是自身唤醒自身。

线程控制：（主要包含进程创建、进程终止、进程进程阻塞和唤醒）

①线程 TCB 里面几乎不记录线程的所拥有资源，因为几乎都是进程拥有资源

②线程 TCB 保存 CPU 上下文（寄存器、栈指针、与进程共享栈空间但有独立栈帧）

③线程创建时不分配新内存/资源

5.进程间通信共享内存，消息传递，管道，信号

进程通信（进程之间的信息交换）：

低级通信 PV 操作

高级通信：共享存储、消息传递、管道、信号

（1）共享存储

①通过共享存储区来实现信息交换；通过系统调用让操作系统分配内存区并返回标识符

②通过共享数据结构（例如数组），可以最低级的实现信息交换

③通过对映射文件的共享，将磁盘文件映射到多个进程的虚拟地址空间”来实现内存共享。

其**重点在于共享的载体是文件**

在初始化时，需要一次状态切换，后续通信无需进行状态切换。

（2）消息传递

操作系统提供消息传递方法，便于两个进程之间直接进行通信，以格式化的消息 Message 作为单位，通过发送消息原语和接受消息原语进行数据交换，隐藏了通信实现细节，通信过程对用户透明，应用最广。消息队列是内核管理的资源，由内核完成数据复制和队列管理 ，所以要往消息队列里面写数据，需要用户态转到内核态

发送方：用户态（准备好数据，进行系统调用后进入内核态）→内核态（进入内核态后，将用户区的数据复制到内核态）→用户态（复制完成后，返回用户态）。

接收方：用户态→内核态（接收请求）→用户态（接收完成）

也可以广泛应用于微内核与服务器、多 CPU 系统、分布式系统、计算机网络

①直接通信：指明对方进程的 PID，如进程重启，通信会中断，这种方式灵活性比较差

发送原语：`Send(P, Message)`（向进程 P 发送消息）；

接收原语：`Receive(Q, Message)`（接收来自进程 Q 的消息）。

②间接通信：不需要指明 PID，只需指明 BOX 信箱号码，支持多个进程共用一个信箱

发送原语：`Send(Box, Message)`（向信箱 Box 发送消息）；

接收原语：`Receive(Box, Message)`（从信箱 Box 接收消息）

优点是：简单、安全、灵活、对用户透明，无需用户手动处理同步（内核会管理队列的空满状态，自动阻塞 / 唤醒进程）

缺点是：效率较低，切换开销大、复制占用内存开销也大，消息传递需经过内核中转（涉及内存复制、进程切换），速度慢于共享存储（共享存储直接读写内存）。

若需高频、大数据量通信，优先用共享内存（追求效率，共享内存效率更高）。并且共享内存仅适用于同一台机器上的不同进程，需要手动处理同步互斥机制、编程麻烦

若需跨进程解耦、简单安全的通信（如服务间指令传递、分布式系统），优先用消息传递（追求易用性和灵活性）。而消息传递可以在网络环境中适用

（3）管道 pipe 文件（也叫无名管道，用于血缘关系的进程之间通信，一种特殊的共享文件，一段特殊的内核缓冲区）通信

一旦读出，数据就消失了，所以一个管道可以有多个写进程，但是只能有一个读进程
YYK
2026/10/11 17:30

①管道先进先出

②对管道必须互斥、同步机制，还要确认对方存在性！

③半双工通信，与消息传递不一样，消息传递是全双工通信，所以管道需要两个来实现全双工

④匿名管道仅支持有亲缘关系的进程（如父子、兄弟），因管道创建时无标识，需通过 fork 继承文件描述符；命名管道（FIFO）可支持无亲缘关系的进程（通过路径名标识）。

⑤内核自动管理管道的空满状态（写端满则阻塞，读端空则阻塞），无需用户手动处理同步；这一点与消息队列类似。

⑥管道缓冲区是内核空间的资源，无论是写数据到管道还是读数据，都必须通过系统调用

进入内核态，由内核完成数据复制和缓冲区管理

⑦必然涉及**多次状态切换和两次数据复制**。其效率低于共享存储（无数据复制、少状态切换），但高于信号（仅传递整数），适合传递流式数据

⑧管道仅仅可以由创建进程访问，当父进程创建一个管道后，子进程会继承管道，管道也是子进程与父进程通信的方式，

⑨管道数据一旦被读取，就被释放该数据空间了。

⑩管道本质是占用内存的缓冲区，所以需要受到内存空间限制（2014 真题）

11.一个管道可以由多个读进程和写进程对其进行操作，但是同时读和同时写是不被允许的，所以需要用互斥锁加以限制！

最重要的是过程，还是通过**系统调用**来实现对管道的读写：

写端：用户态→内核态（调用 write）→用户态（写完成）。

读端：用户态→内核态（调用 read）→用户态（读完成）。

关于管道被阻塞：

若管道未满（缓冲区有剩余空间），则将 write_buf 中的数据复制到内核管道缓冲区

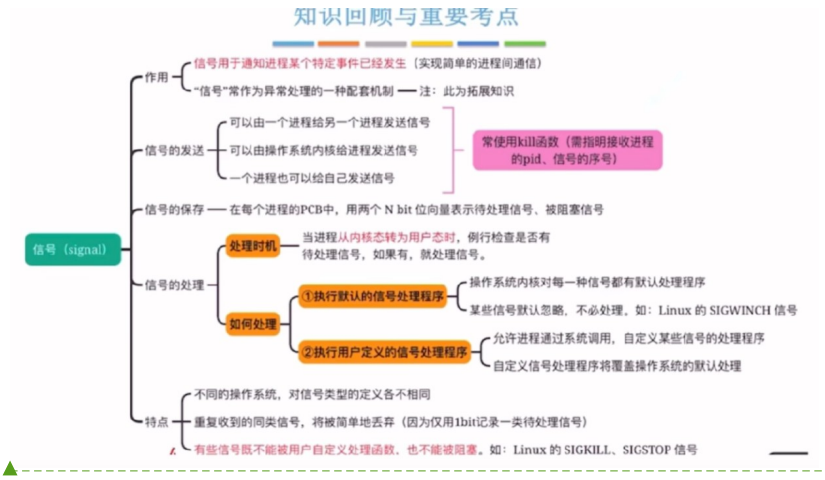
若管道已满（缓冲区无空间），内核会阻塞写进程 A（或返回错误，取决于管道是否设置为非阻塞），直到读端读取数据后管道有空闲空间。

若管道为空，则读进程被阻塞。

（4）信号

信号是一种异步通信机制（发送方 “只管发”，接收方 “被动接”，双方无需协调时序，也不存在相互等待，

为了应对 “突发事件”），传递的不是大量数据，但是事件标识（整数编号），通信过程同样涉及用户态与内核态的切换，但数据流转路径和开销与管道有显著差异。



只允许用户都某些信号自定义处理程序

YYK
2026/10/11 17:02

进程 A：发送信号方

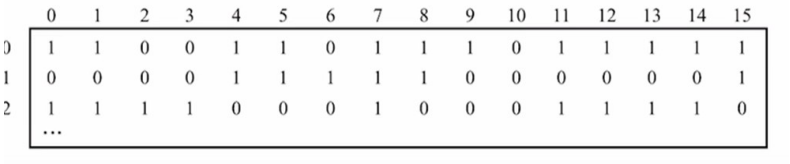
进程 A 在用户态决定发送信号，通过系统调用（如 kill(pid, sig)）指定目标进程 B 的 PID 和信号编号（如 SIGINT 表示中断，编号 2），请求向 B 发送信号。此时信号的“内容”仅为一个整数（信号编号），无需复杂数据结构。

进程 A 执行 kill()系统调用，进入内核态。

信号的发送和管理是内核职责（需验证权限、查找目标进程、维护信号状态），用户态进程无法直接操作其他进程的信号机制。

内核检查发送权限（如进程 A 是否有权向进程 B 发信号）、信号的有效性（是否为系统定义的信号编号）。

若验证通过，内核在进程 B 的“信号 pending 队列”（PCB 中的数据结构，记录了位图，即每种信号的标志位都为 0，当出现该信号时，标记为 1）中记录该信号，标记为“待处理”。



	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	1	0	0	1	1	0	1	1	1	0	1	1	1	1	1
1	0	0	0	0	1	1	1	1	1	0	0	0	0	0	0	1
2	1	1	1	1	0	0	0	1	0	0	0	1	1	1	1	0
	…															

进程 B：信号接收方

进程 B 在执行过程中，每次从内核态切换回用户态前（如系统调用返回、中断处理结束后），内核会检查 B 的 pending 队列。

若存在待处理信号，内核暂停 B 的正常执行流程，触发用户态→内核态切换（进入信号处理上下文）。

内核查找进程 B 注册的信号处理函数、修改进程 B 的用户态栈，设置返回地址为信号处理函数的入口，然后从内核态切换回用户态，执行该处理函数。

进程 B 在用户态执行忽略、默认信号处理程序或者进程自定义处理函数（这是三种情况都可以）（如释放资源、记录日志、退出进程等）。

处理完成后，通过特殊的系统调用（如 sigreturn()）再次进入内核态，恢复进程 B 的正常执行上下文。

内核清理 pending 队列中的该信号，从内核态切换回用户态，进程 B 继续执行被中断前的逻辑

发送方：用户态→内核态（kill()调用）→用户态（发送完成）。

接收方：用户态→内核态（检查信号）→用户态（执行处理函数）→内核态（sigreturn()恢复原进程上下文）→用户态（正常执行）。

同时不止进程可以给另一个进程发送信号，内核本身也可以给进程发送信号，也可以进程

给自己发信号

但也不是系统中的所有进程都可以发送信号，只有核心和超级用户才可以

几个Linux支持的典型信号如下表：

信号	说明	默认操作
SIGCHLD	子进程终止	忽略
SIGFPE	算术异常	终止且进行内存转储
SIGILL	进程试图执行非法指令	终止且进行内存转储
SIGINT	用户产生中断符(Ctrl-C)	终止

当信号产生时，内核将进程taskstruct中的信号相应标志位设置为1，表明产生了该信号。

系统对置位之前该标志位已经为1的情况不进行处理，这说明进程只处理最近接收的信号。

信号产生后并不马上送给进程，它必须等待直到进程再一次运行时才交给它。

每当进程从系统调用中退出时，**内核会检查它的signal和blocked字段，查看是否有需要发送的非屏蔽信号，若有则立即发送信号。**

如果信号的处理被设置为缺省，则系统内核将会处理该信号，否则会执行用户提供的信号处理程序。

信号处理结束后，会返回到进程的下一条指令继续执行

不是每种信号的处理都可以自定义处理函数！

每种信号都有默认处理程序，尽管有些默认处理程序就是忽略它，但它就是有！

（5）socket 嵌套字

一、OS 视角下 Socket 的核心作用

在操作系统中，Socket 的本质是内核提供的一种“跨进程通信接口”，其核心作用是：

允许同一主机内的不同进程通过“本地 Socket”通信。

允许不同主机的进程通过“网络 Socket”通信。内核通过网络协议栈（如 TCP/IP）将本地进程数据封装为网络包，实现跨主机传输，而其他 IPC 仅依赖本地内核，唯一一个支持网络跨主机进程通信的机制

对 OS 而言，Socket 的核心是抽象出统一的“通信端点”，让进程无需关心底层是本地还是网络传输，只需通过内核提供的 Socket API 读写数据。

二、OS 中 Socket 的使用逻辑

1. 创建 Socket：进程调 socket()系统调用，内核为其分配一个 Socket 结构体（包含通信协议、缓冲区、状态等信息），返回文件描述符（类似文件句柄）。

2. 绑定/连接：

本地通信：通过 bind()绑定到本地文件路径（如`/tmp/socket`），内核维护路径与进程的映射。

网络通信：通过`bind()`绑定 IP+端口，内核记录“端口-进程”对应关系。

3. 数据传输：

进程通过 read()/write()系统调用，内核将数据从发送方缓冲区复制到接收方缓冲区（类似管道的“内核中转”机制）。

合作进程可以使用的进程通信方式：管道、嵌套字、信号量、消息队列、信号、共享存储

不能通过全局变量和数据库进行通信

线程通信（线程之间的信息交换）：

线程通信因为不需要内核干预，所以速度更快！

共享全局变量、共享堆内存、信号、线程局部存储（即每个线程的独立副本，存储线程私有数据（如线程 ID、状态），但不用于直接通信）

每个线程拥有进程的地址空间，**但是每个线程之间的栈和线程局部存储是不共享的（2024真题）**，这些用于保存一些基本的函数调用和变量

但文件描述符 fd 虽然不同进程打开同一文件的文件描述符是不一样的，但是该进程内部不同线程的文件描述符 fd 是一样的，因为只有进程才能分配文件描述符 fd

王道书上说的引入线程便于进程通信，本质上想表达的意思是便于了进程内部线程的通信因为他们直接拥有同样的地址空间

补充：进程的地址空间

进程地址空间示意图（32 位 Linux，总大小 4GB）

plaintext

```
+----------------------------------------+  最高地址（0xFFFFFFFF，4GB）
|  内核空间（1GB，用户进程不可直接访问）  |  ← 存放操作系统内核代码、数据、内核栈等
+----------------------------------------+  0xC0000000（3GB）
|  环境变量与命令行参数区                |  ← 存放进程启动时的环境变量（如PATH）和命令行参数
+----------------------------------------+
|  栈（Stack）                           |  ← 从高地址向低地址生长（向下生长）
|  （函数参数、局部变量、返回地址等）     |
+----------------------------------------+  ← 栈顶（动态变化）
|  共享库加载区（.so文件）               |  ← 动态加载的共享库代码和数据（如libc.so）
+----------------------------------------+
|  堆（Heap）                            |  ← 从低地址向高地址生长（向上生长）
|  （动态分配的内存，如malloc申请的空间） |
+----------------------------------------+  ← 堆顶（动态变化）
|  BSS段（未初始化全局/静态变量）        |  ← 初始值为0或未初始化的变量（仅占地址，不占磁盘空间）
+----------------------------------------+
|  数据段（Data Segment）                |  ← 已初始化的全局变量和静态变量（如int a=10;）
+----------------------------------------+
|  代码段（Text Segment）                |  ← 进程的可执行指令（只读，可被共享）
+----------------------------------------+
|  保留区（空指针引用区）                |  ← 最低地址（0x00000000附近），禁止访问（防止空指针错误）
+----------------------------------------+  最低地址（0x00000000）
```

已经初始化的全局变量、定义的常量——数据段

未经初始化的全局变量——BSS 段

局部变量 例如 int a——栈；

函数调用的参数——栈

动态分配 malloc——堆

补充：王道 2.1 课后题

（1）并发进程的运行结果具有不可再现性：例如银行共享账户共计 100 元，取钱 A 取走 100，B 存了 50，最终剩余 150。但是如果 AB 并发进程同时取出数据“100”，然后 A 写入 100+100，但是 B 写入的是 100-50，并且 B 是最后写入的，就导致 A 写的数据被覆盖了。

上述问题也叫做并发进程失去了封闭性
如何约束这种问题，借用锁、同步机制
YYK
2026/10/11 16:22

（2）高级调度的概念---作业调度，从外存调入数据、程序、创建进程！

（3）PCB 里面还存储了 CPU 的状态，没有存储全局变量，这属于代码的部分

这里指的是处理机上下文或硬件上下文

YYK
2026/10/11 16:29

（4）数据堆存储的是 malloc 的内容，数据栈存储的是临时分配的局部变量的值和形参实参；剩余的全局变量保存在正文段

（5）运行到阻塞是进程自行决定的，但是就绪到运行、阻塞到运行都是 CPU 控制的

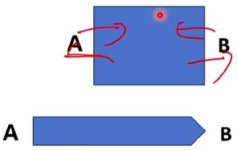
（6）

速度最快的进程通信方式是（ ）。

A．消息传递　　B．Socket　　C．共享内存　　D．管道

本地进程通信：消息传递 共享内存 管道

网络进程通信：Socket



套接字（socket），应用程序可以通过它发送或接收数据，可对其进行像对文件一样的打开、读写和关闭等操作。套接字允许应用程序将I/O插入到网络中，并与网络中的其他应用程序进行通信。网络套接字是IP地址与端口的组合。

共享内存的通信速度最快

YYK
2026/10/11 16:37

（8）线程之间的通信可以通过“共享地址空间”，因为同一进程的不同线程之间，就是共用

同一片地址空间的；而不同进程之间的通信需要通过系统调用来完成

线程包含CPU现场，可以独立执行程序

（9）管道就是一种特殊的文件，所以进程之间可以通过文件系统来传输数据

两个合作进程是不能通过共享全局变量来进行通信，因为两个进程是不同的地址空间；也

不能通过数据库来传递数据

（10）

并发性较好的多线程模型有（ ）。

I．一对一模型　　II．多对一模型　　III．多对多模型

A．仅 I　　B．I 和 II　　C．I 和 III　　D．I、II 和 III

看不懂了，选 C，其中多对一模型只有一个内核级线程来控制，那么对于内部的 n 个用户线程来说，调度是不友好的，并发性很差
YYK
2026/10/11 17:28

（11）父进程与子进程是相对独立的两个进程，拥有独立的地址空间；父进程被撤销掉，

子进程是可能被保留的---孤儿进程

（12）Fd 文件描述符是和进程直接相关联的，与线程无关，线程直接使用了进程的 fd

(二)CPU 调度与上下文切换

1.调度的基本概念

①高级调度任务是从后备队列中挑选一个作业，为其分配内存、IO 设备、建立相应进程，让其从创建态到就绪态。而中级调度是挂起态和就绪态之间的转变。

②2022 年真题踩坑，在就绪队列为空，刚来第一个进程，直接让该进程上处理机运行，这个也算一次 CPU 调度！

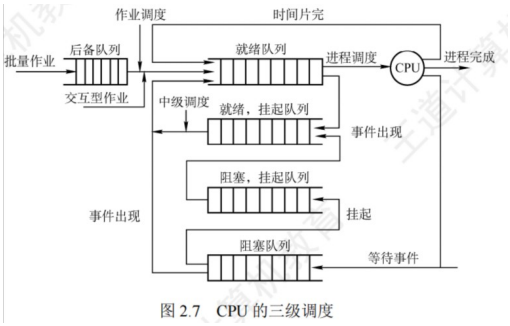


图 2.7　CPU 的三级调度

高级调度（作业调度）	从外存中调度作业进入内存，这个过程是创建新进程 PCB，分配资源，每个作业只调入一次、调出一次（撤销 PCB）
中级调度（内存调度）	把暂时不运行的进程“挂起”到外存，提高内存利用率和系统吞吐量，具备运行条件时再调入内存，发生频率高
低级调度（进程调度）	分配 CPU 的过程，频率最高

2.调度的目标

（1）CPU 利用率，无需多言，主要关注 CPU 的使用时间和空闲时间

（2）系统吞吐量：单位时间内完成作业的数量，所以短作业优先调度算法的系统吞吐量最大

（3）周转时间=作业完成时间-作业提交时间

平均周转时间=周转时间总和/作业数量

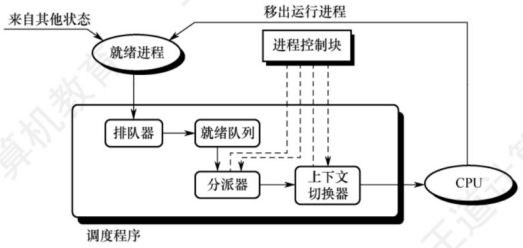
带权周转时间=作业周转时间/作业实际运行时间，**所以它一定≥1**

（4）等待时间：等待 CPU 的时间之和

（5）响应时间：从用户提交申请开始到首次响应的时间

3.调度的实现

调度器/调度程序(scheduler),调度的时机与调度方式(抢占式/非抢占式),闲逛进程，内核级线程与用户级线程调度。



排队器的目的是：按照一定策略给就绪进程排队，然后插入就绪队列

分配器是根据调度程序所选的进程，从就绪队列中取出，将 CPU 分配给它。

上下文切换器：是将上下文环境保存到 PCB 中，而不是栈中！然后装入分配程序的 PCB

便于分配程序，再保存分派程序的上下文到 PCB，再调入新进程的上下文环境（中间需要执行一次分派进程）

由于中间需要加载 load 和 store 指令，以保存寄存器的值，所以，比较耗时间，固然可以采

用两组寄存器，一个供内核使用，一个供用户使用，这样只需要再上下文切换时，修改指针即可。

处理机调度的时机：

请求调度的事件、运行调度程序、调度新的就绪程序，按道理这应该是顺序执行的，但是不一定会立马进行调度和切换！

①处于临界区（临界区是指进程中访问共享资源（如共享变量、文件、设备等）的那段代码。由于共享资源在同一时刻只能被一个进程访问，因此操作系统需要确保每次只有一个进程可以进入临界区。进入临界区的进程通常会执行一些对共享资源的读写操作，这些操作要求原子性，即在执行过程中不能被其他进程打断）的进程也可能因为中断或者抢占而被调度。如果进程请求的是一个需要等待的资源例如打印机（这是属于共享资源，进程处于临界区），那么有可能主动放弃 CPU，请求调度。

② 创建新进程，父子进程选择一个进程上处理机。

③ 某个进程需求的外设准备完毕， 发出 IO 中断，此时可以根据优先级来判断让不让该进程重新上 CPU 执行，或者继续执行中断之前的进程。

④系统调用完成并返回用户态时能进行处理机调度，详细原因解释在模式切换（自己 ctrl+f 搜索模式切换）那一小部分。

不能进行进程调度的时机：

⑤ 原子操作过程中也不能进程调度和切换

⑥在处理中断过程中不能进程调度和切换

进程调度的方式：

非抢占式（非剥夺方式）调度：简单、系统开销小、但不适用于分时系统（因为在非抢占式调度中，时间片到了触发中断不会让该进程下处理机）和大多数的实时系统

抢占式调度：抢占必须要遵循一种原则，不能随机

闲逛进程：

没有就绪进程时，调度闲逛进程上处理机运行！

内核级线程调度需要上下文切换、修改内存映像、使高速缓存失效

进程的切换：

任何进程的切换都是依靠内核完成的

调度本身是一种决策行为，而切换是一种执行行为！

上下文切换需要：

①挂起该进程，将环境保存到 PCB 中，包括计数器和寄存器

②将 PCB 移入相应的队列

③选择另一个进程，先更新 PCB（例如修改该进程从就绪态到运行态，记录该进程本次开始运行的时间用于后续计算 CPU 占用率、响应时间等，重置或更新与调度策略相关的字段（如时间片剩余量，若采用时间片轮转调度，新进程开始运行时需将其时间片重置为系统规定的初始值。）

④将新进程的 PCB 环境恢复到 CPU 中

⑤跳转到新进程的程序计数器指向位置并执行！

4.CPU 调度算法

饥饿和死锁并不一样，前者是调度策略问题、后者是系统环形等待。

同时互斥性并不是调度算法考虑的因素。

名称	优点	缺点	特殊点
FCFS 先来先服务	算法简单、容易实现。 有利于 CPU 繁忙型作业，对长作业有利。	效率低、但是对短作业不利，短作业等了太长时间才轮到自己。不利于 IO 繁忙型作业。（即是自己处于 IO 阶段，也无法让出 CPU，因为要严格遵循先来后到，所以 CPU 利用率很低）	1. 作业、进程调度。 2. 后备作业队列和就绪队列 3. 不饥饿，只要排队就会有 CPU 分配。 4. 非抢占式 5. 不可剥夺算法，IO 期间让不出 CPU
SJF（SPF）短作业优先，J 代表 Job，P 代表 Process	**平均等待时间和平均周转时间最优秀， 最短。**	对长作业不利，可能发生饥饿。紧迫性作业不能被及时处理。算法不能准确的根据运行时间来调度，总有误差。	1. 作业、进程调度 2. 选择估计运行时间最短的。 3. 饥饿，只要有短进程进入就绪队列，长进程就一直被迫等待。 4. 可以是抢占式、非抢占式，默认非抢占式！ 5. 适用于批处理系统

高响应比优先调度算法 HRRN	平衡了 FCFS 和 SJF 的优点，采取以响应比为指标。		1. 作业、进程调度 2. 选择最高的响应比的进行调度（响应比 =（等待时间+要求服务时间）/要求服务时间） 3. 不饥饿 ，等待时间越长，响应比越高，被调度的概率越大。等待时间相同，要求服务时间越短，响应比越大。 4. 非抢占式 5. 需要对等待时间和服务时间进行计算，也需要一定的开销
优 先级调度算法			1. 作业、进程调度 2. 选择优先级最高的进程或者作业 3. 可能饥饿。静态优先级是创建进程时就已经确定，不够精确，可能导致饥饿现象；动态优先级在执行过程中调整，例如随着等待时间

			增大，进程优先级提高，不会导致饥饿现象。执行时间增大，进程的动态优先级降低。系统进程的优先级大于用户进程；交互式进程的优先级大于非交互式进程；IO 型进程大于 CPU 计算型进程 4. 抢占式和非抢占式。可能抢占也可能不抢占，不能固执的认为，优先级算法一定是抢占算法！！！！ 5. IO 密集型进程优先级较高，让 IO 先工作，提高系统工作效率。 6. 时间片轮转也要按照优先级顺序轮转 7.
时间片轮转调度算法	公平！按照先来后到的顺序，分配时间片顺序。如果时间片未用完但进程执行完	时间片的大小对效率影响很大。时间片特别大，则退化为 FCFS。如果时间片特别短，CPU 负	1. 进程调度 2. 均匀分配时间片 3. 绝对抢占式 4. 不饥饿 5. 适用于分时系统

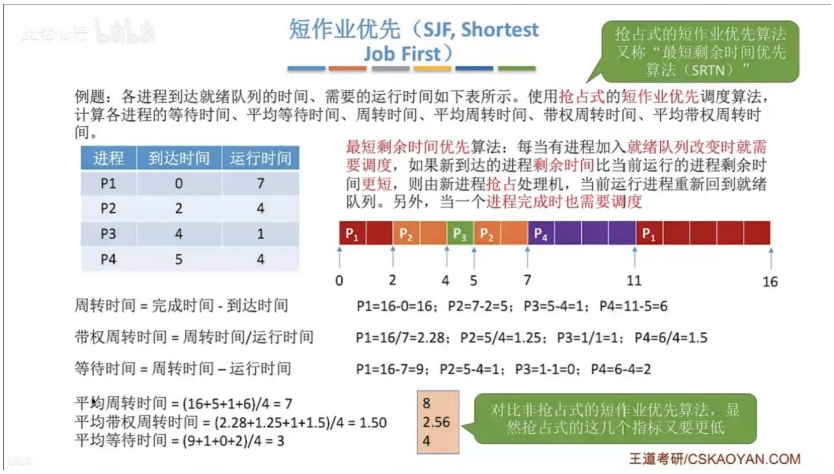
	了，则立即调度新进程上处理机	担增加，导致真正用于处理进程的时间缩短。并且没有区分任务的紧急情况。	6. 增加了系统开销，不会使系统更高效。
多级队列调度算法			1.在多 CPU 系统当中，为不同的 CPU 分配不同的调度策略，满足用户需求。
多级反馈队列调度算法	平衡了时间片和优先级调度算法。短作业优先、周转时间较短、不会长期得不到执行。		1. 进程调度。 2. 算法思路复杂 3. 抢占式 4. 可能饥饿。高优先级队列的进程一直被调度，第二级别的就绪队列迟迟等不到时间片。
保证调度算法	做出性能保证，保证每个进程的性能趋于公平，即每个进程获得的 CPU 时间为 1/n		
公平分享调度	每个用户保证 n		

注意两个算法的不同，一个关注的是用户层面的公平，用户拥有 n 个进程，加起来分配 n 个时间片，但是另一个用户拥有 1 个进程，加起来也是分配 n 个时间片，一个关注的是进程层面的公平；
YYK
2026/10/11 17:53

算法	分之一的 CPU 时间		

（1）为什么先来先服务算法有利于 CPU 繁忙型而不利于 IO 繁忙型的进程，因为 FCFS 不考虑进程的特性，如果一个进程是 IO 繁忙型的，导致 IO 期间，CPU 空闲但是又不能被其它进程使用，效率低。相反来说短作业优先算法 SJF 和高响应比算法可以让更短的 IO 繁忙型进程先运行，降低了整体的平均等待时间。

（2）关于抢占式和非抢占的 SJF 的具体说明，抢占式是指，在某个进程正在运行的时候，剩余运行时间大于刚来的新进程。通常情况下，抢占式的 SJF（最短剩余时间优先调度算法）要比非抢占式的 SJF 性能表现更优秀的运行时间，那么处理机就会被抢占，处理如下图



（3）前三个调度算法的交互性很糟糕，适用于早期的批处理系统。

而对于实时性系统，比较适合用抢占式的优先级高的调度算法，不采用先来先服务调度算法，很简单，自己想复杂了，先来先服务还是要排队，前面有任务没完成，自己就要等，而不是插队！！！

时间片轮转算法可以实现人机交互， 因为时间片很短，很快就轮到自己，运行一小段，在用户看来很快就响应操作了，实现了“交互功能”

（4）对于时间片轮转时间计算问题，一定要自己动手算算。

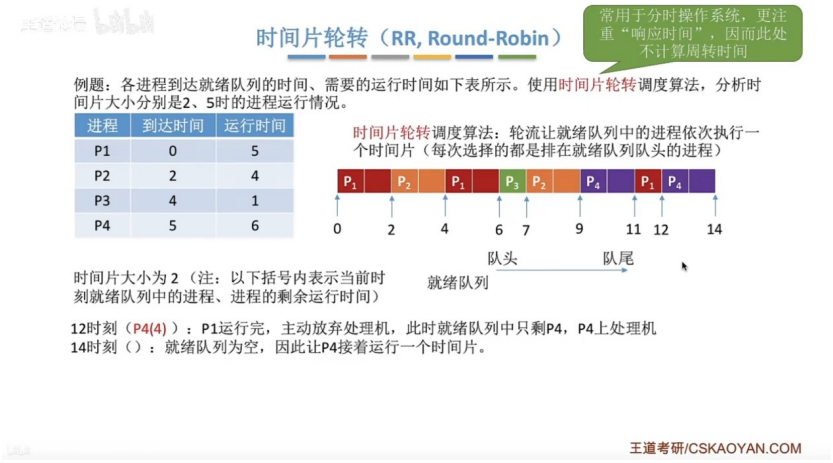
时间片轮转他是靠什么来计时的？换句话说，是依靠什么来判断该不该进行进程切换的？

这个是由于时钟中断来执行的，例如每个进程每次执行的时间片为 10ms，那么我设置一个时钟中断为 5ms 每次，每次中断就让该进程的剩余时间减去 5ms，直到 0 时，内核意识到该进程的时间片用完，该切换进程了。这个时钟中断的时间大小对其他因素是有影响的（逻辑、开销、响应度）。如果没有时钟中断，那么进程就会一直执行下去，还可以更新系统时间、记录进程运行时间、处理一些异常问题、进行动态调整优先级。

时间片轮转我掉进了 2024 年的一个坑，它说十个进程同时进入就绪队列，P 进程排在最后一个，并且它需要的 CPU 时间最短，为 25ms，时间片为 5ms，求 P 的周转时间，我忘记了，其实 1-9 个进程他们的 CPU 时间都大于等于 25ms，经过五轮时间片之后，进程 P 的任务已经结束了，此时已经用了 250ms，无论 1-9 的进程有没有结束，跟进程 P 的周转时间已经没有任何关系了。

还会记录一些内核中时钟变量的值（例如系统时间、下一次中断的时间、系统启动以来锁经历的时钟中断次数）2018 真题，真尼玛变态

当一个进程被阻塞结束之后，然后进入了就绪态，等待处理机调度，像其他就绪进程一样，在被调度时获得一个新的时间片。所以不是阻塞了结束之后给他发时间片，而是调度了之后给他发时间片。（2014 真题）



（5）优先级算法涉及到静态和动态优先级的概念、降低提高进程优先级的合理时机、IO 型和计算型优先级的设置顺序。

必须在该进程用完时间片之后才降低它的优先级，否则在刚开始上处理机就降低优先级，很可能因为优先级较低又被抢占，又有可能因为刚上去被降低而被强占，导致迟迟得不到完整的运行。另外，完成 IO 操作后，进入就绪队列，这不是时间片用完，此时它刚刚并没有占用处理机，没必要降低优先级。（2010 年真题）

此外还可以对长期占用 CPU 资源的进程降低其优先级：对于 CPU 密集型进程，如果它持续占用 CPU 运行了较长时间，为了保证系统中其他进程（尤其是 I/O 密集型进程或交互式进程）能及时获得 CPU 资源，可降低其优先级。还有系统负载过高，也可以降低一些非重要的进程优先级

（6）多级反馈队列调度算法

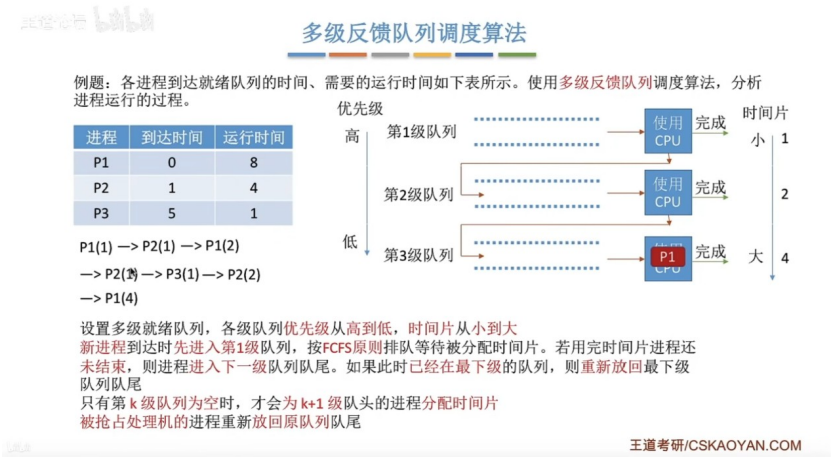
规则比较复杂：①分为不同级别的队列，每个级别的队列时间片依次增加。

② 如果执行完了这个时间片，那么进入到下一级的时间片队列，如果中途被打断，下处理机，重新回去到该就绪队列。

③ 如果有更高级别的队列里面有就绪进程等待被调度，那么先处理它，直到 k 阶就绪队列为空，才能执行 k+1 阶的就绪队列的调度算法。

④ 如果已经是最后一层的就绪队列，但是一个时间片还没有运行完，则重新回到该队列排队。

⑤多级反馈队列调度算法 MLFQ 可以更好地处理优先级不同的进程，因为它不是简单地按时间片平等地分配 CPU 时间。MLFQ 的多级队列设计允许它对不同类型的进程（如 CPU 密集型和 I/O 密集型）进行区分对待，从而更有效地利用资源并提高整体性能。可以动态调整优先级，将计算型的进程提高队列。所以要考虑设计进程在就绪队列之间的迁移条件，这样可以让一些低优先级队列中的进程再返回到高优先级的队列中，避免 P2 队列中的进程长时间处于低优先级得不到较好的调度。如果进程 Q 或 S 在 P2 队列中进行了 I/O 操作，由于 I/O 操作完成后进程的执行特性可能发生变化，通常这类进程更适合在较高优先级队列中尽快得到 CPU 资源以继续执行后续任务。所以当进程 Q 或 S 完成 I/O 操作重新进入就绪状态时，按照迁移条件设定，它们可能会被迁移回 P1 队列进行时间片轮转算法。（2020 真题）



⑦关于等待时间的问题：2019 年真题，先让 P1 先进行 Q1 队列的时间片轮转，此时 P2 在等 P1，然后 P1 进入 Q2 队列，P2 进行时间片轮转，此时 P1 在等 P2（这一步非常容易忘记，不能忘！！！！），然后继续 P1\P2 都进入了 Q2 队列，Q2 队列才能调度运行。

5.多处理机（多 CPU）调度

主要解决两个问题：

（1）第一个是亲和性和负载均衡的概念。

亲和性就是让一个进程只在一个 CPU 上运行，这样保证 Cache 效率，不用频繁去内存取数据。

负载均衡就是多个 CPU 工作的频率是不是近似相等的，不能一个 CPU 一直空闲，另一个 CPU 一直忙碌。

（2）第二个问题就是调度的方案，主要针对于多 CPU 的调度队列

①公共就绪队列：所有进程共享一个队列，队列需要上锁（互斥），防止多个 CPU 同时抢夺进程，天然的解决了负载均衡，亲和性比较差。软亲和是指尽量让进程在一个 CPU 上运行，硬亲和是写入 PCB，该进程只能由某一个 CPU 运行。

②私有就绪队列：每个 CPU 拥有一个私有的队列。天然解决了亲和性的问题。推迁移是系统程序让其他 CPU 的进程来该负载率低的 CPU 上运行。拉迁移从超载 CPU 就绪队列中拉一些进程来我这里运行。推迁移和拉迁移可以并行执行。

6.上下文及其切换机制

（1）①上下文切换：操作王道 P70

CPU 上下文（程序计数器、其它寄存器）保存到 PCB→→将 PCB 移入相应队列→→选择另一个进程，激活更新它的 PCB 内容→→将 PCB 的内容恢复到 CPU 里→→跳转到新进程的 PCB 中的程序计数器的位置重新执行。

②上下文切换消耗大，可以只有一个寄存器组或者多个寄存器组+指针

③模式切换与上下文切换的区别：模式切换（内核态与用户态）可能前后运行同一进程（什么时候不一样呢，例如进程 B 需要访问磁盘，从用户态转为内核态进行系统调用，内核发现磁盘忙碌，需要等待时间，所以将告诉 CPU 调度别的程序来运行，在内核态进行进程调度与切换，在返回用户态后，前后运行的不是同一个进程。），上下文切换（CPU 调度和切换）只发生在内核态，前后不同进程。

（2）线程切换和进程切换本质上有区别

②对于同一进程的不同线程切换，只需要更新程序计数器（不同线程执行的进度不一样）、寄存器组（不同线程的现场数据不一样），栈基址寄存器（每个线程拥有独立的栈）。同一进程不同线程切换时，进程的地址空间和页目录没有变化，所以 PDBR 的内容也不会变化。（2018 真题）

③对于同一进程的不同线程切换，页基址寄存器不需要改动，因为同一进程的不同线程共用同一个虚拟地址空间，原有的 TLB 不会失效。

④要理解栈空间和寄存器组保存信息的差异，寄存器组保存的是一些关键的必要的信息，而栈空间保存一些函数调用和数据信息，线程 TCB 里买呢保存的有栈指针和栈基址寄存器的数据，指向栈空间。

⑤要理解，在进程中可能会存在进程内部的全局变量，这个全局变量供进程内部所有的线程使用，因此，这些线程在并发执行时，对全局变量的操作要互斥进行。（2016 统考真题）

补充：

（1）创建态到就绪态的过程是分配资源、写入 PCB 的过程，属于高级调度；

（3）互斥性是两个进程在访问临界资源的时候不会冲突，这并不是调度算法考虑的因素

（4）处于临界区的进程在退出临界区之前可以被调度；只不过这个临界资源还是被他占有，等待下一次调度即可；

（5）先来先服务算法有利于 CPU 繁忙进程、不利于 IO 繁忙进程

（6）再次体会分时和实时的区别：分时系统讲究交互性、要求时间片轮转；实时系统讲究抢占并尽快处理；

（7）在动态优先级中，随着执行时间的增加，其优先权应该降低

（8）

47. 【2018 统考真题】某系统采用基于优先权的非抢占式进程调度策略，完成一次进程调度和进程切换的系统时间开销为 1μs。在 T 时刻就绪队列中有 3 个进程 P_1、P_2 和 P_3，其在就绪队列中的等待时间、需要的 CPU 时间和优先权如下表所示。

进 程 名	等待时间	需要的 CPU 时间	优 先 权
P_1	30μs	12μs	10
P_2	15μs	24μs	30
P_3	18μs	36μs	20

若优先权值大的进程优先获得 CPU，从 T 时刻起系统开始进程调度，则系统的平均周转时间为（ ）。

A. 54μs　　B. 73μs　　C. 74μs　　D. 75μs

非常细节的一道题，自己忘记了，第一个进程也是在就绪队列中的，把它调度到 CPU 也花费了一微秒；这个机器容易忽略！

YYK
2026/10/13 15:52

54. 【2024 统考真题】假设某系统使用时间片轮转调度算法进行 CPU 调度，时间片大小为 5ms，系统共有 10 个进程，初始时均处于就绪队列，执行结束前仅处于执行态或就绪态。若队尾的进程 P 所需的 CPU 时间最短，时间为 25ms，不考虑系统开销，则进程 P 的周转时间为（ ）。

A. 200ms　　B. 205ms　　C. 250ms　　D. 295ms

这个题目也是需要自己看清楚，队尾的进程 P 所需时间，说明进程 P 在队尾排着呢！！！！
YYK
2026/10/13 15:53

(三)同步与互斥

1.同步于互斥的基本概念

①同步就是确定进程执行的先后顺序；互斥就是间接制约关系，阻止两个进程对同一个临界资源进行访问 。

②临界资源的定义：可以被多个进程访问，但一次只允许一个进程访问。不同进程对临界资源的操作是不同的，所以独立的代码段也就不同，所以一个临界资源的相关临界区是由不同进程的临界区构成的。

进入区（在此区检查能否进入临界区，并设置访问临界区的标志）、临界区（critical section能访问临界资源的代码）、退出区（将正在访问临界区的标志清除）、剩余区（代码的其余部分）

磁盘是共享设备，因为可以并发的访问不同磁盘区域

公用队列是临界资源，只能同时有一个进程访问。

YYK
2026/10/13 22:19

私用数据不满足可以被多个进程访问。

可重入的代码程序是指可以被重复调用执行，不会发生错误，中断后重新执行也不会发生错误的。一次可以被多个进程调用，不属于临界资源。并且可重入代码不允许被任何进程修改。

共享代码必须是可重入的，例如进程映像里的程序、数据、PCB组成，其中共享的程序段可以被必须是可重入代码，并且不允许被修改。

③互斥遵循的原则：空闲让进、忙则等待（已有进程进入临界区，则另一个进程需要等待）、有限等待（对于请求访问的进程，应该让其在有限时间内能够

进入临界区）、让权等待（当进程不能进入临界区时，应该释放处理机，这条不一定必须遵循，2020 真题）

2.基本的实现方法软件方法、硬件方法

	单标志法	双标志先检查	双标志后检查	皮特森算法
算法思路	进入区设置 while 循环检查是否轮到自己使用（turn！=自己）。 临界区使用资源。 退出区将资源让给对方（ture=对方）。	进入区设置 while 循环检查对方是否想用（flag[对方]！=ture）。如果对方不想用，那就把 flag[自己]=true。 临界区使用资源。 退出区将自己设置为不想使用资源	进入区设置 flag[自己]=true，while 循环检查对方是否想用（flag[对方]！=ture）	进入区设置 flag[自己]=true 表示自己想用。再将 turn=对方表示愿意让对方先用,或者说表示轮到对方进入临界区。再 while 轮询（对方想用且 turn=对方），否则自己继续执行。 临界区使用资源。 退出区将自己设置为不想用 flag[自己]=false

这里四道题的代码应该也是要掌握的，但是我没有掌握，对于以后做题还是很有必要的
YYK
2026/07/13 17:30

		（flag[自己]=false）。	。如果对方不想用，则跳出 while 循环，否则一直轮询。 临界区使用资源。 退出区将自己设置为不想使用资源（flag[自己]=false）。	

特殊点	如果上一个使用该资源的是自己，那么必须要等到下一次对方使用过后，自己才能使用	如果在上锁那一步突然发生了进程切换，就可能导致两个进程同时访问临界区。	一方上锁后，进程切换另一方也进行上锁，进而导致临界区都不能被访问且无限轮询	已经将临界区设置为对方，但是又不死心，轮询检查了 while 循环，没有让出处理机，浪费资源。
违背	空闲让进	忙则等待	空闲让进、有限等待	让权等待

皮特森的违背让权等待还是很重要的，关键在于“皮特森算法”不死心，要继续轮询检查
YYK
2026/07/13 17:32

硬件方法：

	中断屏蔽	硬件指令—TestAndSet（TSL）	硬件指令—Swap（Exchang、XCHG）
思路	关中断、执行操作、开中断 这样保证进入临界区不	用硬件执行上锁过程、原子操作不可被中断。所以不需要关中断操作（2016	用硬件执行上锁过程、原子操作不可被中断。先用 key 记录之前是否被上锁，然后

	会被另一个进程打断（因为，关中断屏蔽了时钟信号，所以 CPU 接收不到时钟信号，那就一直运行该进程，限制了切换进程的操作。）（但是用户程序是不能通过关中断来实现进程互斥的，因为关中断是特权指令，用户程序无权限使用 2021 真题）（为什么在 PV 操作中对信号量必须互斥访问？因为信号量 S 都是共享变量，多个进程都可对其进行读写操作，所以必须互斥访问，而且互斥的技术就是关中断和开中断！2021 真题）	真题）	自己主动上锁，再检查 key 是否是 ture，如果不是 ture，代表自己是第一个上锁的，那就直接进入临界区运行。
优点	简单高效	适用于多处理机系统	适用于多处理机系统 简单、容易验证正确性、适用于任何数量的进程、支持

PV 操作本质上是两个操作，即两个低级通信原语

YYK
2026/10/13 22:22

			系统中有多个临界区，只需要为每个临界区设置一个布尔变量即可
缺点	限制 CPU 交替执行程序，系统效率明显降低只适合内核进程、不适用于多处理机系统	while 检查临界区被上锁，则一直轮询，直到对方解锁，这样浪费了 CPU，违背让权等待。因此 TSL 机制下，不存在阻塞态的进程，它会一直轮询检查（2016 真题）	while 检查临界区被上锁，则一直轮询，直到对方解锁，这样浪费了 CPU，违背让权等待

能实现让权等待的也就剩下信号量机制了

YYK
2026/10/15 16:52

信号量实现了让权等待，所以非常重要，要记住，没能获得资源的全都被阻塞。

3.互斥锁

本质就是对临界区的访问进行上锁的一个东西

YYK
2026/07/13 17:38

①硬件实现、原子操作

②通过 acquire 和 release 函数来获得锁和释放锁，获得锁的前提是 avaliable 函数可用（代表锁可用）。

③适用于多处理机系统，类似于 TSL 和 Swap 指令

④缺点是忙等待，一直轮询检查锁是否被释放，浪费了 CPU 的资源。

4.信号量

通过原语执行 PV 操作，本质上是低级进程通信原语，一气呵成，不能被打断

信号量是一个结构体，包含值和等待队列，但这两个属性都不对外暴露。信号量只能

进行 PV 操作，不能进行值判断。太重要了啊！！！！！！

例如 semaphore bowl = 6；

If（bowl>=1）这是一个极其错误的想法！！

wait()=P 操作

signal()=V 操作

整型信号量 S：用整数表示资源数量。

①wait 原语，相当于“进入区”，用 while（S<=0）来检测资源是否可用,若资源数不够，则一直轮询，违反让权等待

②signal 原语，相当于“退出区”，用 S=S+1 来释放资源

记录型信号量：

①不存在忙等现象，即不违反让权等待，原因是因为得不到资源就自我阻塞。

②wait 原语，相当于进入区，先将 S.value 减一，如果 S.value<0，那就调用 block 原语，（先将资源数减了，然后再判断，所以 S.value=0 时代表资源刚好够用），让该进程主动放弃 CPU，由运行态到阻塞态，并将其插入到该资源的等待队列。

③signal 原语，相当于退出区，先将 S.value 加一，如果此时 S.value 仍然小于等于 0，说明仍然有进程在等待该资源，则从等待队列里面 wakeup 原语唤醒第一个进程并分配资源。

④请注意，这里面的记录性型信号量 S=0 时代表仍然有资源可以分配，例如资源数 S=1，进程 B 占用一个资源导致 S=0，然后进程 A 又来申请资源导致 S=-1 并自我阻塞进入等待资源的队列，此时进程 B 释放了资源，导致 S=0，然后 A 获得了该资源，用完资源后释放 S=1；

信号量实现进程互斥：

①设置互斥信号量 mutex 为 1，然后通过 semaphore 定义，只要通过它定义，说明这是记录型信号量，不存在忙等的情况，即对进程有阻塞的功能。

②申请资源使用 P（mutex），释放资源使用 V（mutex），必须成对出现。

信号量实现进程同步：

①设置同步信号量 S 初始值为 0，然后通过 semaphore 定义，只要通过它定义，说明这是记录型信号量，不存在忙等的情况，即对进程有阻塞的功能。

②在前操作之后执行 V 操作，释放资源。在后操作之前执行 P 操作，代表取得资源，通过资源来限制执行顺序。这时候是先执行 V 后执行 P，先释放资源，后取得资源。

③同步信号量 S 初始值不一定必须为 0，也可以是一个正整数，取决于期待的那个条件是否已发生例如，初始值为 0，期望进程 1 执行完释放资源后，资源数为 1，进程 2 才能执行。另一种情况，初始值为 2，进程 1 和进程 2 释放资源后，达到四个资源时，进程 3 才能执行。

信号量实现前驱关系：

①相当于对进程执行顺序做了限制。如果有多个进程、多个需要限制的执行顺序，那么就需要多个信号量，每个信号量初始值都为 0，第一个执行的进程先执行 V 操作，释放资源，随后第二个进程可以获得该资源，执行 P 操作，再释放另一个资源（V 操作），推动下一个进程执行。

5.条件变量（管程）

管程存在的必要性：①由于资源种类比较多，程序员自行安排 PV 操作容易死锁，提出管程自动实现进程互斥，降低死锁可能性。

②帮助实现进程同步和互斥问题（2016 真题）

管程的组成结构：

①名称 name

②管程内部的共享数据结构（对应着共享的资源）资源 S

③对该数据结构定义的操作函数 take_away（），back_away（）

④对共享数据设置的初始值语句 init_source=5

管程的特性：

①将操作函数封装到管程里面（就好像 Java 的类），外部进程只能调用操作函数来实现对资源的获取和释放。

②每次只允许一个进程使用管程，天然实现互斥。

③管程内的数据只能被管程内的操作函数来修改。

④管程是被进程调用的：进程通过调用管程中定义的操作来访问共享资源。

⑤管程是语法范围：管程是通过编程语言的语法定义的，是一种语言级别的抽象。

⑥无法创建和撤销：管程的生命周期与程序的生命周期一致，不能在运行时动态创建或销毁。

条件变量的定义：

①本质就是进程进入管程后被阻塞，将该进程被阻塞的原因定义为条件变量。

②不同进程被阻塞的原因可能不同，所以有多个条件变量。

③每个条件变量都保存一个等待队列，如果因为该原因被阻塞的进程，就会被插入到该条件 1 变量的等待队列

④如果资源不够，则执行 x.wait，看清楚是先判断资源是否够数，才执行 x.wait 操作。如果资源够了，则执行 x.signal 唤醒该等待队列的进程，这里的 x 代表条件变量，即可被阻塞的原因。

条件变量和信号量的区别：

①操作函数类似，都可以实现进程的阻塞和唤醒。

②信号量有值，代表资源数；条件变量没有值，只实现排队等待和唤醒功能，管程里面的资源数用共享数据结构记录。

③管程的 signal 操作和信号量的 V 操作还是有所不同的，V 操作一定会引起信号量的改变，signal 操作可能不会产生任何影响（这是指在没有进程因为条件变量而阻塞的情况下）。

6.经典同步问题生产者-消费者问题；读者-写者问题；哲学家进餐问题。

生产者消费者问题：

限制条件：

①缓冲区没满，生产者才能生产（同步关系）

②缓冲区没空，消费者才能消费（同步关系）

③生产者消费者互斥访问缓冲区（互斥关系），这就引入了一个新关键点，不仅生产者可以唤醒消费者（当缓冲区空时，消费者阻塞，生产者生产产品，唤醒了消费者），而生产者也可以唤醒生产者（当生产者生产时，另一个生产者被阻塞，第一个生产者结束生产时，第二个生产者被唤醒）

④生产者消费者共享一个初始为空，大小为 n 的缓冲区。

信号量总共有三个，第一个是互斥信号量 mutex 代表对缓冲区的访问互斥，第二个是同步信号量 empty，表示空闲缓冲区的数量，第三个是同步信号量 full，表示产品的数量，也即非空缓冲区的数量。

生产者：

生产一个产品

P（empty）//消耗一个空缓冲区

P（mutex） //互斥访问缓冲区

放入缓冲区

V（ mutex）

V（full）//增加一个产品

消费者：

P（full）//消耗一个产品

P（mutex） //互斥访问缓冲区

放入缓冲区

V（mutex）

V（empty）//增加一个空缓冲区

消耗一个产品

思考：能否改变相邻 PV 操作的顺序？

答：P 操作肯定不能，假设缓冲区满，生产者先上锁，再去执行 P（empty）消耗一个空缓冲区，

但并没有空缓冲区了，所以阻塞等待消费者消耗一个产品，但由于此时已经上锁，消费者无法

进入临界区，固然导致二者互相循环等待，导致死锁。

但是 V 操作可以交换顺序

思考：能否将生产一个产品、消耗一个产品放到 PV 操作之间执行？

答：理论上是可以的，但是这样会导致临界区代码变成，上锁时间变长。

多生产者多消费者问题：

限制条件：

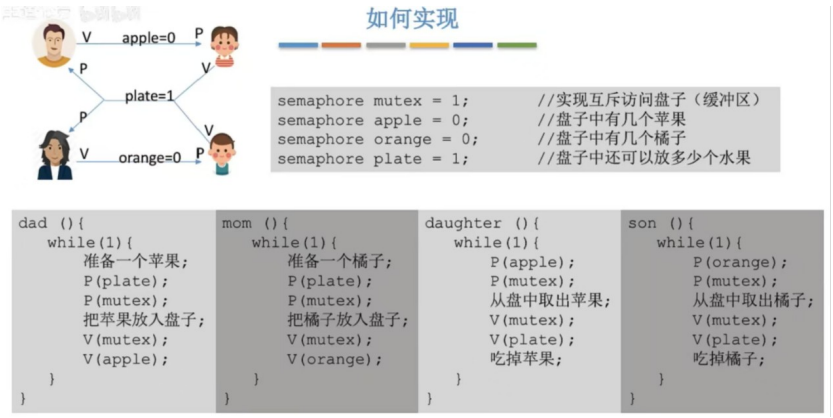
①盘子为空，父亲或者母亲才能放入水果（plate 同步关系）

②父亲放入苹果后，女儿才能取走苹果（apple 同步关系）

③母亲放入橘子后，儿子才能取走橘子（orange 同步关系）

④生产者消费者共享一个初始为空，大小为 1 的缓冲区，但互斥访问缓冲区（互斥关系）。

信号量总共有四个，第一个是互斥信号量 mutex 代表对缓冲区的访问互斥，第二个是同步信号量 plate，表示盘子还可以放入多少个水果，第三个是同步信号量 apple，表示苹果的数量，第四个是同步信号量 orange，代表橘子的数量。



思考：如果不设置互斥信号量 mutex，能否实现互斥访问？

答：肯定可以，原因在于缓冲区数量为 1，并且 apple、orange、plate 在任何时刻最多只有一个是 1，那么四个人执行 P 操作时，最多只有一个人不会被阻塞，那么也就成功实现了 P 操作。

缓冲区数量为 1 只是有可能会天然实现互斥访问，具体情况具体分析。但是缓冲区数量为 2 时，必然需要 mutex 来实现互斥访问

读者写者问题：

限制条件：

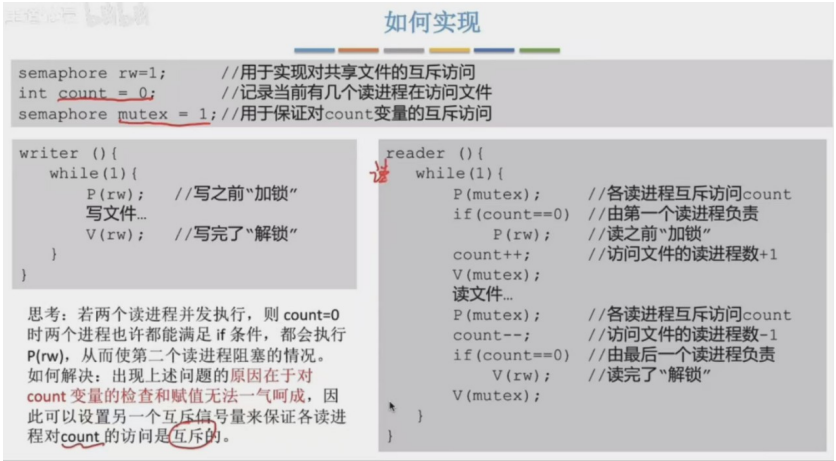
①写进程与写进程之间互斥访问（记录型信号量 rw、互斥关系）

②写进程与读进程之间互斥访问（记录型信号量 rw、互斥关系）

③多个读进程可以同时访问（这里是一个特殊的地方，不是互斥关系也不是同步关系，但还是需要设置一个记录型信号量 mutex 来对 count 的值进行互斥访问，count 代表读进程的数量）

④记录下读进程的数量 count

信号量总共有两个，第一个是互斥信号量 rw 代表对文件的访问互斥，第二个是整数变量 count，记录读进程的数量，第三个是互斥信号量 mutex，表示对 count 变量的互斥访问。



思考：为什么要对 count 的值设置一个 mutex 信号量来互斥访问？

答：假设进程 1 是读文件的进程，判断 count=0，说明它确实是第一个读文件的进程，此时发生跳转进程 2 也是读文件的进程，判断 count=0，它也认为自己是第一个读文件的进程，但是此时又发生跳转到进程 1，进程 1 进行加锁，此时又跳转到进程 2 要进行加锁操作，由于已经加过锁了，进程 2 加锁失败导致阻塞，随后的读进程 3456 倒是可以正常读，但是进程 2 却被一直阻塞。这个原因是判断和加锁的过程没办法一气呵成，此时就需要 mutex 信号量对 count 变量进行加锁。

思考：如果一直有读进程在读，写进程不就一直饿死了？如何调整才能让写进程优先？

答：再设置一个记录型信号量 fair，初始值为 1。这里解释第四种情况，先有一个读进程执行了 P（fair），相当于排在了队头请求获得访问文件的控制权，那么再交出 fair，让下一个想要访问文件的写进程也获得令牌，起到一个排队的作用，无论读写进程谁来了，都必须给我排队，保证了写进程不会被插队，只要等，就一定会轮到写进程。准确来说不能叫写优先，而更应该叫公平读写。

读进程：

```
c

reader() {
    P(fair);              // 1. 先排队
    P(mutex);
    if (read_count == 0)
        P(write_mutex);   // 第一个读者阻止写者
    read_count++;
    V(mutex);
    V(fair);              // 2. 释放排队锁，允许后续进程排队

    /* 读取数据 */

    P(mutex);
    read_count--;
    if (read_count == 0)
        V(write_mutex);   // 最后一个读者释放写锁
    V(mutex);
}
```

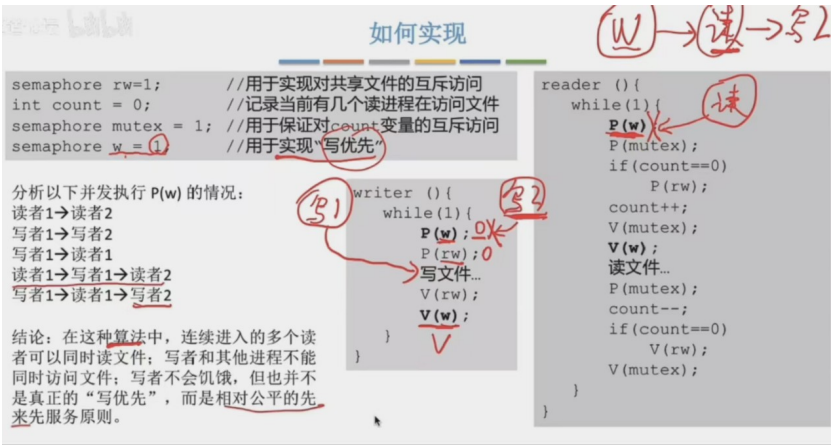
写进程：

```
c


writer() {
    P(fair);
    P(write_mutex);
    V(fair);

    /* 写入数据 */


    V(write_mutex);
}
```



另一个比较标准的读者写者问题：

46.（8 分）某进程中有 3 个并发执行的线程 thread1、thread2 和 thread3，其伪代码如下所示。

```
//复数的结构类型定义
typedef struct
{
  float a;
  float b;
} enum;
enum x, y, z; // 全局变量

//计算两个复数之和
enum add( enum p, enum q)
{
  enum s;
  s.a=p.a+q.a;
  s.b=p.b+q.b;
  return s;
}
```

```
thread1
{
  enum w;
  w=add( x, y);
  …
}

thread2
{
  enum w;
  w=add( y, z);
  …
}
```

```
thread3
{
  enum w;
  w.a=1;
  w.b=1;
  z=add( z, w);
  y=add( y, w);
  …
}
```

请添加必要的信号量和 P、V（或 wait()、signal()）操作，要求确保线程互斥访问临界资源，并且最大限度地并发执行。

其中线程 1 和线程 2 无需互斥访问 y，因为他们都是读线程

线程 1 和线程 3 需要互斥访问 y,设置一个 mutex_y1=1

线程 2 和线程 3 需要互斥访问 y，设置一个 mutex_y2=1

线程 2 和线程 3 需要互斥访问 z，设置一个 mutx_z=1

不能把 y1 和 y2 的锁设置为同一个，否则线程 1 和 2 之间就形成了互斥关系，这是错误的
YYK
2026/10/17 17:53

哲学家进餐问题：

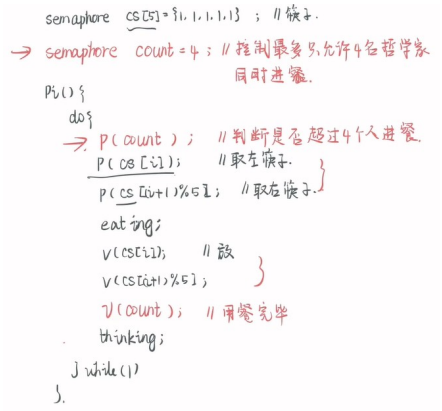
限制条件：

①只能一根一根拿起筷子，拥有两根筷子之后才能进餐。

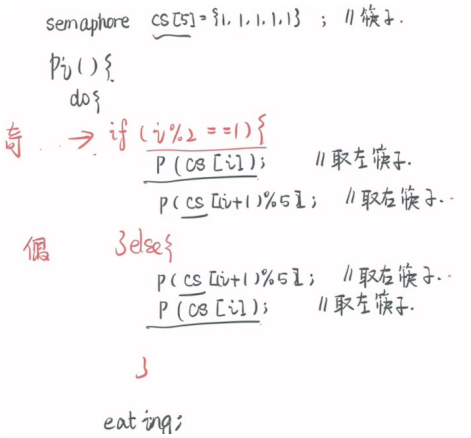
解决方案：

①第一种方法是限制最多有 n-1 个人可以进餐，这样至少有一个能获得两根筷子，避免死锁。

此时需要 n-1 个同步信号量。（这是破坏了循环等待条件）

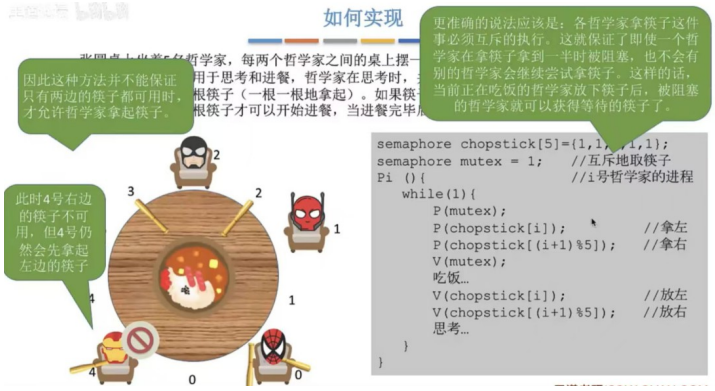


②第二种方法是要求奇数号先拿左筷子，偶数号先拿右筷子，也可以避免死锁，



③第三种方法是拿起前先检测，只有两根筷子都可以用时才拿起。（这种说法并不严谨，准确来说应该是有一个哲学家拿了一根筷子被阻塞，其它哲学家就不能申请再去拿筷子了，直到拥有筷子就餐完的哲学家释放筷子，解除刚刚那个哲学家的阻塞，进而释放 mutex）（这种方式可以预防死锁，**并且不会导致饥饿**，自己产生了误解，以为旁边的筷子可以被多次拿起，但在这种条件设定下，一个哲学家用完餐后总会放下筷子， 并且不再拿起，所以一定会等到两根筷子都空闲的那个时刻，所以肯定不会产生死锁！）

方法三的并行度比较差，所以最后考虑，并行度最高的还是限制 n-1 个人就餐的情况 ----来自于 2019 真题
YYK
2026/10/17 18:29



信号量总共有两个，第一个互斥信号量 mutex 保证基于一个进程拿起筷子的权力，第二个互斥信号量是 chopsticks[01234]，保证每个筷子只能被一个人拿起。

补充：

若一个系统中共有5个并发进程涉及某个相同的变量A，则变量A的相关临界区是由（ ）个临界区构成的。

A．1　　B．3

C．5　　D．6

这道题需要区分临界区和临界资源的区别，链接资源指的是A，临界区是每个进程中访问临界资源（本题中为变量A）的那段代码，本题总共有五个进程，所以总共有两个临界区构成

YYK

2026/10/13 22:36

（2）管程中的 signal 操作和 V 操作只是类似，而不是相同

(四)死锁

1.死锁的基本概念

死锁的定义（互相循环等待+各个进程都被阻塞）

死锁、饥饿、死循环的区别：

①发生饥饿的进程可以只有一个，死锁进程必然≥2 个

②饥饿进程可能在等 CPU（就绪态）或者其它资源（阻塞态，即获得不到其他资源也叫饥饿），而死锁必然等待其他资源（阻塞态）

③死锁和饥饿是操作系统的问题，死循环是程序员写的代码的问题

产生死锁的 2 条原因：

①系统资源争夺，只有对不可剥夺资源（打印机、磁带机运转过程中不能被其它进程夺走）的竞争才会导致死锁，可剥夺资源（CPU 和主存可以被其它进程夺走）不引发死锁。（这里的不可剥夺限制和可剥夺的宾语可以是基本资源、也可以是 CPU）

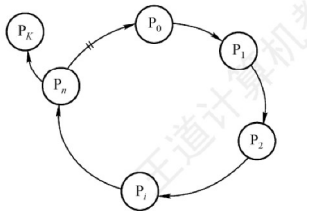
②进程推进顺序非法，例如两个进程需要同样的资源 R1 和 R2，但互相都持有其中一个资源而等待另一个资源，导致互相等待引发死锁。（本质也就是信号量使用不当也会导致死锁）

产生死锁的 4 条必要条件：

互斥、不可剥夺、请求与保持条件、循环等待条件

（循环等待和死锁的关系是什么，为什么发生循环等待未必导致死锁？死锁因为各种资源在循环互相等待，形成了一个环，环内的任何一个资源得到释放，死锁才会被解除。但是循环等待是可以等待环外的资源，它得到释放后，环也可以被解除，所以循环等待是死锁的必要条件。

这相当于是同类资源数大于 1，环外没有资源。**但如果同类资源数只有 1，那么发生了循环等待一定导致死锁。**）



即上面这种等待的环外资源释放之后，这个死循环就解除了，所以这并不是一个死锁

死锁处理三大策略内容、优缺点王道的比较表格。

表 2.5　死锁处理策略的比较

	资源分配策略	各种可能模式	主要优点	主要缺点
死锁预防	保守，宁可资源闲置	一次请求所有资源，资源剥夺，资源按序分配	适用于突发式处理的进程，不必进行剥夺	效率低，初始化时间延长；剥夺次数过多；不便灵活申请新资源
死锁避免	是预防和检测的折中（在运行时判断是否可能死锁）	寻找可能的安全允许顺序	不必进行剥夺	必须知道将来的资源需求；进程不能被长时间阻塞
死锁检测	宽松，只要允许就分配资源	定期检查死锁是否已经发生	不延长进程初始化时间，允许对死锁进行现场处理	通过剥夺解除死锁，造成损失

死锁的避免是在进程运行过程中不断调整安全的顺序，所以需要知道未来的资源需求。

死锁预防就是从根本上限制不存在出现死锁的可能性。

2.死锁预防 注意二者的区别

	破坏互斥	破坏不可剥夺条件	破坏请求并保持条件	破坏循环等待条件
内容	将互斥资源改为共享资源	已经保持某种资源，申请新资源得不到满足，那就必	有两种实现方式：①预先静态分配所有资源，否则不投入	顺序资源分配，给使用的资源进行编号，先拿到前面的

		须释放已经拥有的资源。	运行。 ②先获得初期使用的资源，使用后释放全部已用过的资源，再去申请下一步需要的资源。	资源的进程才有资格申请后面的资源。已经持有大号资源的，不能再去申请小号资源。（原名叫资源有序分配算法）
优点			①的实现简单 ②改进了①的缺点	
缺点	几乎不可能实现	实现过程复杂。反复申请、释放，增加系统开销，可能导致前一段时间的工作失效。降低系统吞吐量。	方法①系统资源被浪费（有些资源在进程最后才用到，但是却被保持了很长时间）、可能导致饥饿（长期持有，但是不释放）。	编号趋于稳定，不能增删资源设备。用户编程麻烦。编号顺序和资源使用顺序不一致，导致资源浪费。（例如 1 号资源是最后所需要的，但我最先持有，就导致别的只需要 1 号资源的进程被长时间阻塞。）

3.死锁避免

系统安全状态的定义：

系统能根据已有的资源让进程按照任意一种顺序执行下去，并且顺利完成，这这种顺序叫做安全序列，如果系统无法找到一个安全序列，则称系统处于不安全状态。

注意死锁避免和死锁预防的区别，太重要了！！：

死锁避免是设计安全的执行顺序（银行家算法），来避免死锁。死锁预防是指破坏了四个死锁的必要条件。

不安全性状态和死锁的关系？

①不安全性状态可能会导致死锁，也可能不会导致死锁。**可能不会导致死锁的原因是**进程被提前终止释放了资源，或者进程在执行过程中主动释放了一部分资源，或者实际需要的资源数小于声明的资源数。

②处于安全状态，一定不会发生死锁。

③发生死锁，一定处于不安全状态。

银行家算法：

特点：

①著名的死锁避免算法。

②分配资源前先试探是否会导致系统不安全状态，若不会，则将资源分配给它。

③按照银行家算法，只要保证系统中进城申请的最大资源数小于或等于系统所拥有的 m 个同类资源，就一定存在一个安全序列。假设 n-1 个进程各自拥有一个资源，最后一个进程拥有 m 个资源，所以系统所需要的最大资源数是 n+m-1；

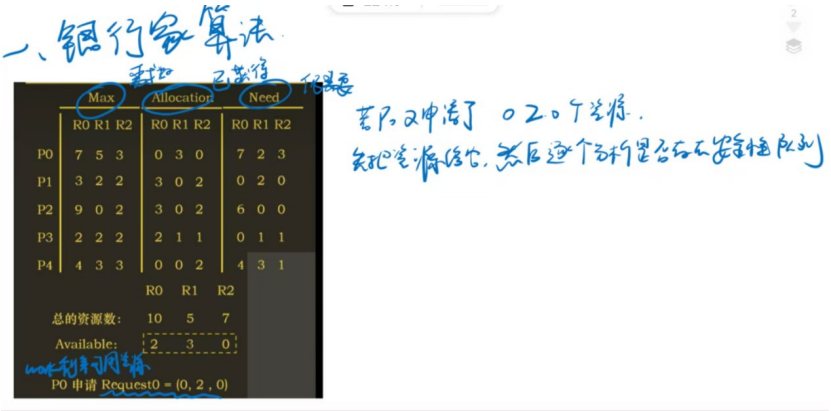
数据结构描述：

①可利用资源向量 Avaliable：含有 m 个元素的数组，代表每个元素 i，拥有 j 个可用的资源

②最大需求矩阵 Max：描述每个进程对每类资源的最大需求量

③分配矩阵 Allocation：描述给每个进程已经分配的每类资源数量

④需求矩阵 Need：描述每个进程还需要多少每类资源数量



4.死锁检测和解除

死锁检测和避免的区别：

死锁避免需要知道**整个生命周期的所有资源请求**，**死锁检测是检测某个时刻是否发生死锁**，只需要知道当前时刻的资源请求。

死锁定理的作用是帮助系统检测死锁的存在。

为什么死锁检测不关心进程运行所需的资源总量信息？死锁检测只检测系统中是否发生死锁，即循环等待的换路，一旦发现，就要采取接触死锁的措施，因此不关心各个进程所运行的资源总量

资源分配图（属于死锁检测）：

①表示方法：大圆圈表示进程，方框表示一类资源，小圆圈表示资源的数量。请求边表示进程申请某类的一个资源（还未得到），分配边表示该类资源已经拿出一个分配给某个进程。

②简化资源分配图：首先找到一个不阻塞也不孤立的进程，让它执行完毕，随后将它所链接的线撤销掉（意为移出去整个系统），紧接着对下一个进程进行同样的操作。如果直到最后还有边相连的进程，即发生了死锁。

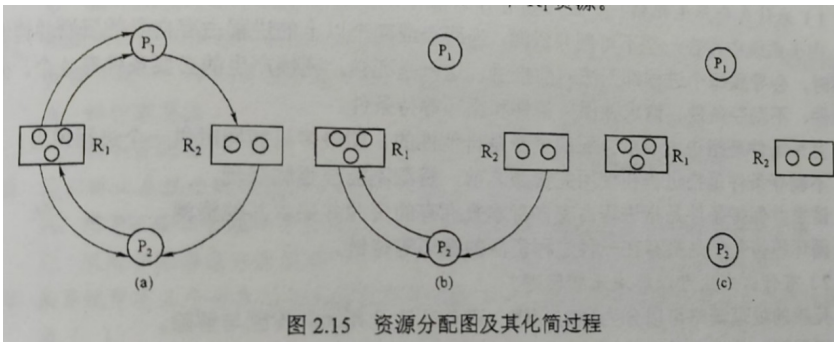


图 2.15　资源分配图及其化简过程

死锁解除的三种方法:

①剥夺资源

②撤销进程：可以撤销部分或者撤销全部；可以根据优先级撤销；代价较大

③进程回退法：让死锁进程回退到避免死锁的底部，这个过程中，进程资源释放资源而非被剥夺；另外还要求进程保留历史信息，设置还原点。

三、内存管理

(一)内存管理基础

1.内存管理的基本概念

逻辑地址空间与物理地址空间的地址变换，内存共享，内存保护，

内存分配与回收

内存管理的定义：

操作系统对内存的划分和动态分配，内存管理的具体机制是透明的，看不见的。

内存管理的五种功能：

①记录已分配的内存空间和空闲空间，回收进程结束后的空间。

②逻辑地址转为物理地址

③虚拟存储技术实现内存扩充

④内存共享，允许多个进程之间访问同一个内存空间

⑤存储保护，防止进程之间干扰各自的内存空间

逻辑地址与物理地址的转换：

编译→链接→装入

程序编译后，将代码的各个部分行程不同的模块，每个模块地址从 0 开始。

链接时将各个模块链接起来，形成一个新的地址空间从 0 开始的大模块，对于 64 位机器，逻辑

地址为 0^{64}～1，这就叫逻辑地址空间。

操作系统通过 MMU 部件将逻辑地址转为物理地址的过程称为地址重定位。

用户源程序到内存中可执行程序的过程：

①编译：将代码各个部分编译成若干目标模块

②链接：将目标模块链接起来，链接之后形成最终的逻辑地址

③装入：由装入程序将装入模块装入内存运行

三种链接方式：

①装入内存前静态链接：多个目标模块和库函数链接成一个完整装入模块。需要修改模块内相对地址，变换外部调用符号（函数），将每个模块所用的外部调用符号（函数）都变换成相对地址，以便在内存中被调换位置时，还可以通过相对地址正确调用函数。

②装入时动态链接：边装入边链接（需要用到模块时才将他调入内存并链接），便于修改和更新（链接前单独的模块的大小比较小，便于程序员修改，链接后模块较大也不利于解析并重构修改）、便于对目标模块实现共享（模块小，在内存中调入一个 start.so 文件模块，就可以供很多文件共享链接）

③运行时动态链接：边运行边链接，运行时才进行链接，加快了程序的装入过程，节省内存。链接过程快，但是执行效率低。（分段存储管理有利于动态链接，方便把不同的段放入磁盘，需要哪个段才引入哪个段）（页式虚拟存储的程序是在运行时动态链接的，因为在运行时需要什么页面才把页面调入内存）

基本分页存储管理系统适用于静态链接或者装入时动态链接、请求分页存储管理系统适用于运行时动态链接
YYK
2026/07/17 16:42

动态链接：是指装入时或者运行时才链接程序模块，而不是在装入前把所有程序的模块全都链接一起

程序的动态链接与程序的逻辑结构有关，通常链接的都是一个个模块，所以分段更适合动态链接。（固定式、可变分区都不适合动态链接）

相对来说，分页也适合动态链接
YYK
2026/07/16 16:07

三种装入方式：

①绝对装入：单道程序环境，逻辑地址=物理地址，程序员写程序时就固定了绝对地址，编译汇编时期就绑定了运行时的地址。

②可重定位装入（静态重定位）：多道程序环境分区存储，分配完整的连续全部内存空间，否则不装入，装入后固定位置不移动，只要简单的加法器就能实现静态重定位。（页式存储方式就不适合静态重定位，因为页式不连续而且内存位置动态变化）

③动态运行时装入（动态重定位）：装入内存后程序地址仍为相对地址，需要重定位寄存器帮忙，运行时才转为绝对地址。优点是：分配不连续的内存空间、动态申请空间、部分代码调入即可运行、便于程序段的共享。（动态重定位不依赖目标程序是指完全由操作系统和硬件组成，地址转换过程是透明的，地址重定位只依赖于基址寄存器、可重定位装入程序、重定位寄存器、地址变换机构 MMU。）

静态重定位也限制了单一基址转换，即每个程序只能有一个基址，但是分段存储管理方式中，每个段都有一个基址，所以静态重定位并不适合分段
YYK
2026/07/16 15:33

这个还是第一次了解到
YYK
2026/07/16 14:58

注意三种装入方式的时期
YYK
2026/07/16 14:55

重定位寄存器，在整个系统中只有一个，在发生进程切换时，切换重定位寄存器中的内容即可，为每个进程都安排一个重定位寄存器还是太奢侈了

重定位寄存器也不是一个寄存器，而是一组寄存器，包括界地址寄存器和基地址寄存器。物理地址 = 基地址 + 逻辑地址，界地址只是用于检测越界的
YYK
2026/07/16 15:11

为什么不依赖目标程序呢？目标程序是指经过编译、汇编之后的可执行的二进制机器码程序，不依赖是指不需要修改该文件，与该文件内容无关，只需要操作系统和 MMU 协力进行地址转换即可
YYK
2026/07/16 15:17

重定位寄存器如何帮忙实现装入内存后程序地址仍为相对地址？

重定位寄存器保存的是进程的基地址，整个系统中只有一个重定位寄存器，每次切换进程时，顺带切换重定位寄存器的值即可。

为什么不仅仅使用加法器就行，还要去使用 MMU，它有什么额外的功能吗？

MMU 在加法器的基础上还实现了其他功能

功能需求	简单加法器能否实现?	MMU 如何解决?
非连续存储（如页式）	不能	通过页表查询页框号，支持分散地址映射
内存越界检查	不能（需软件配合）	硬件级检查（对比界限寄存器或页大小）
权限控制（读 / 写 / 执行）	不能	页表项中存储权限位，MMU 硬件校验
虚拟内存（缺页处理）	不能	检测缺页并触发中断，配合 OS 完成页交换
进程隔离	不能	独立页表 + 进程切换时更新页表基址
动态地址映射（程序移动）	不能	动态更新页表 / 段表，无需修改程序代码

进程的内存映象五个要素：

①代码段：只读文件，可被多个进程共享（调入内存就确定了大小）

②数据段：全局变量和静态变量（调入内存就确定了大小）

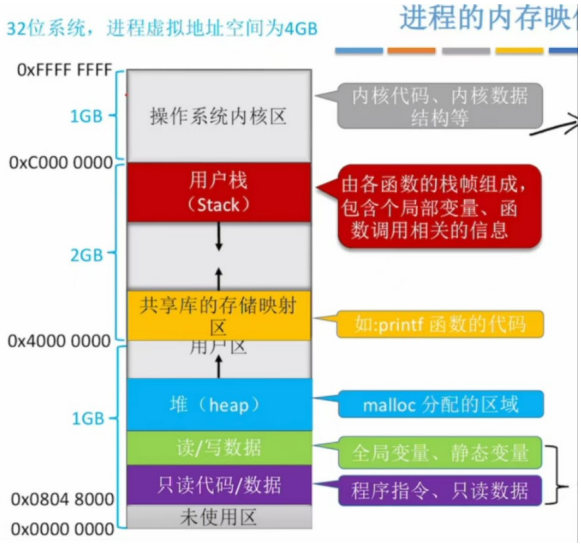
③PCB：存放在系统区

④堆：动态分配的变量，malloc 函数向高地址空间分配（内存空间动态变化）

Malloc 函数返回的是虚拟地址

YYK
2026/07/17 17:34

⑤栈：函数调用，从最大地址向低地址增长（内存空间动态变化）



内存保护 （防止越界访问，需要操作系统（负责制定保护规则）和硬件（负责执行和检测，例如 MMU\页表基址寄存器、异常中断硬件，发生越界了就需要中断处理）共同实现）：

①上下限寄存器，存放进程在内存中的上下限地址，CPU 判断进程访问地址是否越界。

②第二种方式是界地址寄存器+重定位寄存器，首先进程给出相对地址，与界地址寄存器（存放的进程最大逻辑地址，也是进程的最大长度）比较，不超过后，才能与重定位寄存器（存放的是进程的首地址）相加，得到物理地址。（重定位寄存器也叫基址寄存器，界地址寄存器也叫限长寄存器）

再次深入理解何为越界访问：
1. 本进程内的虚拟地址空间有些是已经分配的、有些是未分配的，访问未分配、或者没有权限的的虚拟地址空间就是越界访问。
访问超过了虚拟地址空间大小的地址，也叫越界访问。
由于每个进程都有一个页表，页表记录的是虚拟地址到物理地址的映射，但是有些恶意程序篡改了页表，导致该进程访问了其他进程的物理空间，这也叫越界访问。
YYK
2026/07/16 17:52

内存共享：

①只读区域才可共享

②可重入代码：纯代码、允许同时访问但是不允许任何修改，给每个进程配备了局部的数据区，存储可能临时修改的数据

内存分配与回收：

单一连续分配（单道程序，独享内存，内存利用率低）进化为固定分区分配（多道程序，内部碎片，分区固定，无法容纳更大的程序）

固定分区分配进化为动态分区分配（多道程序，外部碎片，根据内存动态分配空间，管理复杂）

连续分配（单一连续、固定分区、动态分区）进化为离散分配（段、页、段页结合）

外部碎片的回收可以通过紧凑技术实现，将每个进程向低地址空间移动，在高地址空间省出大量连续空间。但是需要动态重定位寄存器（界和限寄存器），移动费力，大量耗时。

2.连续分配管理方式

单一连续分配	固定分区分配（早期使用静态重定位）
①分为系统区（大小固定）和用户区（被一个进程独占） ③优点是：简单、无外部碎片、不需要内存保护 ④缺点是内部碎片、单用户单任务、内存利用率低	①每个分区只装入一个作业，即使不占满，也只能装一个 ②可分为固定大小分区和不固定大小分区（多个较小分区，适量中等分区，少量大分区），还采用了分区使用表便于分配与回收。

这个王道讲的比较模糊，实际上，线代操作系统早就实现了动态重定位的固定分区分配。

当静态重定位时，也就没有必要借助 MMU 来计算地址了，直接简单的加法器就可以
YYK
2026/07/16 15:08

为什么单一连续分配和固定分区分配是由**装入程序和操作系统**来完成地址转换，而不是硬件设施完成？因为单一连续分配逻辑地址与物理地址一一对应，无需变换（最多计算一下物理地址=用户区基址+逻辑地址），固定分区重定位直接把程序内的逻辑地址改为物理地址（**逻辑地址** + **分区起始地址** = **物理地址**），不允许动态改变

什么是内部碎片和外部碎片？内部碎片是相对进程来说的，进程大小小于页的大小，就产生了内部碎片。外部碎片是因为这个存储区太小，无法分配给任何一个进程。

分区号	大小（MB）	起始地址（M）
1	2	8
2	2	10
3	4	12
……	……	……

分区使用表

③缺点是不能实现多个进程共享一个主存区，内存利用率低、内部碎片、程序太大有可能放不进任何一个分区、小程序占用固定分区造成内存浪费。

④**分区存储管理是代价最小的存储管理方式（在多道程序的基础之上）**，因为他不需要特定的数据结构和硬件支持，适合嵌入式等微型设备

动态分区分配（动态重定位，需要 MMU 硬件支持）（随着时间推移，外部碎片越来越多）

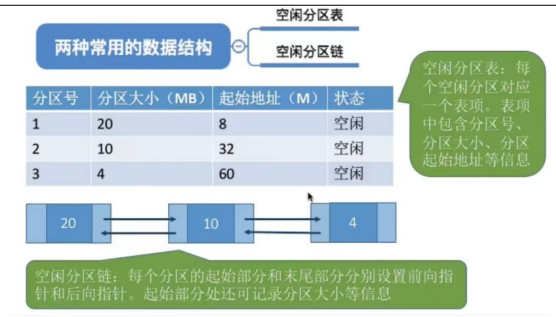
（程序装入时动态建立的）

动态分区一个非常重要的地方：它会在程序运行时，可能会触发空闲分区合并，这就导致了静态重定位 ----- 装入内存后物理地址永久保持不变 ---- 不使用，所以静态重定位并不适合动态分区分配
YYK
2026/07/16 15:30

操作系统要使用什么数据结构来记录已分配的分区数据？

①空闲分区表

②空闲分区链



空闲分区链是双向的指针

操作系统要使用什么内存回收方法？

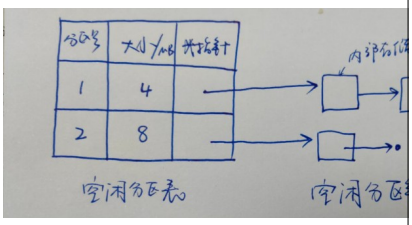
四种情况：着重说下列两个情况

①当回收的内存前后都有空内存，则三个空闲区进行合并，取消后面的分区的表项。

②当回收的内存前后都没有空闲区，则新建表项，插入空闲分区链。

操作系统要使用什么内存分配算法？

基于顺序搜索的分配算法（小型系统，因为顺序搜索太慢）	基于索引搜索的分配算法（大中型系统）
	对每类（大小相同）的空闲分区设置一个空闲分区链，设置索引表管理空闲分区链，顺序搜索索引表，找到对应分区的索引头指针，根据头指针找到该类分区的空闲分区链



首次适应 First Fit	邻近适应	最佳适应 Best Fit	最坏适应 Worst Fit	快速适应	伙伴系统	哈希算法

最佳适应算法：
最容易产生一些无法利用的碎片，因为假设分区大小为30.1KB，进程所需内存为30KB，那么会按照最佳适应算法把这个分区分给他，只剩下0.1KB无法利用的空间，可以预见的是，这个0.1KB几乎不会被用到了。

首次适应算法和最坏适应算法也会产生内存碎片，但这些碎片平均大小要比上边那个大的多，会有更大的概率被利用。
YYK
2026/07/16 19:47

快速适应算法，顾名思义就是快！也是基于索引的，记得是把相同大小的空闲分区穿成一条链子，找到第一个合适的链子，取出合适的分区。

但是要注意，每一条链子上的所有分区大小不都是一样的，这个每条链子上的分区大小都是预先设定的，例如4KB、8KB、16KB..... 如果产生了3KB的分区，也只会被放入到4KB的链子上，而不是单独创建一个3KB的链子。

如果同时存在两个3KB的链子，也不会被合并6KB
分配时不用遍历所有分区，导致“小分区难以被充分利用”（浪费内存）——这正是快速适应算法“空间换时间”的核心矛盾。
YYK
2026/07/16 15:43

	（也叫循环首次适应）Next Fit				（2024 真题）	
空闲分区按照地址递增顺序排列，进程找到第一个满足条件的空闲区。	从上次查找结束的位置继续查找（包括自己）	将分区按照大小排列，选择第一个能满足进程大小的空闲区。	将分区从大到小排列，选择最大的空闲分区，割一部分。	找到索引表中能容纳它的最小空闲分区链表，取出链表中第一块分区	规定所有分区大小均为 2 的 k 次幂，若需要大小为 n 的进程空间，$2^{i-1}<n\leq 2^{i}$，先在 2^{i} 空闲分区链中找，若找到直接分配，若没找到则去 2^{i+1} 空闲分区链中找，若找到，则一分为二，若没找到则去 2^{i+2}	根据空闲分区表的规律，建立哈希函数，构建一张以空闲分区大小为关键字的哈希表，每个表项记录空闲分区链中的头指针，通过哈希函数来计算哈西表中的位置

					空闲分区链中找。	
优点是高地址保留大内存块。开销最小，性能最好。	低地址和高地址同等概率被分配。相比于最佳和最差适应算法，其不需要对分区大小进行顺序排列	**最佳**，留下大空闲分区	不容易产生碎片	查找效率高，不会产生内存碎片		
缺点是低地址存在小碎片，每次都需要经过这些小碎片并比较，增加开销。	导致高地址也被划分，无大分区可用，通常比首次适应算法**更差**	留下最多的内部碎片，**需要每次排序，性能很差**	导致无大空闲区可用，需要排序，性能很差	回收分区时需要进行有效合并，算法复杂，系统开销大	会收拾需要将相邻的空闲伙伴分区（这两个相邻的伙伴大小必须相等，2024 真题）合并成更大的分区	

3.页式管理

（引入页式存储器目的是实现**离散分配**和提高内存利用率）

一些基本概念：

①物理空间和逻辑空间都被划分成大小相等的页（页面），因此分页管理不产生外部碎片。（分页是由操作系统完成的，大小一定是固定不变的）

②主存空间中的页（也叫物理页、实页、页框），对每个实页进行编号叫页框号（物理块号）

③虚拟地址空间中的页（也叫虚拟页、虚页），对每个虚页进行编号叫页号

④页面大小最好是 2 的整数次幂，页面太小会导致页表太长、占用内存、增加硬件地址转换开销、降低页面换入换出的效率，页面过大导致页内碎片增多，降低内存的利用率。

⑤通过硬件实现，对用户完全透明（包括页号和页内偏移，都是透明的）

⑥页面在物理内存中只能从页面大小的整数倍地址开始存放！！

分页逻辑地址结构：

页号 P + 页内偏移量

页表（页面映射表）：

①每个进程一个页表，这个页表驻留在内存中，轮到哪个进程执行了，进程 PCB 里的页表起始地址和页表长度装入页表寄存器。

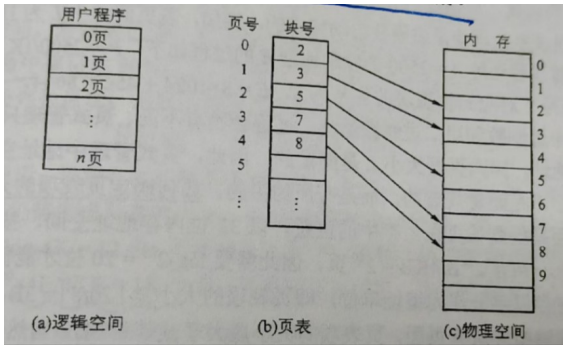
为什么页表需要常驻内存？

让页表常驻内存可以加快虚实地址转换，因为 CPU 去找这个虚拟地址对应的物理地址，就在内存中，直接完成地址转换，顶级页表、二级页表都是通常情况下常驻内存，但是也可能会被调

出到磁盘里面，等到地址转换的时候发现页表不在了，会触发中断，把页表调入内存然后执行即可。

逻辑页号与物理块号的映射关系

在页表中，**页号（页号就是虚拟页号！！）**按照顺序，所以是隐含的，可以不占用存储空间。



分页基本地址变换机构：

①CPU 给出逻辑地址拆分成页号 P 和页内偏移量 W

②**首先要进行越界检测**，检测页号是否大于页表长度，若是，进入越界中断（异常）。

由 MMU 来实现

YYK
2026/11/16 17:49

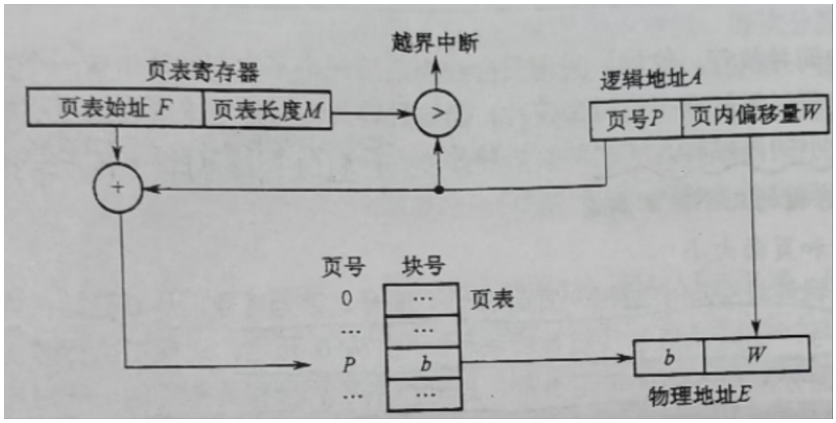
③将逻辑页号加上页表起始地址 F 去找页表中的映射表。

④物理页号加上页内偏移地址=物理地址

注意点：

① 在多道程序并发执行时，进程的页表大多数存留在内存中，**被长期阻塞的进程的页表可能会被调出到磁盘中**，单 CPU 系统只有一个页表寄存器 PTR（意思是多 CPU 中有多个 PTR，**PTR 当中存放的是页表的物理起始地址**），在进程未被执行时，起始地址和页表长度存放在 PCB 中，运行时页表的起始地址存放在页表寄存器 PTR 当中（利用寄存器可以加快转换速度）

②查询页表的这一过程是由硬件 MMU 来完成的，界比较、拼接、表项比较



页表项的大小：

①页表项的目的是找到该页在内存中的位置。

②页表项仅仅存储物理块号，由于物理块号与内存大小息息相关，以 32 位内存地址空间为例，代表内存大小为 2^{32}B，一页是 4KB 大小，那么就会有 2^{32}B/4KB=2^{20} 页，所以需要 20 位才能容纳一个物理页号的地址，同时由于内存是以字节为单位，所以页表项的大小大于等于 3B。

在数据交换场景中，对主存的访问以字节为单位（这是 CPU 与主存交互的基本粒度，比如读取某个变量的一个字节、两个字节等），但虚拟内存与主存之间的页面调度（页面的调入 / 调出）是以页面为单位。

主存与磁盘之间的数据交换是以磁盘块为单位
YYK
2026/07/16 16:32

分页管理存在的两大问题以及相应的解决方案？

主要是地址转换效率和页表大小两个问题

①每次进行地址转换都要在内存的页表中逐个查找对应的页表项，如果页表多，页表项多，查询效率较慢。（引入快表）

②如果逻辑空间较大，则页表项的数量较大，导致页表所占内存特别大（页表还必须连续存放，显然占这么大的空间不现实），有时候也没必要把进程的所有页表全都导入内存，影响内存利用率，也增大了系统开销开销。（引入二级页表）

具有快表的地址变换机构：

概念：

①若页表全部在内存里，则取数据或指令需要一次或者两次访存，首先访问 CPU 中的快表，取出数据或指令的物理地址，然后是根据物理地址从内存中取出数据或指令

②快表存放在相联存储器。

③地址变换机构有了新的进步，即现在快表中查询是否有命中，有命中则直接取出物理地址，然后访存一次取出数据或者指令。若不命中，则再去内存的页表中找，取出数据，访存两次，并且把页表项（而非页表）调入快表。

④快表的命中率一般在 90%以上，利用了局部性原理。

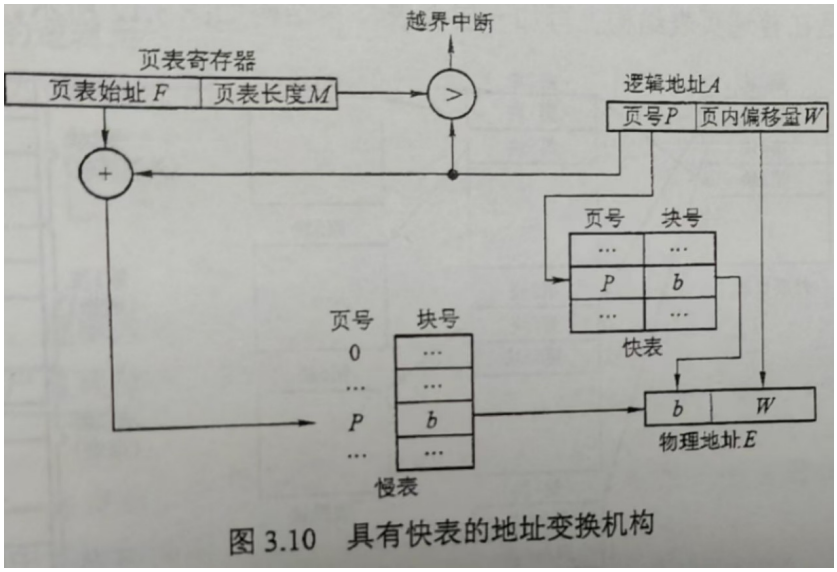
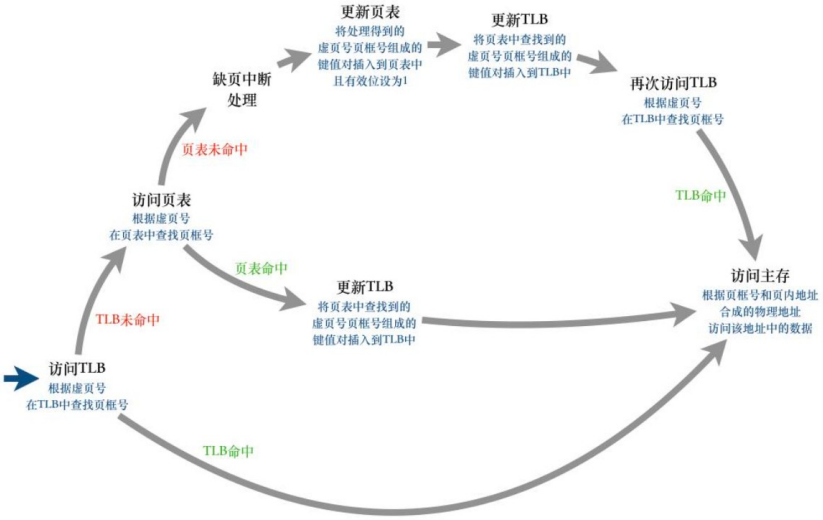


图 3.10 具有快表的地址变换机构

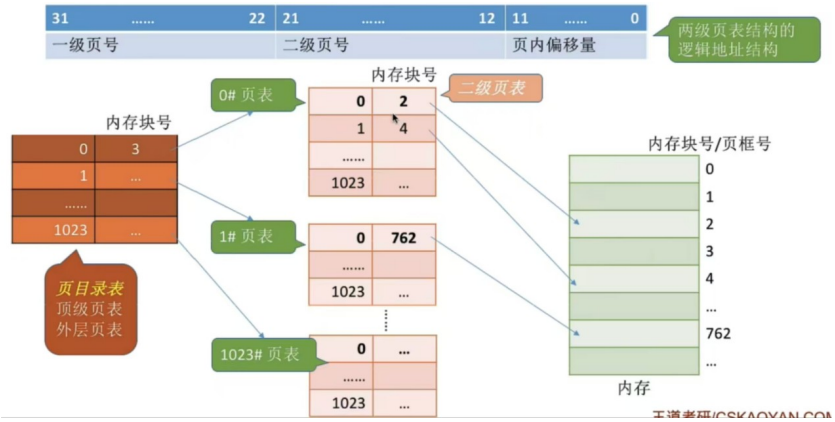
为什么提高页面大小可以提高快表的命中率？提高页面大小可以减少页表项，从而让一定容量的快表中覆盖更多的页表项，**另外采用多级页表改变不了快表的命中率**



两级页表：

解决上述问题的方法有两种，一种是引进二级页表（相当于建立索引），一种是引进虚拟内存技术（将需要的页表导入内存，不需要的页表放在磁盘中。）

下列方法是二级页表，即把页表项也存入一个页表，把不同的页表的页表项整理成一个顶级页表，从顶级页表中去找想要的页表项的页表。



地址转换过程：

①在系统中增设外层页表寄存器（也叫页目录基址寄存器）（一旦出现页目录号，就说明一定是多级页面存储方式！！！！2018 真题）存放顶级页表的物理地址。

②通过页目录基址寄存器中的起始物理地址，根据一级页号，在顶级页表中找到对应的二级页表的页表起始物理地址。

③通过二级页表的起始物理地址地址，根据二级页号，对应找到物理页号映射关系。

④通过物理页号（实质上还是一个物理地址，只不过每隔 4KB 进行一次编号）在内存中找到对应数据或指令。

两级页表的计算问题：

【例】页式存储管理系统，逻辑地址空间 256TB，页表项大小为 8B,页面大小为 4KB，该系统应该采用几级页表？

主存以字节为存储单元，256TB=2^{48}个存储单元，页面大小为 4KB，即一个页面需要 2^{12}个存储单元，共有 2^{36}个存储页面，一个页面可以存储 2^{9}个页表项，则共需 $2^{36}/2^{9}$=4 级页表。

4.段式管理

一些基本概念：

①一个进程只分配一个段表（分清楚段表和段表项的概念，每个段拥有一个段表项，所有段表项构成一个大的段表）

②段内要求连续，段间不连续，进程的地址空间是二维的（段号，段内偏移）。

③分段对低级语言程序员和编译程序都不透明，他们都知道分段的逻辑关系，低级语言程序中段号和段内偏移都由用户提供，高级语言程序由编译程序完成。

④段号和段内偏移量由用户显式提供（在高级语言编程中，由编译程序给出段号，程序员通常不知道如何分段）

⑤用户编程时（汇编语言）决定的如何分段

⑥分段也便于动态链接和动态增长

这个增长有两方面，一方面是段内的数据段可以增加，比如数组长度增加了；另一方面也可以动态链接一些新的模块，那么段也就增长了
YYK
2026/07/16 16:30

引入段式存储的目的是满足用户的需求，哪些需求？方便编程、分段共享、分段保护、动态链接和增长。程序运行时动态链接共享库的代码段，而非编译时静态绑定，减少内存占用并支持库的独立更新，递归函数调用导致栈空间不足时，操作系统自动扩展栈段。**（实现离散分配并不是分段的目的，而是分页的目的。）**

逻辑地址结构：

段号 S+段内偏移量 W

段表：

段号+段长+本段在主存的起始地址（段表项连续存放，所以段号可省略）

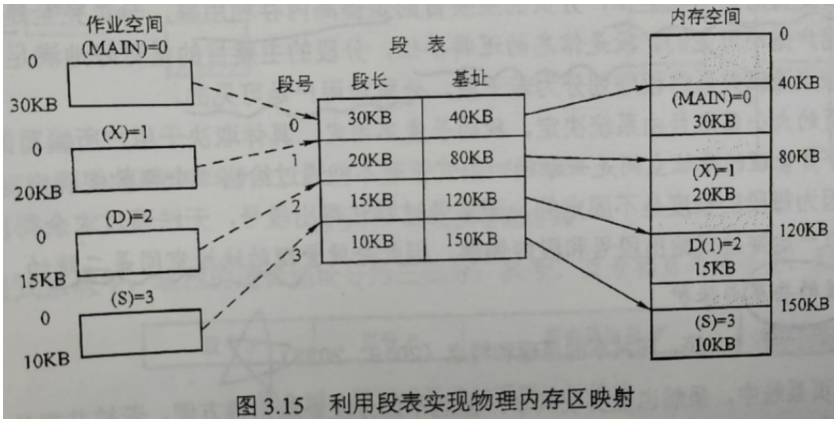


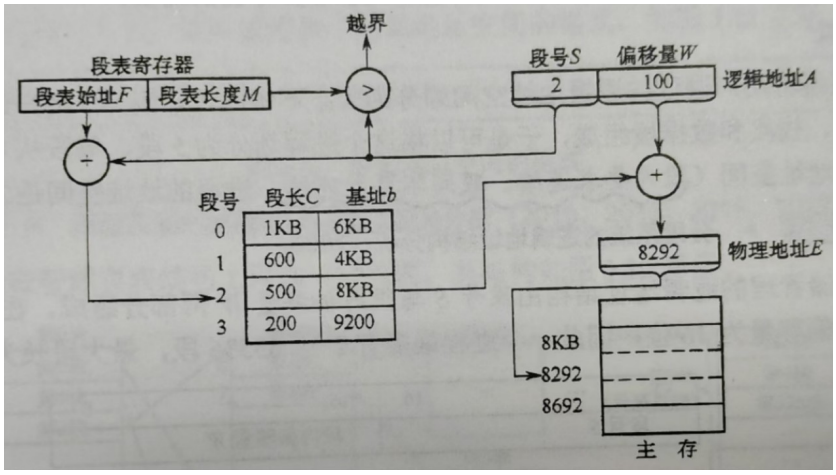
图 3.15　利用段表实现物理内存区映射

分段基本地址变换机构：

①系统设立了一个段表寄存器，记录段表的起始物理地址

②首先取出段号，与段表长度进行比较，若不越界，则根据段表起始地址 + 段号找到对应段表项。

③根据段表项找到段长和物理起始地址，再加上段内偏移量得到最后的物理地址。



2009 真题，我自己搞错了概念，一旦有地址长度 32 位，这表示逻辑地址和物理地址的长度，**与段表的段表项长度无关**。所以逻辑地址总长 32 位=段号+段内偏移，段号占 8 位，段内偏移最大值就等于最大段长

分段与分页的对比：

①页是物理单位，是为了提高内存利用率的系统行为；段是逻辑单位，满足用户需求的用户行为。

②页长固定（系统决定）和段长不固定（代码决定）

③分页管理地址是一维的，分段管理地址是二维的。

段的共享：

①页也可以实现共享，但是实现过程极其复杂，因为页表有可能被分散在内存的不同区域，联构起来比较麻烦。

②段内连续，所以可以按照段进行共享，例如共享代码段。

③系统配置共享段表（实现多个进程共享一段代码或一段数据），不同进程使用共享段表中同一个段表项，这个段表中的段表项，指的是某个代码段、数据段（2019 真题）

④某个段被不同进程共享，在进程的段表中是不具有相同的段号的（他们的段号是没有任何联系的）（2019 真题）

段号	段长	内存始址	状态位	外存始址	共享进程数
				共享段未被加载到内存，记录其	记录多少个进程共享。当 count=0 时，回收内存。

				磁盘地址	

段的保护：

①段共享时，共享的代码都是可重入代码（不能被任何进程修改），所以每个进程必须配备局部数据区，便于进程修改数据。**段共享时配置的是共享段表**

②存取控制保护。

③地址越界，需要检测两方面。一方面是检测段号是否大于段表长度，另一方面是检测段内偏移是否大于段长。（页内偏移是不可能大于页长的，因为页内偏移位数最大就等于页长）

5.段页式管理

一些基本概念：

①先分段，再分页。

②每个进程一张段表，每个段一张页表（因此段页式管理每个进程可能有多个页表）（页表、段表与上述知识相同）

③系统含有一个段表寄存器（指出段表起始地址和段表长度）

④单独采用分页和分段管理，提供给用户的物理地址空间要减去页表和段表占的空间，因为段表和页表所占空间是不能确定的，所以提供的用户物理空间大小也就不能确定。

为什么段页式分区管理会产生内部碎片？如果一个段是 5KB，一个页是 4KB，那么就需要给这个段分两个页，那么就会有页内的内部碎片 3KB。

逻辑地址：

段号 S+页号 P+页内偏移量 W（段号可隐含）

段页式基础地址转换：

①首先通过逻辑地址分出段号 S，先进行越界判断，若无越界，则与段表起始地址相加得到段号，根据段号再段表中找到页表长度和页表起始地址。

②取出逻辑地址中的页号+页表起始地址=逻辑页号的地址，找到物理块号的映射关系，物理块号+页内偏移量=物理地址

(二)虚拟存储管理

1.虚拟内存基本概念

（内存空间扩充有三大技术：覆盖、交换和虚拟内存）

1.传统存储方式的两大特征

①一次性：0 限制不能让大量作业同时运行，并发度下降。

②驻留性：装入内存，如何部分都不会被换出，作业进程结束才调出，甚至遇到 IO 阻塞也不会调出内存

2.局部性原理

快表、页高速缓存、虚拟内存技术都属于高速缓存技术，他们都依赖了局部性原理

①时间局部性：之前用到的数据马上要再使用，例如循环操作 i=i+1

②空间局部性：下一个使用的数据与该数据空间相邻，例如数组，直接从相邻的缓存中取出数据

局部性原理和高速缓存技术有什么联系？时间局部性将近来、使用的指令和数据保存在高速缓冲中，空间局部性是将数组预取机制，需要更大的缓存器

虚拟存储器的三大特征：

①多次性：分多次将页面调入内存

②对换性：不用的页面调出内存

③虚拟性：逻辑上扩充了内存

这些技术都是认定了程序具有时间局部性和空间局部性的基础之上，被研究设计的新技术。而不能认为是程序经常访问之前访问到的数据，而具有局部性原理，这样是错误的，而应该认为是局部性原理就是程序的一个固有属性

YYK
2026/07/17 14:37

快表、页高速缓存、虚拟内存技术如何依赖局部性原理？
快表是指经常被命中的表的表项，方便直接从 CPU 中获得表的物理地址，而不用访问内存中的慢表。所以利用时间局部性原理，这个数据可能经常被访问。
页高速缓存技术是指将磁盘中的数据页缓存到内存中，他跟虚拟内存技术还是有区别的，这样方便以后直接从内存中取出数据，利用了空间局部性和时间局部性。
虚拟内存技术专注于构建内存 - 外存的两级存储结构，基于局部性原理，知道程序在某一时间段内主要访问的是**哪些页面**的信息，**所以只需调入一小部分页面，这就叫基于局部性原理**，把常用的页面调入内存，不常用的调入磁盘

YYK
2026/07/17 14:29

这个对换、调入对用户来说都是透明的

YYK
2026/07/17 14:47

虚拟存储器的容量大小：

进程的虚拟地址空间寻址由系统的地址结构决定，这决定了虚拟存储器的最大容量，与内存和外存容量无关，若磁盘空间无限大，作业也不能拥有任意大的编制空间，因为受限于硬件设施，比如 CPU 地址总线位数

虚拟内存的实现方式与技术支持：

实现方式有请求分页存储管理、请求分段存储管理、请求段页存储管理（**自己犯了误区，以为只有请求分页存储管理**）

为什么没有那些连续分配的内存管理呢？一整个分区全都调入内存，根本就没有意义了
YYK
2026/07/17 14:47

技术支持硬件：

①一定容量的内存和外存（ROM 和 RAM，磁盘）

②页表机制（段表机制），作为数据结构，MMU+寄存器的硬件支持

③中断机构：缺页中断机构硬件（因为是由硬件检测到缺页中断信号、触发缺页中断机制）支持

④地址变换机构：逻辑变物理，地址变换机构 MMU 硬件

MMU 本质上也是 CPU 的一个组件，是它来访问页表、段表的结构和内容的，一般都是提前构造好的每个结构占多少位，然后取出相应的位数进行比较、检测
YYK
2026/07/17 14:55

⑤查询快表，快表 TLB本身也是硬件

快表本质上就是 Cache ，由 SRAM 组成， SRAM 具有速度快的特点，能够实现快速的数据读取和写入操作。
YYK
2026/07/17 14:54

⑥查询页表，先要找到页表的初始地址（存放在页表基址寄存器、段表基址寄存器中，这也是硬件）

2.请求页式管理

请求分页系统=基本分页系统+请求调页（外存调入内存）+页面置换（内外存页面调换）

基本分页系统本质就是把一整个程序的所有内容分成了若干页面，然后把这些页面分散存放在内存。

但是请求分页系统是利用了局部性原理，只调入当前可能用到的页面，其余页面用到时才调入内存
YYK
2026/07/17 15:00

请求页式管理和基本页式管理有什么区别？基本分页是静态、简单的内存管理方案，适合资源受限或实时性要求高的场景，但牺牲了内存灵活性和利用率。请求分页通过按需加载和虚拟内

存机制，实现了动态内存分配和高资源利用率，**是现代操作系统的基石**，尽管引入了缺页中断的开销。

页表机制

请求分页系统的页表项结构：

逻辑页号	物理块号	状态位 P	访问字段	修改位 M	外存地址
		是否调入内存	访问次数、最近已经多长时间未被访问	是否被修改，决定是否写会外存	通常是物理块号（磁盘标识）

这里的表述物理块号，通常也把外存划分成物理块（页）了，这里指的不是内存的物理块号
YYK
2026/07/17 15:07

页表机制里面已经有了物理块号，为什么还要专门设置一个外存地址数据？页表中的物理块号是内存的划分单元，该数据存在与否，取决于页面是否在内存中。当页面不在时，就需要外存地址来找到它，这样在发生缺页中断时，操作系统才能从正确的磁盘位置调入页面。

缺页中断机构：

①页面不在内存，触发缺页中断，由操作系统的缺页中断处理程序处理，缺页进程阻塞，进入阻塞态，页面调入内存后唤醒，进入就绪态。

②若内存中有空闲页框，则直接调入内存。没有空闲页框，根据置换算法选择一个页面调出，并根据是否修改过来写回外存。

③需要保护 CPU 环境、分析中断原因转入中断处理程序、恢复 CPU 环境等步骤

④指令执行过程中发生缺页中断，属于内部异常（内中断），一次指令执行过程中可能发生多次缺页中断

例如数组 a[0][0] 和 a[0][1] 恰好处于两个页面的边界处，所以两次都缺页了，甚至会发生 100 次、 1000 次中断，例如一些链表结构的内容，需要从头到尾遍历找到想要的内容，而每一个链表原子内容都处于不同的页面，而恰好也都缺页
YYK
2026/07/17 15:12

地址变换机构：

在基本分页系统的基础上，增加产生和处理中断、页面置换的功能

主要过程如下：

①MMU 检测是越界

②访问 CPU 中的快表看是否命中

②若快表未命中，则访问页表，若页表命中，取出物理地址同时将页表项写入快表。

③若页表也未命中，则进行缺页中断处理（保护 CPU 现场、在外存中找到缺页、内存若满了则执行页面置换算法、原页面被修改了则写回外存、OS 命令 CPU 从外存中读入缺页、启动 IO 硬件、换入成功），操作系统更新页表和快表。

这个过程不能忘记，还是比较重要的，也稍麻烦了一点

2013 真题：已经发生缺页之后，不可能在进行处理越界错误了，越界中断检查是在刚开始就进行的，后续是不在进行的，越界中断的本质就是检查页号是否大于了页表长度，别的没有了！
YYK
2026/07/17 15:18

④物理地址=物理块号*物理页面大小+页内地址

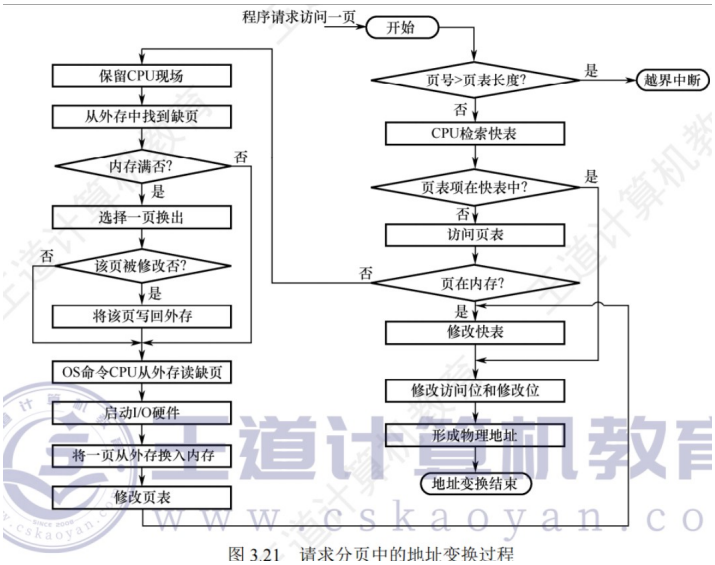


图 3.21　请求分页中的地址变换过程

3.页框分配

驻留集的定义：

给一个进程分配的页框的集合叫做驻留集。

而进程某段时间 t 间隔内，进程实际访问的页面的集合被称作它的工作集（若工作集窗口大小为 6，则过去的 t 时刻的工作集集合包涵过去的六个页面，且包含重复的，**不需要因为页面重复而再往前推进若干个不重复的页面**）。

理想情况下，驻留集要包含工作集，这样回有效避免缺页，但是实际情况下，工作集并不是驻留集的子集，有可能工作集需要的页面没有在驻留集内，这就发生了缺页，也有可能驻留集的部分页面不是工作集所需要的页面。

驻留集太大，内存浪费，并发度下降；驻留集太小，驻留在内存中的进程会更多，但缺页率高，消耗大量 CPU 时间来处理缺页。缺页时，内存中可能有空闲页框，直接调入内存新的页面，不需要淘汰旧的页面

工作集并不是驻留集的子集，再次明确工作集的定义，是在过去某个时间间隔内，进程运行所需访问的页面的集合，例如 ACD ，但是此时的驻留集是 ABCE ，因为 D 已经被换出去了，明显的 ACD 不是 ABCE 的子集
YYK
2026/07/17 16:46

想一想缺页率是怎么计算的：缺页次数除以总的访问页面次数
YYK
2026/07/17 17:45

内存分配策略：

固定分配+局部置换： ①分配固定数量的物理块 ②从进程内部的页面换入换出 缺点是：难以确定固定数量的物理块，太少太多都不行。	固定分配+全局置换 全局置换意味着进程的物理块数必然改变，故该方案不存在！
可变分配+局部置换 ①先分配固定数量物理块，若发生频繁缺页，则再次为其分配物理块，直至缺页率正常。若几乎不发生缺页，也可以剥	可变分配+全局置换 ①先分配一定数量物理块，根据情况可以增加或减少。 ②从全局的空闲物理块（不管是否已经被分配出去了）中选一个

夺部分物理块。 ②从进程内部换入换出 优点是：保证了并发能力，不会过多的调页。 缺点是：实现复杂，开销大。	分配给进程。 优点是：比较灵活，动态增加进程的物理块。 缺点是：可能会盲目给进程增加物理块，导致并发度下降。

局部淘汰策略：

只考虑当前进程所拥有的页面，即只在当前进程的合法驻留集（或称为工作集）中进行页面置换。（2009 真题）

采用固定分配策略的物理块分配算法：

①平均分配

②按进程大小比例分配

③按进程优先级分配。

有时候也会采用一部分按比例分配，一部分按优先级分配

页面调入的时机：

①预调页策略（进程运行前调入）：预测未来可能被调入内存的页面，**主要用于首次调入**，可以调入多页，程序员指出应该先调入哪些页。

②请求调页策略（进程运行中调入）：其实就是中断后，调入页面，一次调入一页，增加了磁盘开销。

从何处调入页面：

请求分页系统的外存分为两部分（对换区的磁盘 IO 速度更快）

①文件区-存放文件-离散分配方式

②对换区-存放兑换页面-连续分配方式

这里指的是在磁盘内划分一块连续的地址空间用于当做对换区，而文件区采用离散分配方式，找到一个完整的文件还需要磁头寻道、比较麻烦，故而IO速度更慢
YYK
2026/07/17 15:29

系统调入缺页就有以下三种情况：

①拥有足够的对换区：将整个文件全都复制到对换区，提高调页速度

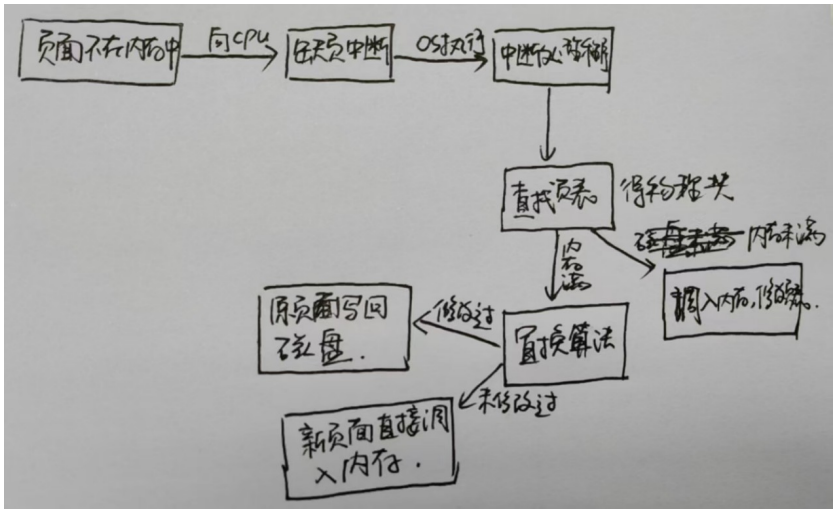
②缺少足够的对换区：凡是不会被修改的文件都从文件区调入，换出时因为没有修改所以不必写回外存。有可能修改的部分必须从对换区调入调出，因为读的速度比写的快，利用对换区的速度提高平衡一下写的速度。

③UNIX：进程有关的文件放在文件区，曾经运行过但又被换出的页面，放在对换区。进程请求的共享页面若被其他进程调入内存，则不需要再从对换区调入。

这两段需要多回顾一下

YYK
2026/07/17 15:33

如何调入页面：



是由缺页中断处理程序执行的，查找页表得到该页的物理块

4.页置换算法

（算法目的是追求更少的缺页率，缺页时不一定发生页面置换。缺页率等于缺页次数/总访问次数）（下列所有算法都有可能出现抖动）

最佳置换 OPT Optimal Page Replacement	先进先出 FIFO	最久未被使用 LRU Least Recently Used	时钟置换 CLOCK （NRU）	改进时钟置换 CLOCK
替换掉以后永不使用或者最长时间内不再使用的页面	淘汰最早进入内存的页面	最近最长时间未被使用。（注意与 OPT 的区别，OPT 是未来最长时间不会被访问）	首先进行循环扫描，找到第一个访问位为 0 的页面，扫描过程中顺便把扫描过的页面从 1 改为 0，那么即使第一轮全为 1，第二轮也一定会有访问位为 0 的页面。 最多进行两轮扫描	在原来基础上，添加一个修改位，因为未被修改的页面是不需要写会外存的，所以优先淘汰。一轮扫描先找到第一个未被访问过，且未被修改过的页面（本轮不进行修改），若找不到则第二

缺页时，也可能会有多余的空闲页框，所以直接调入即可，不需要页面置换
YYK
2026/07/17 15:34

这个缺页次数在王道上有个题，错了四次，总结来说就是在工作集为空时，缺页次数一定大于等于进程所需页面个数
YYK
2026/07/17 16:36

抖动的定义是：刚刚换出的页面马上又被调入内存，与驻留集大小密切相关。所有的算法都可能出现抖动！注意要和贝拉蒂异常区分开来
YYK
2026/07/17 15:55

				轮找未被访问过，但被修改过的页面（本轮的要把访问位为 1 修改为 0）；第三轮是找到第一个未被访问过且未被修改过的页面（本轮不进行修改）；第四轮是找到第一个未被访问过，但被修改过的页面。四轮之后一定可以找到该页面，即最多进行四轮扫描
优点是性能最好（顾名思义最佳置换算法啊！）	实现简单	实现简单，性能好	性能不错	性能不错 淘汰优先级是 00 > 01 > 10 > 11
缺点是无法实	没有利用局部	需要寄存器或栈	没考虑页面是否被	开销更大

总结来说，未经过操作的永远优先于经过操作的；访问位的重要性大于修改位的重要性
YYK
2026/07/17 15:52

现预测未来	性原理，导致频繁调入调出，性能差。可能导致 Belady 异常（分配的物理块更多，缺页次数反而越多了）2014 考题	的硬件支，跟踪访问，对页面进行排序，开销大	修改过	

没有利用局部性原理，也算是 FIFO 的一大败笔

YYK
2026/07/17 16:31

那就用一个 FIBA 来记忆这个 FIFO 和贝拉蒂异常的关系吧，FIBA（国际篮联）

YYK
2026/07/17 15:43

LRU 开销高的原因不在于需要寄存器和栈的硬件支持，而是需要对页面访问进行排序，开销大

可以使用纯软件实现如通过时间戳、链表，来实现对其排序。而用硬件电路的高速特性，替代部分软件操作，减少 CPU 的额外开销，例如通过一组寄存器来记录访问序号”，硬件自动更新访问序号，选择序号最大的换出即可，速度是软件的 100 倍，栈也可以实现这个功能。

YYK
2026/07/17 17:02

5.页框回收

页面缓冲算法：

影响页面换入换出效率的因素：

①页面置换算法，低缺页率，减少换入换出的开销

②将已经修改过的页面写回磁盘的频率，写速度太慢，影响速度

③将磁盘内容读入内存的频率。越低越好

页面缓冲算法=页面置换算法+已修改页面链表。

两种链表：

①空闲页面链表：读入一个页面，取出该链表的第一个页框并装入该页。当一个未被修改过的页面被换出时，实际上是将该页面所在的页框挂到空闲链表的表尾。（这些空闲链表中可能存在部分数据，当进程需要是，可直接读走，避免去磁盘读，减少页面开销）

②修改页面链表：将已修改的、需要写会磁盘的页面排成链表，达到一定数量，一起换出，减少页面换出开销。显著降低了页面换入换出的频率，磁盘 IO 开销减少。

（为什么页缓冲队列长度不影响缺页率，而会影响页面置换的速度？置换操作包括淘汰一个页面、将它写回磁盘，以及从磁盘加载新的页面。如果页缓冲队列长度较短，则会被更快的写回到磁盘中，使用时间比较长，但是里如果队列长度比较长，我可以等一等再一块写回磁盘，2022 年真题）

再次记忆一下什么叫页缓冲队列！
一些页面被淘汰后不立即释放其物理内存空间（或不立即写入外存），而是通过队列暂时缓存这些页面，以便在后续需要时快速复用，减少磁盘 I/O 和内存分配的开销。
YYK
2026/07/17 17:32

页框回收：

①并非所有页框都可回收，内核页框大部分都不能回收

②系统中设定一个守护进程 kswapd，当页框数低于某个阈值，就要启动进程回收页框。

③将物理页面换出的过程即为页框的回收。要区分页面置换和页框回收的区别，前者是磁盘和内存页面的互换，后者是将页面写会磁盘，腾出页框空间。

④回收页框也会进行空闲页框的合并操作

为什么空闲页框数不能为 0？因为内核可能需要请求空闲页框，若不能满足要求，会导致系统崩溃。

6.内存映射文件(Memory-Mapped Files)

内存映射文件的定义和概念：

①是操作系统给应用进程的一个**系统调用**，是磁盘文件部分或全部与其他虚拟空间之间建立映射关系，也是可以在内存中直接修改文件内容。

②只是建立了一个映射关系，程序员相当于把文件看成了大的数组，根据逻辑地址来对文件进行访问，而磁盘的读写由 OS 完成，对进程来说是透明的。

③映射文件时，不会读入文件的页面，只有需要访问时，才把该页读入内存，关闭文件后，OS

才把修改的页面写会磁盘。

④各进程对于该文件的虚拟地址都是相互独立的，没有什么关联

内存映射的两条好处：

①程序员编程方便

②多个进程共享文件（文件页面的修改也会被同步看得见）

7.虚拟存储器性能的影响因素及改进方式

分页系统性能的影响因素分析（缺页率受页面大小、分配给进程的物理块数、页面置换算法、写回磁盘的频率、程序的局部化程度）

这里指的是时间局部性和空间局部性，局部性越好，则存储器性能越好
YYK
2026/07/17 16:09

缺页率是主要因素：缺页率大，性能差，而缺页率又受到页面大小、分配给进程的物理块数、页面置换算法以及程序编制方式的影响。

①页面大小

②分配给进程的物理块数

③页面置换算法

④写回磁盘的频率（所以搭建了一个暂时存放不写回磁盘的链表）

⑤程序编制方式：如若数组按行存储，就要按行访问，否则缺页率高。

当磁盘交换区的利用率是 90%时，是因为频繁的换入换出，而使用更快的磁盘交换区毫无意义
YYK
2026/07/17 17:22

四、文件管理

(一)文件

1.文件的基本概念

文件定义：

①硬盘为载体、信息集合、多种类型（文档、图片、程序）

② 在系统运行时，以进程为基本单位进行资源调度和分配；在用户输入输出中，以文件为基本单位。

③文件必定包含分类和索引的信息、访问权限信息

文件结构（自底向上结构定义）：

①数据项：最低级的数据组织形式，基本数据项是数据中的最小逻辑单位，组合数据项是由多个基本数据项组成。

②记录：一组相关数据项的集合。用于描述一个对象在某方面的属性。

③文件：有结构文件和无结构文件。

文件属性（不同文件系统的差别很大）：

①名称

②类型

③创建者 ID

④所有者 ID

⑤位置：指向设备和设备上文件的指针（例如 C 盘 TEST 文件下的 xxx 文件）

⑥大小

⑦保护：文件保护信息

⑧创建时间：最后一次修改和最后一次存取时间

文件分类：

①性质用途：系统文件、用户文件、库文件（编程的共享模块，例如 math.h）

②数据形式：源文件（.cpp .java）、目标文件（编译后的二进制）、可执行文件（链接后的 exe，直接在操作系统上执行）

③存取控制：可执行、只读、读写文件

④按组织形式和处理方式：

	普通文件	目录文件	索引文件	特殊文件
核心作用	存储数据	（目录文件存放了直接子目录文件和直接子数据文件的目录元数据如名称、位置、大小）	加速数据检索的辅助文件，通过建立关键字与数据位置的映射提高查询效率。	用于设备驱动、进程间通信等。特殊文件包括设备文件（字符设备文件和块设备文件）和命名管

总结来说就是一个大文件，该文件里保存着该目录中所有子目录和数据文件的目录，这里一个文件夹就是一个目录文件
YYK
2026/07/18 14:44

这些文件的读取交给了设备驱动程序来执行

YYK
2026/07/18 14:50

				道文件等
操作	读写编辑	创建、删除子项	通过键值加速定位	实现设备驱动交互
示例	图片、文档	/home 文件	数据库索引，记录映射关系	命名管道,命名管道（FIFO）：进程间通信。 套接字文件：网络通信端点 符号链接

2.文件控制块和索引节点

文件控制块：

①一个文件拥有一个 FCB，一个 FCB 就是一个文件目录项，文件目录项的有序集合就叫文件目录。（**这里需要说明，FCB 方案和索引节点方案时两个不同的文件系统采取的方案。第一种是 FCB 当做文件目录项，第二种是文件名+索引编号=文件目录项**）

②一个文件目录也被视为一个文件，称作目录文件。

③典型 FCB 示例：

	文件名
	物理位置

在一个文件被用户进程首次打开的过程中，操作系统会为该文件分配相应的文件描述符和数据缓冲区。然而，操作系统并不会直接将文件的数据缓冲区首指针返回给用户进程。用户进程无需直接获取数据缓冲区的首指针，而是通过系统调用来进行文件读写操作，让操作系统处理数据缓冲区的传输和管理。这样的设计可以提高系统的安全性、可靠性和效率。

文件读取：当用户进程调用读取操作（如 read()）时，操作系统会将文件描述符与对应的数据缓冲区关联起来，并将数据从文件缓冲区复制到用户进程提供的缓冲区中。在这个过程中，操作系统会使用数据缓冲区的首指针来指示数据的位置，并将读取的数据从文件缓冲区复制到用户进程的缓冲区中。

YYK
2026/10/29 21:33

在 Windows 中常常采用 FAT 文件系统，该系统就是以 FAT 文件分配表 + 位示图管理存储在磁盘上的文件的位置。也结合 FCB 方案来实现目录项的存储

在 Linux 中常采用索引节点（Inode），目录项由文件名 + 索引节点编号组成

YYK
2026/07/18 14:52

物理位置的核心信息就是 “文件第一个数据块的块号”（可以是扇区号、簇号等，取决于存储设备的分配单位）

YYK
2026/07/18 14:59

基本信息	逻辑结构
	物理结构
	类型
	文件大小
存取控制信息	文件权限（分为文件主、核准用户是指除了文件主之外的合法用户，一般用户）
使用信息	文件数据块指针
	文件建立时间
	上次修改时间

物理结构决定了文件如何分散存储（连续、链接、索引）

YYK
2026/07/18 14:57

FCB 属于目录项，必须连续存放

FCB 通过文件名和物理地址之间建立映射关系

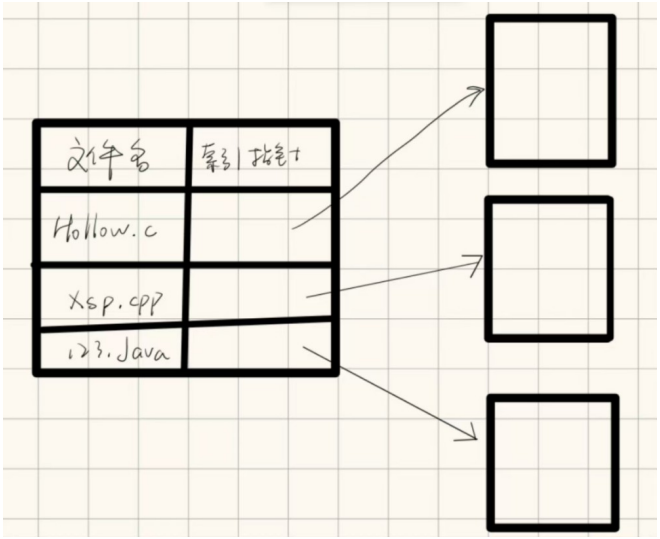
索引节点：

查找文件目录的过程：

先将存放目录文件的第一个盘块中的目录项调入内存，用给定的文件名与文件目录项逐个比较，

若未找到，则调入下一个盘块的目录项。

以上过程只用到了文件名，故而引入了文件名与文件信息分离的方法。让文件索引信息成为一个称为索引节点的数据结构，每个文件目录项仅由文件名和索引节点号（指针）构成。



索引节点与文件容量计算问题：

理解将 FCB 看作目录项升级为将索引看作目录项的内存优化。索引节点是对文件控制块的优化，极大节省系统开销，把文件控制块的冗余信息放到索引节点里，在目录项中只留下索引节点指针

①索引节点的个数：其实本质就是文件的个数，但 2020 年出的题具有迷惑性（2020 真题），并且索引节点个数与单个文件长度无关（2013 真题）。

磁盘索引节点（存放在磁盘里）：

每个文件有唯一的磁盘索引节点，下列是结构状态

主标识符	拥有该文件的个人或小组 ID
文件类型	普通文件、目录文件、特别文件
文件存取权限	对各类用户的存取权限，不同用户的权限是不同的，可以用 ACL 表

	来存储。
文件物理地址	每个索引节点含有 13 个地址项，iaddr0～iaddr12，他们以直接或者间接方式。
文件长度	以字节为单位的长度
文件链接计数	系统中所有指向改文件的指针计数
文件存取时间	本文件最近被进程存取、修改的时间、索引节点最近被修改的时间
索引结构	记录了文件存储的磁盘块的位置信息的组织形式（读清楚是位置信息的组织形式，而不是位置信息数据）。它可以是连续分配、隐式链接、显式链接、单级索引、多级索引、混合索引等。

为什么要有文件链接计数？链接计数记录了一个文件被多少个目录项直接指向，创建文件时仅自身文件名指向它，其链接计数为 1，当该数为 0 时，文件系统会将该文件从磁盘中释放。通过增加硬链接，即使原始文件被删除，仍可通过其他链接访问数据。不同用户通过硬链接指向同一份数据，避免复制文件，节省存储空间，并确保数据一致性。

内存索引节点（存在内存里）：

文件被打开时，需要将磁盘索引节点复制一份到内存的索引节点。

增加了以下内容：

索引节点号	标识内存索引节点
状态	指示该索引节点是否被上锁或修改
访问计数	每当有新的进程访问该节点，则+1
逻辑设备号	磁盘被划为不同的文件分区，每个分区是不同的文件系统，标记逻辑号来区分不同的文件系统，用于指明该文件所在的存储设备如硬盘分区、U 盘、网络存储等
链接指针	设置分别指向空闲链表（内存空闲区域）和散列队列（加速查找文件或目录）的指针

系统注册表：

系统注册表（System Registry）是 Windows 操作系统特有的一个核心数据库，用于存储系统和应用程序的配置信息。它是 Windows 操作系统的"中央信息仓库"。（2009 真题）

3.文件的操作：建立，删除，打开，关闭，读，写

文件的基本操作：

①创建文件：分配外存、创建目录项

②删除文件：根据文件名查找目录，删除目录项和 FCB 控制块，回收内存空间和磁盘空间

③读文件：根据文件名查找目录，读出文件在外存中的地址，在目录项中还有一个指针用于对文件进行读操作。

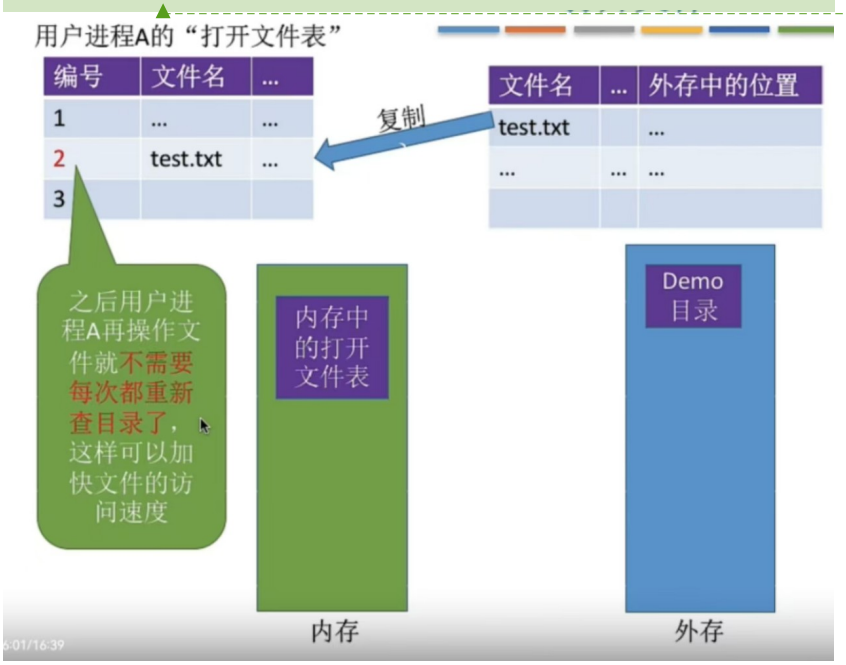
④写文件：根据文件名查找目录，使用写指针对文件进行写操作。

文件的打开：

①首次打开文件时，先利用系统调用 open（ ）将该文件打开（这个括号里传输的是文件的路径名，对于不同路径，打开的文件实体可能是一样的，因为存在硬链接），系统文件维护一个打开文件表，打开的意思是系统检索到指定文件的目录项后，将该目录项从外存复制到内存的打开文件表的表目里（这就是打开的过程，**而查找该目录项的过程并不是文件打开的过程**），并将改表目的索引号（文件描述符）返还给用户（不同进程对同一文件操作得到的文件描述符是不同的）。**以后系统便可以通过文件描述符（文件句柄）对文件进行操作，而不是通过文件名操作（多年考点）**

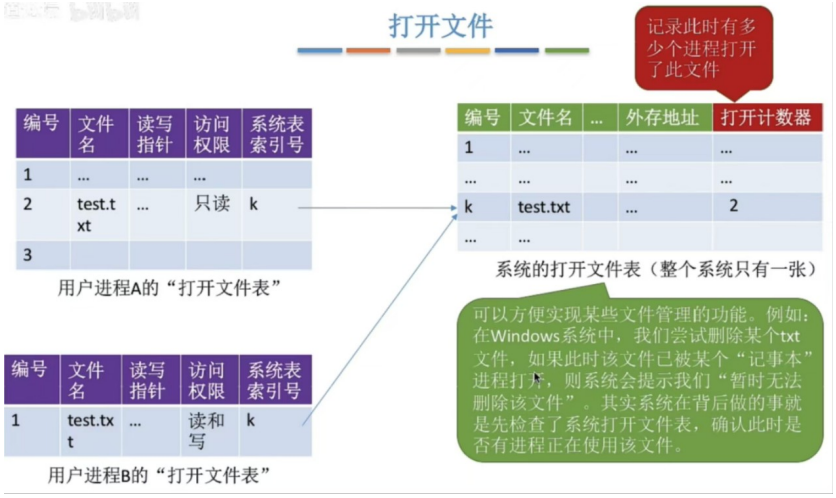
不同进程对同一文件所持有的文件句柄是不一样的，简单来说这个文件句柄就是一个 int 型变量
YYK
2026/07/18 15:07

这个 open 操作只把文件的元数据读入内存，但是文件内容仍然保存在磁盘里，只有在发生读写操作时，才把文件内容读入内存
YYK
2026/07/18 15:11



②在多个进程可以同时打开文件的操作系统中，包涵两个表：整个系统表（存储与进程无关的信息，例如文件大小、磁盘位置、访问日期）和每个进程表（进程对文件的使用信息，当前文件的读写指针、文件访问权限、指向系统表中适当条目的指针）。

③系统打开文件表记录所有被打开的文件目录项信息，包括一个打开计数器。用户打开文件表只记录该进程打开的文件，并且映射索引到系统打开文件表当中的目录项上。



④读写指针是操作系统内核为每个打开的文件描述符维护的一个整数值，表示下一次读写操作的起始位置，不同进程对于同一文件的读写指针位置是不同的

⑤对于用户打开表，open 一次，就会有一条表目，因为里面记录了偏移量，防止不同进程发生读文件的错乱。对于系统打开表，open 一次，只会建立唯一的表目

⑥open 操作不一定会导致文件 IO 操作，当文件已经被打开时，就不需要在从磁盘输入文件的元数据，因此也就不一定在系统文件打开表中增加一个新的表目

⑦每个打开文件都需要文件指针、文件打开计数、文件磁盘位置、访问权限，具体为什么请自己分析。

同一用户对同一个文件进行多次打开，会产生多条表目，即用户打开文件表是按照“打开次数”记录表目的，
YYK
2026/07/18 15:09

这里的 IO 操作，指的是对文件元数据的 IO ，而不是对文件内容的 IO
YYK
2026/07/18 15:13

文件的关闭：

①当某个进程不再使用该文件，**先将文件的控制信息写回磁盘，**则从该进程的文件打开表中删除对应条目，并且在系统打开文件表中的打开计数器-1，直到打开计数器为 0 时，在系统打开文件表中删除该条目。

关闭文件就必须把文件内容写回磁盘，这并不意味着它就是关闭文件的核心工作（因为文件数据的写回通常由缓冲机制异步处理），而关闭文件的核心工作是把文件控制信息例如 FCB 写回磁盘

YYK
2026/07/18 17:36

文件的删除：

①删除文件的 FCB（或者是采用索引节点文件系统的话，需要删除索引节点）

②删除文件的目录项（有可能是 FCB+其他信息，也有可能是文件名+索引编号），记得是删除文件所在的目录项，而不是所在的目录，因为该目录还存在着其它文件。（2013 真题）

③删除文件内容，释放磁盘空间。

文件的读操作：

①由于 open 文件打开了，所以以后对该文件的操作就是通过文件描述符进行对文件从 IO 磁盘读入内存了。

对文件进行读写操作时，系统不会直接把磁盘数据交给应用程序，而是先把“一整块数据”读到内核缓冲区，应用程序再从缓冲区取数据（后续读相邻数据时，直接从缓冲区拿，不用再读磁盘）。
写文件时：应用程序先把数据写到缓冲区，等缓冲区满了或手动触发“刷新”，系统再一次性把缓冲区数据写到磁盘

YYK
2026/07/19 14:09

而连续的读取 read 同一文件，第一次必然需要磁盘 IO，第二次可能不需要，因为操作系统会有页缓存机制，保存刚刚读入的文件的页面到页缓冲空间中，若页面未被淘汰，则下一次可以直接从内存中的页面缓存空间中读出，避免磁盘 IO；

但是如果相隔的时间长，页面缓存中的页面已经被淘汰，那就需要重新磁盘 IO 读文件内容

YYK
2026/07/18 15:16

文件的写操作：

①不一定在文件关闭时才写入磁盘，在其他情况下，内核会要求写回。

②要注意题目条件，“文件第 x 条逻辑记录完成修改后，共启动了磁盘几次”，这时候要加上将记录写会磁盘的这一次磁盘 IO 操作。

另外读操作可能是从文件的中间位置进行读取，那么就要先获取文件的逻辑结构，根据逻辑编号（该逻辑内容所在的文件中的位置）找到（这个找的过程也要依赖文件的物理结构，比如索引、顺序、链接方式）该逻辑内容位于的磁盘块号，然后读入该磁盘块

YYK
2026/07/18 16:36

设置文件读/写位置：

这个本身也是文件的一种操作方式，避免每次打开文件都从头进行读或写，通过这个操作可实现文件内容的随机存取

对文件的写操作，一般都对内存中的文件进行写，先保存在内存的缓冲区中，当缓冲区满了或者时间达到写回磁盘的时间点刷新之后，才会写回磁盘，但是，如果此时要求关闭文件，那就必须要把更新后的文件立即写回磁盘

YYK
2026/07/18 17:10

③文件的关闭过程（关闭文件不等于写文件，关闭文件只需要把未能的控制信息从内存写回磁盘）

4.文件保护

文件访问类型：

①读写

②执行：文件装入内存并执行

③添加：将新信息添加到文件结尾部分

④删除

⑤列表清单：列出文件名和文件属性

⑥文件重命名：高层功能通过系统程序调用底层系统调用实现

⑦文件复制、打印：通过读请求来完成

⑧文件编辑

①防止系统故障导致文件受损，所以采用备份方式。

②若进入系统时，不同用户需要进行登陆注册，那么该级别是系统级的。

写模式和添加模式有什么区别？

写模式：打开文件时会立即清空文件内容，将文件大小截断为 0 字节。即使后续没有写入任何数据，原内容也会永久丢失。添加模式：保留文件原有内容，文件指针初始位置位于文件末尾，新写入的数据会自动追加到现有内容之后

访问控制（文件访问权限等于=用户访问权限+文件属性，而不是用户优先级）：

这句话写的也很矛盾，但还是按照书上记忆吧，只有用户访问权限和文件属性限制文件访问权限
YYK
2026/07/19 16:12

（1）第一种方法是 ACL（Acess control list）

实现方法是根据用户身份，为每个文件和目录增加了访问控制列表 ACL，来规定每个用户以及访问权限。缺点是长度无法预计，可能导致复杂的空间管理。

下列是精简的访问控制表结构

拥有者	创建文件的用户
组	一组需要共享文件且具有类似访问的用户
其它	系统内的其它用户

表 4.1 一个精简的访问控制列表

用户类型	访问类型			
	读 取	写 入	删 除	执 行
拥有者	1	1	1	1
组	1	0	0	0
其他	0	0	0	0

这个文件权限不能用两位数二进制数代表 4 种用户，用 2 位二进制数代表 4 种访问权限。而是应该建立一个二维表格写清楚每种用户的每种权限的 01 状况，即不能认为一种用户只能有一个权限，而是它可以拥有多个权限，针对不同的权限，都要不同存储（2017 真题）

（1）第二种方法是口令

在用户创建文件时提供一个口令存储在 FCB 里面，同时告知可以访问该文件的用户，通过口令来访问文件。缺点是不够安全，存储在系统里可能被盗取。

字段名	作用	示例值
用户名（username）	用户身份标识（唯一）	"zhangsan"
用户 ID（user_id）	系统内部唯一标识（数字或字符串）	1001
口令哈希（password_hash）	口令经加密算法处理后的结果（非明文）	"a1b2c3d4..."（SHA-256
盐值（salt）	加密时添加的随机值（增强安全性）	"x9y8z7..."（随机字符串）
权限等级（role）	关联用户操作权限（如 "管理员""普通用户"）	"admin" / "user"
账号状态（status）	标识账号是否可用（如 "正常""锁定"）	"active" / "locked"

这是用户口令表的一个基本的格式

YYK
2026/07/19 14:00

（2）第三种方法是密码

对文件进行加密，访问时需要密钥，保密性强，节省了存储空间，编码和译码需要一定时间。

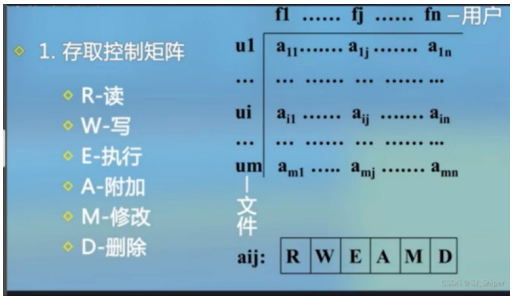
下面两种方法都是防止文件被窃取，并没有限制用户对文件的访问类型。

（3）存取控制矩阵

这种存取控制矩阵可以利用二维数组，而每个数组元素的内容可以是 0100101 ，代表对第 2 、 5 、 7 个权限有效，没有其他权限

YYK
2026/07/18 16:13

03 文件保护



优点是对用户和文件权限关系记录的非常清晰，但是占用太多存储空间，它也只起到记录权限

作用没办法保证文件系统不受损。

5.文件的逻辑结构

文件逻辑结构和物理结构的区别？

文件逻辑结构时用户角度看到文件的组织形式，即数据是如何被组织起来的。物理结构是文件存储在外存的存储组织形式，用户看不见。

无结构文件：

流式文件，字符流构成。

有结构文件：

定义：有一个以上记录构成的文件，根据记录长短，可分为定长和变长。

①变长记录：记录长度不同，检索只能顺序查找，很慢。

②定长记录：检索速度快，相同字节数为一个记录，记录在文件的同一列位置。

③记录成组分解技术，就是一条记录无法填满磁盘块，那就多条记录填充磁盘块，若剩余空间无法填入一个完整的记录，那就空出来或者填充指针之类的。如果一条记录超过了磁盘块的大小，那就多用几个磁盘块来存储该记录。

（1）顺序文件

① 可定长可变长

② 串结构（记录顺序与关键字无关，而与记录时间顺序有关，必须从头开始检索，费时）或者顺序结构（按照关键字排序、定长时可以采取折半查找）

③ 顺序文件效率最高，但查找、修改、增删性能较差。

④ 顺序存储 + 可变长依然无法实现随机存取

（2）索引文件

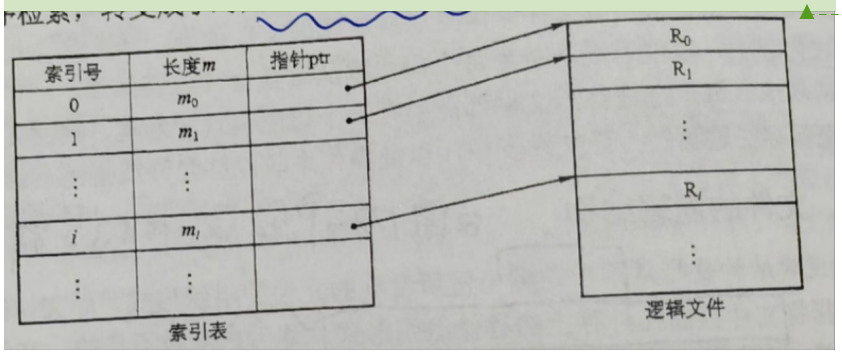
索引文件，本身也是普通文件，记录内容的文件，主要适用于变长记录文件。但是它通常需要搭配一个索引表来使用
YYK
2026/07/18 15:26

① 可定长（公式查找，长度*条数=地址）可变长（只能顺序查找索引，效率低）

② 可以建立一张索引表，索引表为定长记录的顺序文件，这样加快了检索，但增大了存储开销。

③ 索引表，每一行为一个索引项。

④ 要理解索引表（对文件内容的索引，记录文件内每一行内容的“小目录”）、索引块也叫索引表（里面存的是文件在磁盘内存储的盘块号）、索引文件（这是一种文件内容的逻辑结构，对索引文件进行存取时，必需先查找索引表）、索引项（是指索引表中的每一行内容，记录索引号+该行内容长度+指针指向逻辑文件中的起始位置，并且只能指向索引文件里的一个内容）



（3）索引顺序文件

①等于组间索引文件（将不同的记录根据相同的键值分为不同的大组，组间建立索引表，索引查找）+组内顺序文件（组内的所有记录有相同的键值，组内无序排列，顺序查找）。

②顺序文件查找时平均查找二分之N次。索引顺序文件查找，N条记录，分为根号N个组，每个组根号N个记录，组间查找为二分之根号N，组内查找为二分之根号N次，所以总共根号N次，有效提高查找效率。

索引表的查找过程是可以通过折半查找的，效率比较高，索引表的功能就是对只能顺序查找的文件改造成了可以随机查找的文件
YYK
2026/07/18 16:56

一个索引文件是可能配置多张索引表的，根据不同的关键字类型，例如图书借阅记录表中的借阅者姓名、学号都可以当做关键字来分开配置一张索引表。

而且索引表中是不记录文件内的数据的，需要根据索引表表项中的指针（指向物理地址）来找到对应文件内容
YYK
2026/07/18 16:55

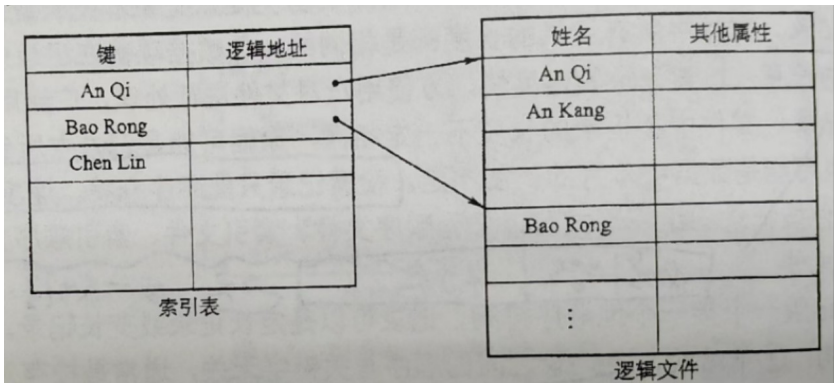
这个指向逻辑文件中的起始位置，指的是物理地址

YYK
2026/07/18 16:52

记清楚这个运算的逻辑

YYK
2026/07/18 15:38

③缺点是，需要建立索引表，增大了存储负担。



（4）直接文件或散列文件

直接文件是指的文件组织方式。即关键字本身可以直接作为（或经过简单转换后作为）物理地址，无需通过索引表或目录顺序查找。

举例：若一个文件的记录关键字是“100”，而它恰好存储在物理块 100 中，则该文件为直接文件。

YYK
2026/07/18 15:35

哈希文件是，它通过“哈希函数（散列函数）”将文件的关键字（如记录的关键字段）映射为物理地址（如物理块号），从而直接定位文件记录。

核心逻辑：关键字 → 哈希函数 → 物理地址（如 addr = H(key)）。
关键问题：哈希冲突（不同关键字映射到同一地址），需通过“开放定址法”“链地址法”等方式解决。
举例：用哈希函数 H(key) = key % 100 映射关键字，若关键字为“256”，则映射到物理块 56（256%100=56）。

YYK
2026/07/18 15:35

哪种文件组织适合‘频繁按关键字查询单条记录’的场景？
哈希文件！

YYK
2026/07/18 15:36

①通过散列函数换的键值决定记录的物理地址，很有可能冲突。

②哈希存储更适合磁盘块的分配与回收

动态分配：基于 Hash 函数快速定位物理块，分配效率高，即使大量数据插入，只要合理处理 Hash 冲突，仍能保持高效。与索引存储相比，无需维护庞大索引表；与链接存储相比，不用遍历链表查找位置。

动态回收：回收简单高效，标记空闲后，后续分配可快速利用，因 Hash 结构本身特点，空间利用相对更紧凑，碎片问题相对不突出。

6.文件的物理结构（本质是如何为文件分配磁盘块）（**内存与磁盘进行数据交换都是以块为单位，块大小与页面大小通常相同**）（逻辑文件存储到存储介质上采用的组织方式与存储介质特性有关）

1.连续分配：

①占有连续的块。

②优点：顺序访问和随机直接访问，寻道数、寻道时间、磁头移动距离最小。

③缺点：分配连续的空间，会产生外部碎片，必须事先知道文件大小，插入删除也需要进行物理移动很麻烦。

④自己犯了一个很大错误，当文件连续分配的磁盘块需要插入一个新的磁盘块时，要先检查尾部是否有空闲的磁盘块，然后将最后一个磁盘块读入内容，向后移动一个磁盘块的位置，重新写入磁盘，以此类推，直到写到插入磁盘块的位置。但是如果尾部没有空闲磁盘块，则要把整个文件全部的磁盘块依次读入内存，写入新的连续空闲磁盘块的位置，磁盘 IO 巨大！

⑤相对比之下，链接分配、索引分配只需要改动指针或者改动索引块中的磁盘块号即可。

29. 某 500 个盘块的文件的目录项已调入内存（若为索引分配，其索引块也在内存中）。若需要在文件中增加一块，下列分配方式中磁盘 I/O 次数最多的是（ ）。

A. 连续分配　　B. 隐式链接分配　　C. 显式链接分配　　D. 索引分配

还有一个角度：移动磁盘块必须要从尾部移动吗？如果前面有空闲的区域，不可以移动前面的磁盘块吗？这肯定可以啊！所以有时候如果移动前面的磁盘块来插入数据更方便的话，那就移动前面的磁盘块（某一年真题）
YYK
2026/07/18 15:47

在考虑在文件中添加一个磁盘块的时候，磁盘 IO 不仅要考虑读磁盘还要考虑写磁盘。

对于连续分配，我需要读出后 n 个磁盘块的内容，全都向后移动一个位置并写入，所以为读 n 次、写 n 次，共计 2n 次

对于隐式链接分配，读出前 m 个磁盘块，找到对应的指针，写入新的磁盘块 1 次，还有一次是原先的第 m 个磁盘块要指向新插入的磁盘块，所以这又是需要写 1 次，共计 m+2 次
YYK
2026/07/19 13:41

2.链接分配（FAT 文件系统）：

（1）隐式链接：

①文件目录项中含有头磁盘指针和尾磁盘指针，对用户透明。

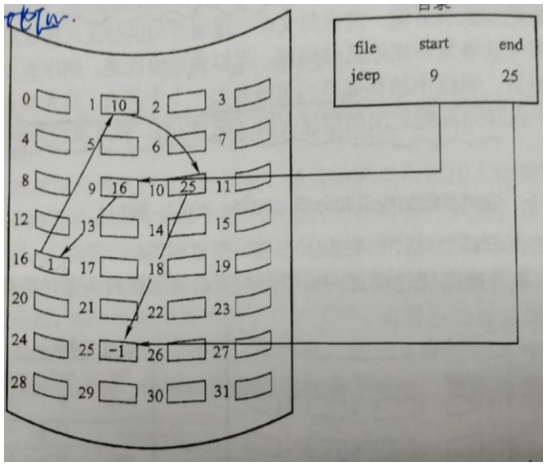
②缺点：只支持顺序访问，从头指针开始查找下一块磁盘，直到找到磁盘块。稳定性差，有一个指针坏了就彻底崩盘。大量指针浪费空间。

③可以引入簇（一个簇等于若干个连续的磁盘块），按照簇来顺序指针查找，减少指针占用空间，加快查询速度，但代价是增加了内部碎片。同时如果引入簇，那么系统给文件分配空间时要分配整数个簇的空间，即如果一个簇=4 个磁盘块大小，即使再分配一个磁盘块就够了，也不行，必须只能分配一个簇（2017 真题）

隐式链接的指针对用户来说是透明的，用户根本不知道这个指针的实际内容
YYK
2026/07/18 15:49

这个尾部磁盘指针是为了定位文件的最后一块磁盘块
YYK
2026/07/18 15:48

这个知识点再次回顾一下，如果文件系统引入了簇，那么最低分配单位就是簇，而不是磁盘块
YYK
2026/07/19 14:33



某文件共有 8 个记录 L1～L8，采用隐式链接分配，每个记录及链接指针占一个磁盘块，主存中的磁盘缓冲区的大小与磁盘块的大小相等。假设文件目录已读入内存。为了在 L5 和 L6 之间插入一个记录 Lx'（已在内存中），需要进行的磁盘操作有（ ）。

A. 4 次读盘和 2 次写盘　　B. 5 次读盘和 1 次写盘

C. 5 次读盘和 2 次写盘　　D. 4 次读盘和 1 次写盘

这个题非常好，自己忘记了，虽然 LX 是必然要写回磁盘的，但是 L5 的指针发生了改变，由指向 L6 变成了指向 Lx ，所以 L5 也要进行写盘操作，所以至少要 2 次写盘操作
YYK
2026/07/18 17:46

（2）显式链接：

①引入文件分配表 FAT（File Allocation Table），它本身也是一个文件，也具有 FCB，整个文件系统只设置一张，用户只能间接查看。

FAT 文件表式存储在磁盘特殊区域的，是不能直接查看的，用户根本打不开，但是可以通过特殊软件 diskpart 来执行特殊操作间接查看
YYK
2026/07/18 15:51

②文件目录中只引入一个起始块号。

③优点：支持顺序访问和直接访问，FAT 在开机系统启动时就被读入了内存（常驻内存），检索记录是在内存中进行的，显著提高检索速度，明显减少访问磁盘次数（隐式链接分配向下探索下一块磁盘块不需要每次都访问磁盘）。还有一个优点就是可以显示磁盘中的空闲磁盘逻辑块号，方便外存空闲磁盘管理，2019 真题。

④缺点：FAT 需要占用一定空间，并且还必须是连续空间

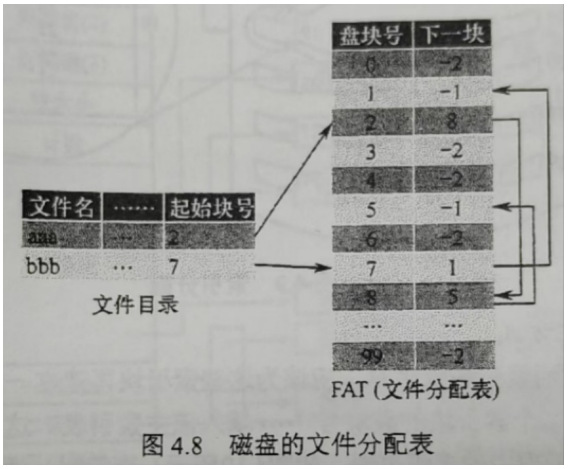


图 4.8 磁盘的文件分配表

FAT 支持的最大文件长度可以理解为整个分区都是该文件的存储空间（2016 真题）

其次，上图给出的 FAT 文件分配表有所误解，正确的应该是不分配第一列的存储空间的，因为它天然就是有序的，只需要保存指向的下一个磁盘块号即可。

这个跟页表一样，虚拟页号也是不存储，天然排序，这个 FAT 的盘块号也是天然排序
YYK
2026/07/18 15:55

3.索引分配（常用于 Linux 系统，直接将索引块存储在索引节点当中）：

（1）单级索引

这个单级索引就是通过占用磁盘中的一个数据块来存放该文件所占用的磁盘块号，从第一个到最后一个依次排列
YYK
2026/07/19 13:19

①没必要将整个 FAT（存储所有文件的磁盘关系）全都调入内存，只需要将所用文件的占用磁盘块调入内存。所以为每个文件分配一个索引块（表），存储该文件的所有盘块号。

要注意这里的索引表与文件逻辑结构的索引表的区别，给文件物理结构配备的索引表是为了记录该文件所在的磁盘块号，甚至有二级索引、混合索引；

但是给文件逻辑结构配备的索引表是为了记录文件逻辑内容和物理位置而建立的索引
YYK
2026/07/18 17:24

②一个盘块大小为 4KB，一个索引号大小为 4B，所以可以存储 1024 个盘块号。对应着 1024*4KB=4MB 的文件大小空间。

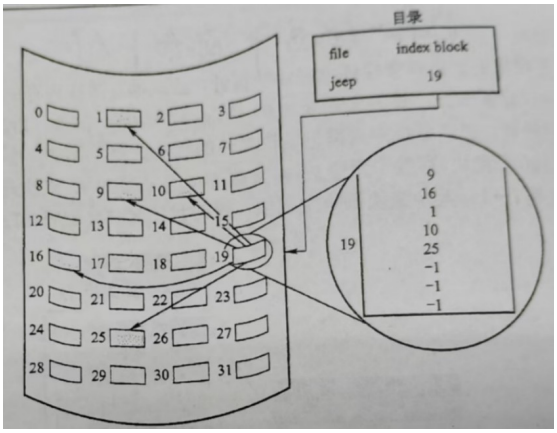
③在索引块里面，盘块号是按照文件存储的顺序排列的，索引块存放的磁盘块号是文件内容在磁盘中存放的物理位置。

④优点：可以直接访问文件存储的第 i 个磁盘块（从索引块中找第 i 个条目即可），不产生外部碎片。

索引分配是可以实现随机访问的，通过知道随机访问的内容的所在字节数，计算出第几个磁盘块，直接访问索引表中的第几个磁盘块即可
YYK
2026/07/19 14:31

⑤缺点：索引块增加了存储开销。文件太小，导致索引块后面空间浪费。文件太大，又需要多

个索引块来存储，用指针来链接多个索引块，效率低下。



（2）多级索引

这个多级索引就是用于大文件的，看清楚它的功能

YYK
2026/07/18 15:58

①若文件特别大，可能需要多个索引块，索引块太多，则需要建立多级索引。

②第一级索引称为主索引，主索引存放第一个索引的盘块号，第二个索引的盘块号。

③若一个盘块大小为 4KB，一个索引条目大小为 4B，则一个盘块可以存储 1024 个索引条目，

那么采用二级索引存储的最大文件=1024*1024*4KB=4GB

④优点：加快查找速度。

⑤缺点：多级索引，访问磁盘数增加，效果并不理想。

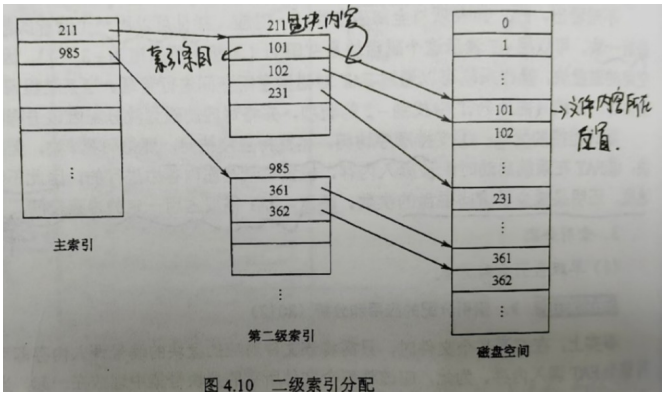


图 4.10　二级索引分配

（3）混合索引分配方式

根据不同文件大小采取不同的分配方式，UNIX 系统采用索引节点中设置 13 个地址项，即 iaddr.0~iaddr.12。

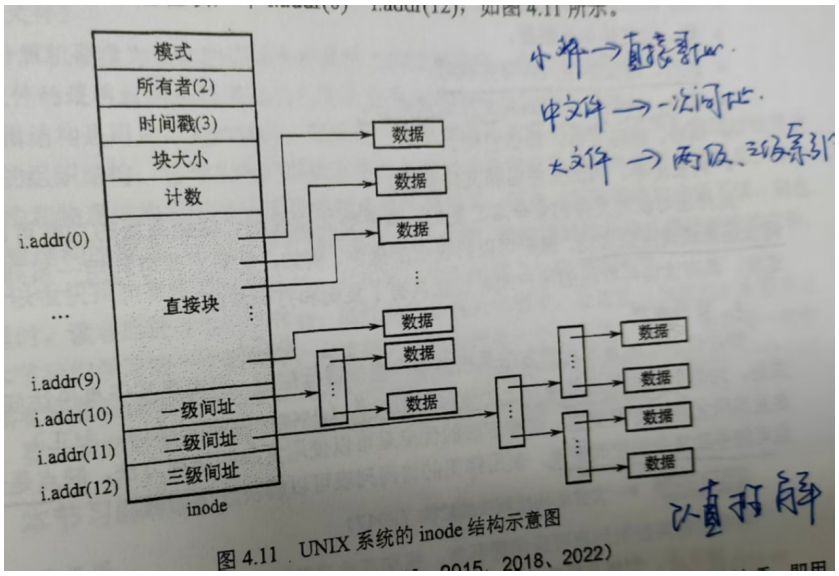


图 4.11　UNIX 系统的 inode 结构示意图

①小文件直接寻址：直接将盘块地址（盘块号）存入 FCB，直接从 FCB 中获取文件的盘块地址，总共分配了十个直接地址，iaddr0～iaddr9，每个格子 iaddr 存储一个直接地址，每个地址指向一个盘块号，则最大存储 10*盘块容量（假设 4KB）=40KB。

②中文件一次间址：利用单级索引，FCB 存入一个索引块地～～址（iaddr.10），指向一个索引块，该索引块中存储该文件的盘块地址，最大文件大小=4MB+40KB

③大文件多级索引：设置多个地址项，利用 iaddr.11 存储二级间接地址，可以存储 4GB，利用 iaddr.12 三级间接地址存储 4TB，所以文件最大为 4TB+4GB+4MB+4KB

文件系统的性能

①目录项由 FCB 升级为文件名+索引编号，这样节省空间，提高查找目录的效率，提高 IO 速率。

②文件高速缓存：数据读入内存，后续直接从内存读数据，减少磁盘 IO；延迟写回磁盘，先将内容写回页缓存中，积攒数量强制触发写回机制，减少磁盘 IO。

③磁盘调度算法：影响了写入内存的磁盘块的顺序，自然影响寻道时间，进而影响写入的 IO 时间。

④异步 IO 是指，发出 IO 请求后，CPU 去干别的事情，提高 CPU 的利用率，与磁盘 IO 没任何关系，所以不会提高文件系统的性能。

(二)目录

1.目录的基本概念

①文件控制块 FCB 的有序集合称为文件目录，一个 FCB 就是一个目录项。但是目录项太长占据空间，所以就优化为索引，将名字和索引指针变成索引目录，索引指针指向索引节点（存储文件信息）。

②多用户系统中还要加入对文件的控制访问信息，允许不同用户对不同文件采用相同的名字。

③FCB 是文件系统内部用来管理文件的结构，需要知道文件在存储设备上的实际位置才能进行读写操作。目录项是用于描述文件在文件系统中的位置，而不是文件在存储设备上的实际位置，只需要知道逻辑地址。

④有一个非常容易弄混淆的概念：目录项，它记录了文件名+指针与 inode 的逻辑映射关系，通过逻辑映射找到对应的 inode，inode 存储的是文件的元数据。

2.树形目录及其它三种结构

（树形结构不便于实现文件共享）（文件打开后，通过文件描述符来对文件进行访问）

单级目录结构：

整个系统只能有一张表，要求命名不能冲突，而目录项是

YYK
2026/07/19 14:54

①整个系统只有一张目录表，目录表中记录所有文件的目录项。

②建立新文件时，先检查是否文件名冲突，然后再目录表中新建一项，建立对应 FCB。

④删除一个文件时，先按照文件名在目录表中找到对应目录项，回收文件所占空间和删除 FCB。

⑤特点：实现按名存取，查找速度慢，不允许重名，不便于文件共享，不适用于多用户操作系统。

文件共享需要让多个用户或程序访问同一个文件，但单级目录中文件的访问权限等信息直接记录在目录项中，且目录表没有“用户区分”或“共享标识”的设计。

若要共享，只能让多个用户记住同一个文件名，但无法针对不同用户设置不同的访问权限（如甲可读写、乙仅可读）；
且一旦原文件删除，所有共享者都会失去访问入口，没有灵活的共享管理机制。
YYK
2026/07/19 14:59

⑥**读入一个文件的索引节点的访问磁盘次数=查找目录项平均查找次数+1 次读入索引节点**

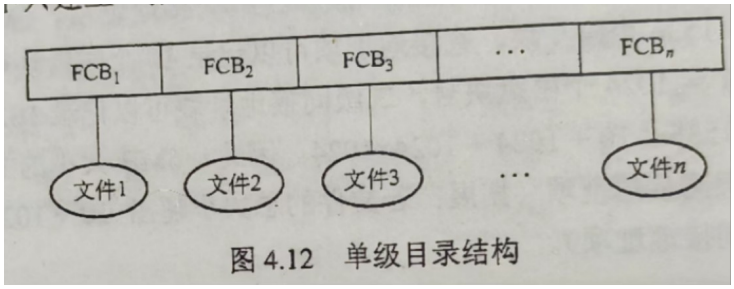


图 4.12 单级目录结构

多用户系统的核心需求是“用户隔离”和“资源独立管理”。

不允许重名：不同用户可能需要创建同名文件（如“工作计划.xls”），但单级目录会禁止；
缺乏权限隔离：所有用户共享一张目录表，难以限制“用户 A 访问用户 B 的文件”，存在安全风险；
管理混乱：多用户创建的文件全部混杂在一张目录表中，用户难以区分“自己的文件”和“他人的文件”。

YYK
2026/07/19 15:00

两级目录结构：

①分为两层，一层主文件目录 MFD（Master File Directory），一层用户文件目录 UFD（User File Directory）。

②主文件目录里面存储主文件目录项，主文件目录项=用户名+用户文件目录（这是一个文件）所在存储位置

③用户文件目录记录该用户所有文件的 FCB。

④特点：解决了多用户问题，解决重名问题，提高了检索速度，但是两级目录缺乏灵活性，不能对文件进行分类。

不同用户的文件管理互不干扰（例如用户 A 删除文件不会影响用户 B）。
支持权限控制：主目录可记录用户权限，操作系统能通过主目录判断用户是否有权访问系统，符合多用户操作系统的需求。
YYK
2026/07/19 15:03

总检索次数从“全局文件数”减少到“用户数 + 单个用户文件数”，显著提升效率（尤其用户多、单个用户文件少的场景）。

YYK
2026/07/19 15:04

两级目录严格限制为“主目录（第 1 层）→ 用户目录（第 2 层）→ 文件（终点）”，中间不能增加新的层级。
所有文件只能在用户目录下平行放置
YYK
2026/07/19 15:04

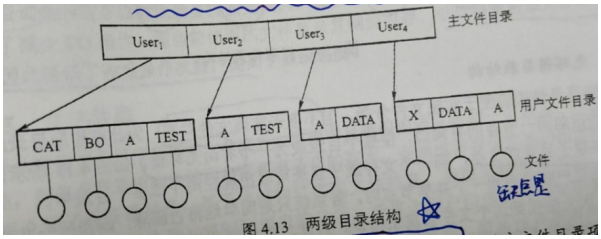


图 4.13 两级目录结构

树形目录结构 ：

①利用文件路径来查找文件，每个文件唯一路径。

②从根目录出发的路径称为绝对路径，但由于每个进程的文件局限于一个范围，所以每次从绝对路径的根目录查找比较麻烦所以，可以为每个进程设置一个当前目录（工作目录），从当前目录找会更加方便。

③特点：对文件进行分类，系统登陆哪个用户，就会自动进入该用户的工作目录，可以赋予不同存取权限。但是每次逐级访问中间节点，增加了磁盘访问次数，影响查询速度。不便于实现文件共享

④明显提高目录的检索速度和文件系统性能

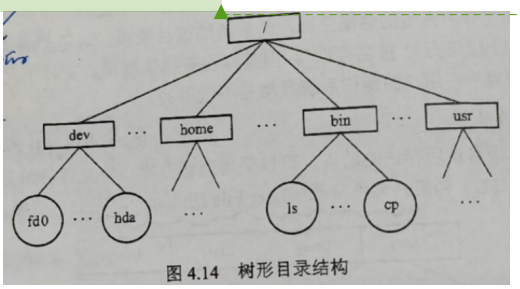


图 4.14 树形目录结构

每次访问一个中间节点，就需要从磁盘调出该目录文件的内容
YYK
2026/07/19 15:06

原因是因为每个文件和每个目录都只允许有一个父目录
YYK
2026/07/19 15:28

跟上边一样，对文件进行分类后，扫描的文件数量下降了，自然检索速度快
YYK
2026/07/19 15:05

无环图目录结构：

①在树形目录结构的基础之上，增加有向边，成为一个有向无环图。

②为每共享节点设置一个共享计数器，直到计数器为 0 才将文件删除。

③特点：实现了文件共享，但是系统管理更复杂。**只是把共享文件的盘块号复制到了对应目录，如果一个用户增加内容导致增加了盘块，这个盘块号只有该用户可见，其他共享用户不可见，这就是其弊端。**

这种方式实现了不同用户共享同一个文件或者同一个目录
YYK
2026/07/19 15:10

特别关注这个盘块号的问题，只是把盘块号写入了目录里面。但是如果有别的进程对文件进行了添加的写操作，写入了新的磁盘块，而其他用户是不知道这个变化的
YYK
2026/07/19 15:40

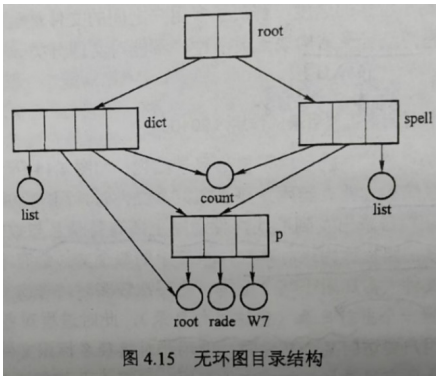


图 4.15　无环图目录结构

目录的实现：

1. 线性检索法：

例如某个一个路径 usr/test/2025-7-19/康永万里/OS,就需要先从根目录下顺序找到 usr 字段所在的目录项，找到 usr 目录文件所在的盘块号，然后在该盘块号内 usr 目录文件中顺序查找 test 字段的目录项.....如果有一个文件分量名未能找到，就立即停止查找，返回“文件未找到”信息

这个过程最终找到的是 OS 的目录项，目录项包含文件名和索引节点编号，“可不是找到了文件的物理位置，请注意分别”
YYK
2026/07/19 15:57

2. 哈希方法

建立一张哈希索引文件目录，利用哈希方法进行查询；若目录表中对应的目录项是空的的，则说明无该文件，若与指定文件名相匹配，则可以找到该文件所在的物理位置；如果在目录表的响应目录项中的文件名与指定文件名并不匹配，则发生了冲突，应该在哈希值上加一个常数（与目录长度互质），形成新的索引，继续查找。

3.目录的操作

①创建目录：（一般都是要看创建的文件名是否与集合内的文件名冲突）树形目录结构中，用户可以创建自己的用户文件目录，并可以再创建子目录。

②删除目录：一种是只有在删除目录中的所有文件，并递归删除所有子目录，才可以删除本目录。一种是可以直接删除目录和文件和子目录。

这种方法比较危险，很容易造成数据损失

YYK
2026/07/19 15:26

③移动目录

④显示目录

⑤修改目录

4.文件共享硬链接和软链接

（建立文件 F1 时，引用数值就要加 1）

基于索引节点的共享方式（硬链接）

①目录项=文件名+指针，指针指向索引节点，索引节点存储文件的各种信息，包涵链接计数 count。

②当一个用户创建新文件时，它是该文件主人，count 此时=1。只有当 count=0 时才删除整个文件。

③硬链接直接到达，所以查找速度更快。

④创建文件 F1，则引用数=1，创建文件 F1 的硬链接 F2，则 F1 和 F2 的引用数都等于 2，其实本质上 F1 和 F2 都是同一个文件，指向的是同一个索引节点。

这里的引用数啊，其实就是索引节点被调用的个数，所有用户的目录项不持有索引节点，而是持有指向索引节点的编号（即指针）

YYK
2026/07/19 15:45

⑤如果两个进程分别打开了用户 A 的文件和用户 B 的文件（这两个文件指向的是同一个索引节点，如下图），第一次打开需要通过用户 A 的目录找到索引编号，在磁盘中找到对应索引节点并将索引节点读入（复制）内存，请注意此时需要在系统打开表中新建条目：包涵指向内存中的索引节点的指针、打开计数，用户文件打开表也要新建条目。

此时进程 2 想要打开用户 B 的文件，查找目录之后发现索引编号是同一个，那么**就直接跳过 IO**，从系统打开表中找到内存中已经加载过的索引节点指针。

⑥上述⑤的问题续集，两个进程对该文件的访问权限可能是不同的。

⑦上述⑤的问题续集，由于进程 1 已经将文件打开了，那么后续对文件的读取操作都通过文件描述符来进行，而不是绝对路径作为系统参数。两个进程的**文件描述符是不同的**，但底层都指向同一个索引节点，**而且文件描述符都各自指向自己的用户文件打开表中的一项（2017 真题）**。

文件已经打开了，在系统文件打开表中有一个条目，用户打开文件时会被分配一个 fd 文件描述符（不同用户的即是对同一个文件的 fd 是不同的）
YYK
2026/07/19 15:47

⑧上述⑤的问题续集，两个进程虽然对同一文件读取，但是各自拥有**独立的读写指针**存在用户进程打开文件表（2017 真题）。

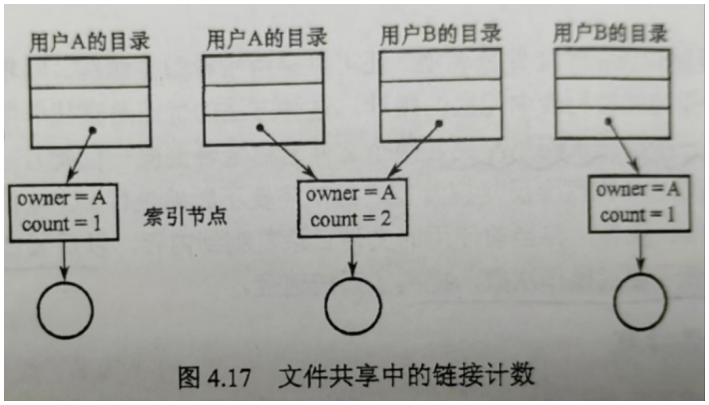


图 4.17　文件共享中的链接计数

基于符号链接实现文件共享（软连接）

无论是软链接还是硬链接，每增加一个链接，就会增加一个文件名，这样就可能会对一个共享文件产生多次拷贝
YYK
2026/07/19 15:50

①创建一个 LINK 新文件，将文件 L 写入想要共享的用户目录，LINK 中写入该文件的路径名。

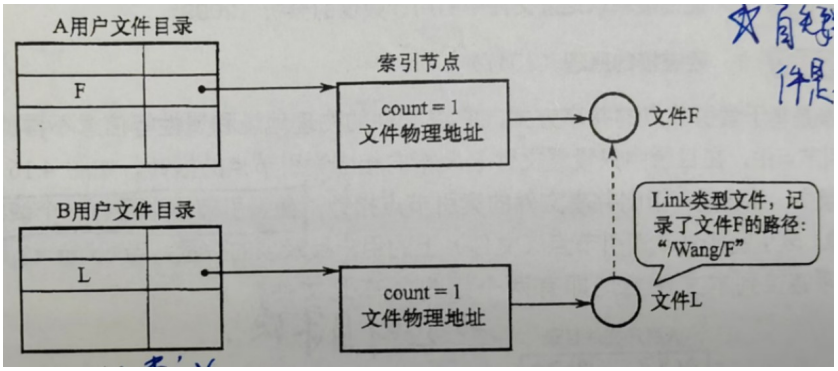
②此时被软连接的文件的引用数保持不变，而软连接新建的文件（里面写有链接文件的绝对路径）引用数一直都是 1。

③当文件主将源文件删除之后，其它共享用户再去通过 LINK 中的文件路径查找共享文件报错，然后将符号链接删除。

④特点：访问共享文件可能需要多次读盘；LINK 本身也是文件，其索引节点要耗费磁盘空间。

这个很有可能出一个考点，问需要多访问的磁盘次数，关键在于又多了访问文件 F 的路径的磁盘次数，如下图
YYK
2026/07/19 15:51

⑤当利用符号连接实现网络共享时，只需要提供机器所在网络地址和文件路径名。


A用户文件目录
F
索引节点
count = 1
文件物理地址
文件F
B用户文件目录
L
count = 1
文件物理地址
Link类型文件，记录了文件F的路径："/Wang/F"
文件L

(三)文件系统

1.文件系统的全局结构(layout)

文件系统在外存中的结构，文件系统在内存中的结构

文件系统结构：

文件系统模型分为

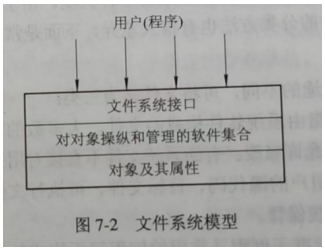


图 7-2　文件系统模型

(1)对象及其属性：文件、目录、磁盘存储空间

(2)管理软件集合：文件存储空间管理、文件目录管理、文件逻辑地址转化成物理地址、

文件读写管理、文件共享和保护

实现以上功能用了四层软件结构：

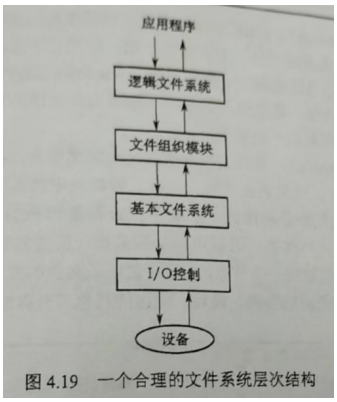


图 4.19　一个合理的文件系统层次结构

① IO 控制层：中断处理、由磁盘驱动程序组成

②基本文件系统层：处理内存与磁盘之间的数据块交换

③基本 IO 管理层：完成与磁盘 IO 相关的事务，例如文件逻辑块号转换成物理块号，管理磁盘空闲盘块、IO 缓冲指定

④逻辑文件系统：实现按名存取、文件权限验证、文件**元数据（详细的元数据或者目录）**管理、**把文件名转化为文件描述符 fd**、允许通过 fd 访问文件和记录、对文件和记录进行存储保护

这个可能是最重要的一层结构

YYK
2026/07/20 14:45

（3）文件系统接口

命令接口：用户与文件系统直接交互，通过终端直接获得文件系统提供的服务

程序接口：通过系统调用获得文件系统的服务，read、open 等

用户调用 read(fd, buf, n) 时，数据处理的正确层次顺序是?
答案：逻辑文件系统→基本 IO 管理层→基本文件系统层→IO 控制层。
YYK
2026/07/20 14:18

：简述用户执行 write(fd, buf, 1024) 时，文件系统各层的工作。
：
逻辑文件系统：验证文件权限，查找文件 inode，确定逻辑块号。
基本 IO 管理层：将逻辑块号转换为物理块号，分配磁盘空间。
基本文件系统层：组织数据块，添加磁盘地址信息。
IO 控制层：生成磁盘 IO 请求，驱动磁盘控制器传输数据。

YYK
2026/07/20 14:19

碎碎念：

①在用户看来，引入文件系统的目的是实现按名存取。

②一个文件系统管理的文件数量受限于文件控制块（在文件系统搭建初期，就已经固定了 FCB 的最大数量，但现代系统可能要求没有那么严格，采用的是动态的 FCB 数量）的数量，所以文件系统使用的最大文件数量不等于所在磁盘的容量。

③采用 RAID 磁盘阵列，可以将一个文件分开放在不同的磁盘上，所以不是必须将文件放在一个磁盘上。

④单个文件的大小不是只受磁盘剩余容量大小的限制，而是还受到 FAT 表、FCB 等结构的限制

文件系统在外存中的结构：

每个分区都有一个独立的文件系统，保存有启动存储在那里的操作系统的方式、总的块数、空闲块的数量和位置、目录结构以及各个具体的文件

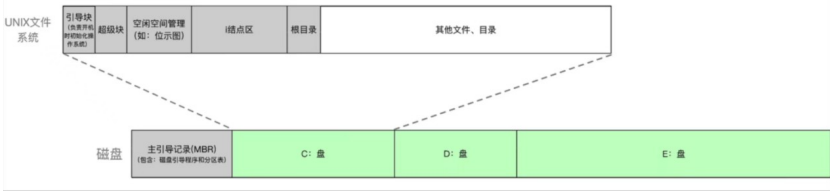
这里的分区是指 C 盘 D 盘等，例如笔记本的固态硬盘可以分成不同的区
YYK
2026/07/19 16:54

①主引导记录 MBR：位于磁盘 0 号扇区，引导计算机，后面是分区表（记录每个分区的起始、终止地址）

②MBR 执行引导块（这个引导块是指装有操作系统的分区的第一个块）中的程序，该程序负责启动该分区中的操作系统。

③超级块：（用于描述文件系统的）包涵文件系统的所有关键信息，分区块的数量、块的大小、空闲块的数量和指针，空闲的 FCB 数量和 FCB 指针

④文件系统中空闲块的信息：用位示图和指针链接的方式给出。



一、文件系统基础属性
文件系统类型与版本：例如 FAT32。
文件系统总大小：整个文件系统占用的磁盘总字节数（或总块数）
块大小（Block Size）：文件系统的基本存储单位（如 4KB）
inode 总数量（整个文件系统可创建的文件 / 目录总数上限）
inode 大小（每个 inode 存储的元数据容量）；
inode 块的起始位置（inode 区在磁盘中的起始扇区 / 块地址）。
创建与修改时间：文件系统首次格式化的时间、最后一次挂载 / 修改的时间（用于日志和状态追踪）。
二、空间管理元数据
用于管理磁盘块和 inode 的分配与释放，是文件读写时 “找空间” 的关键：
空闲块与空闲 inode 数量：当前文件系统中未被使用的磁盘块总数、未被使用的 inode 总数
数据块分配策略：例如是否启用 “预分配”（为大文件提前预留连续块，减少碎片）、块分配的优先级规则等。
超级块是文件系统的 “全局目录”，记录了从 “文件系统多大、块多大” 到 “空闲空间有多少、是否需要修复” 的所有核心信息。操作系统挂载文件系统时，第一步就是读取超级块——只有获取这些信息，才能进一步解析 inode、数据块，最终实现文件的读写管理。

YYK
2026/07/18 17:12

文件系统在内存中的结构：

内存中的信息用于管理文件系统，并且缓存一定的数据来提高系统性能。

①内存中的安装表：包含每个已安装文件系统分区的有关信息。

②目录结构的缓存：存储最近访问的目录的信息

③系统的打开文件表：FCB 副本和计数器

④进程的打开文件表：保存有文件描述符、读写指针和对应表项的指针（指向系统的打开文件表）

通过在每个空闲磁盘块末尾加一个指针，把空闲磁盘块全都穿起来，形成一个链表结构
YYK
2026/07/19 16:58

2.外存空闲空间管理办法

基本知识：

①卷：文件系统的分区，C 卷、D 卷。它可以是磁盘的一部分，也可以是整个磁盘或者 RAID 磁盘阵列。

②文件区：存放文件数据

文件区和 FCB 的空间（目录区）是分开的

YYK
2026/07/19 17:01

③目录区：存放 FCB

④物理盘和逻辑盘

空闲表法（这属于空闲磁盘块管理方法）：

①属于连续分配，为所有空闲区建立空闲表，每个表项表示一段连续的空闲区，按照磁盘序号顺序排列。

②分配方法：首次适应算法、最佳适应算法、最坏适应算法等，与内存的动态分配类似。

③回收：回收空间，前后合并。

④优缺点：较高的分配速度和减少磁盘的 IO 频率，对于小文件可以采取连续分配空闲磁盘的方法。

⑤可以得到连续的空闲磁盘空间，或者离散的空闲磁盘空间。

表 4.2 空闲盘块表

序　号	第一个空闲盘块号	空闲盘块数
1	2	4
2	9	3
3	15	5
4	—	—

空闲链表法（这属于空闲磁盘块管理方法）:

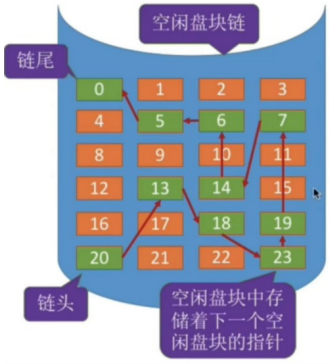
空闲链表适用于离散分配、难获得连续空间，因为是随机的不同的盘块被头尾指针链接

（1）空闲盘块链：

①将空闲盘块穿成一条链，需要分配磁盘块时从链上取出，若释放空闲盘块则添加到空闲盘块链的末尾。

②优缺点是：过程简单，但若连续的盘块，需要多次重复操作，效率低，而且空闲盘块链很长。

③最难得到连续的空闲磁盘空间。

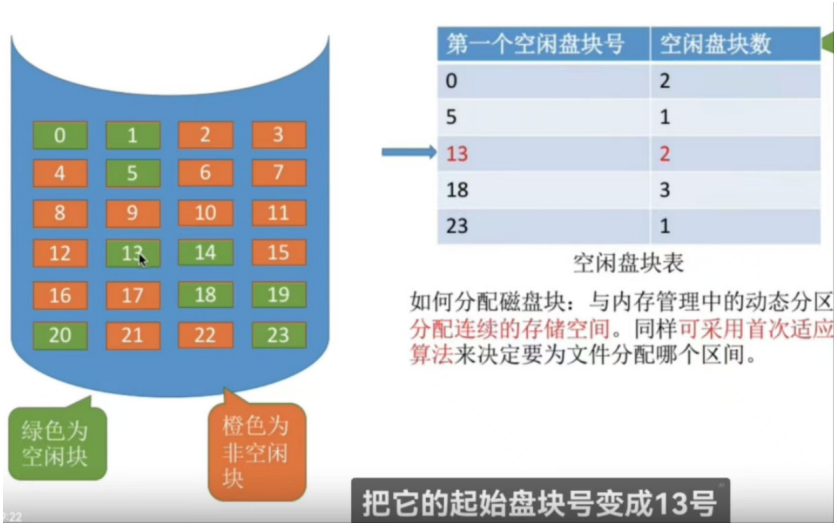


（2）空闲盘区链：

①将所有空闲盘区拉成一条链，每个盘区包含若干盘块，每个盘区记录下一个指针和该盘区的盘块数量。

②优缺点是：分配和回收的效率较高，并且链子较短，但分配与回收的过程比较复杂。

③最容易得到连续的空闲磁盘空间。

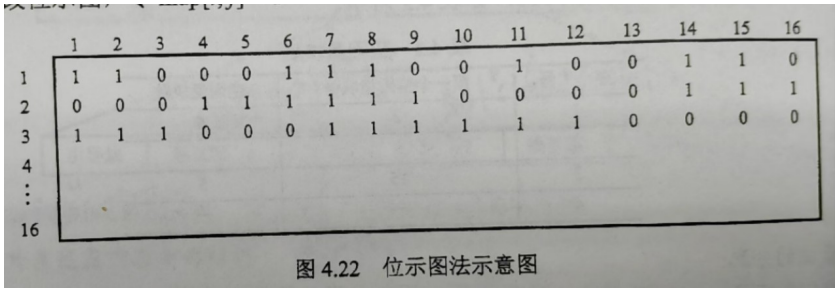


位示图法（这属于空闲磁盘块管理方法）：

位示图在文件系统挂载完成后被加载到内存中并且常驻内存，因为位示图是需要经常被使用的，不需要调回磁盘中
YYK
2026/07/18 17:04

①将所有磁盘行列对应二维数组，若已分配则数组记为 1，未分配记为 0，位示图适用于离散分配和连续分配。

②优缺点：很容易找到空闲磁盘块或者空闲磁盘区，占用空间很小（**一位仅占用 1bit，而不是整数型的 int 占 4B，2015 和 2023 真题**），可以保存在内存中，节省许多磁盘 IO。但也只适用于小磁盘的计算机，大磁盘的计算机位示图太大。



	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	1	1	0	0	0	1	1	1	0	0	1	0	0	1	1	0
2	0	0	0	1	1	1	1	1	1	0	0	0	0	1	1	1
3	1	1	1	0	0	0	1	1	1	1	1	1	0	0	0	0
4																
⋮																
16																

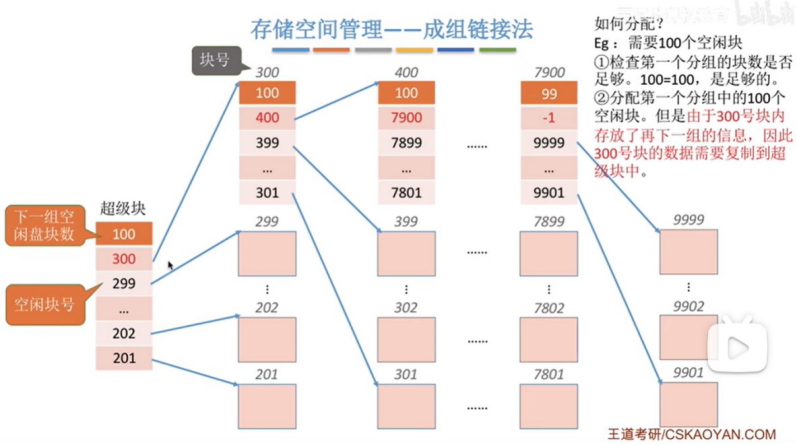
图 4.22　位示图法示意图

☆☆☆☆成组链接法（这属于空闲磁盘块管理方法）（**理解不够**）：

它克服了空闲表或者空闲链表太大的问题。

①空闲盘块号栈：用来存放当前可用的一组空闲盘块的盘块号。每组的第一块叫做索引块。第一个数字 N 代表盘块号栈的数量。

②文件区中的所有空闲盘块被分成若干个组，比如，将每 100 个盘块作为一组。假定盘上共有 10 000 个盘块，每块大小为 1 KB，其中第 201～7999 号盘块用于存放文件，即作为文件区，这样，该区的最末一组盘块号应为 7901～7999；次末组为 7801～7900……；第二组的盘块号为 301～400；第一组为 201～300，

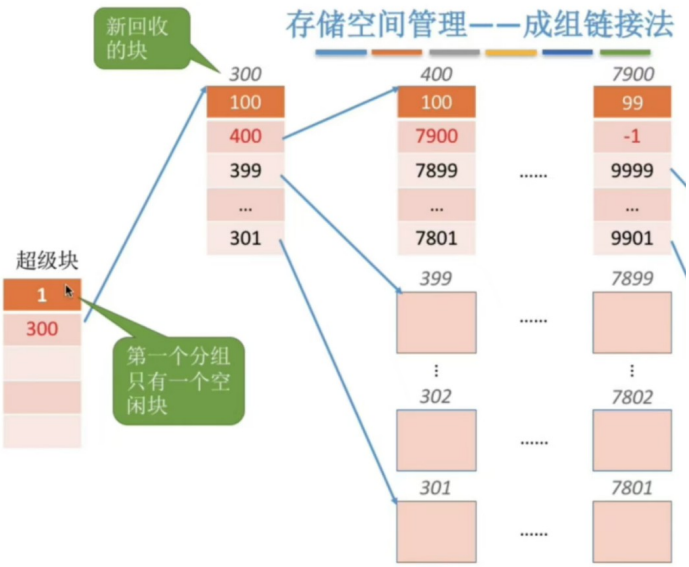


③如上图所示，超级块（已经读入内存了！！！）的第一块表示该块存储的块的数量，第二个数表示下一个存储空闲盘块的盘块号。其余 99 个数表示 99 个空闲盘块。如上图所示普通块第一个盘块号表示下一个存储空闲盘块的盘块号，其余 99 个数表示 99 个空闲盘块。不过盘块内的第一个数为-1 或者 0，则说明这是最后一个链接的盘块组号。

④分配磁盘块：先从栈顶分配即 S.free[1]，然后如果再分配 S.free[0]，王道的第 300 块磁盘块存储的是下一组磁盘块的数量和盘块号，要把第 300 个磁盘块分出去之前，先把块号为 300 存储的组别信息保存到超级块当中，防止链接断掉。

⑤盘块的回收：如果超级块没满，就回收到超级块中。若超级块回收满了，则把超级块中的内容复制到一个新的盘块中，把该盘块号填入超级块的第一个数据中。

⑥可以得到连续的空闲磁盘空间或者离散的空闲磁盘空间。



FAT（File Alocation Table）表（这也属于空闲磁盘块管理方法）：

①原本这属于文件分配的磁盘块链接，显性链接索引，但是不要忘记 FAT 表中记录了所有的磁盘块，如果该磁盘块不属于任何一个文件，那么可以把它记作-2，表示空闲。（2019 真题）

3.虚拟文件系统

虚拟文件系统 VFS的三条存在概念：

它不提供存储功能，只提供一个合适的接口，接收上层应用程序的需求，转化为执行的要求分发给下层的文件系统（无论你是 FAT、NTFS 文件系统，VFS 可适配任何符合接口的文件系统）
VFS 可以管理本地和网路多种文件系统
YYK
2026/07/19 17:36

①由于不同外存的文件系统各不相同、接口和实现方式不一样，所以采用虚拟文件系统为上层提供统一接口的服务、屏蔽下层的各个文件系统差异。

②下层实际的文件系统必须满足虚拟文件系统的规定和要求（严格限制各个函数的功能和参数）

③打开文件后，在**主存**中新建一个统一的 vnode 数据结构（**vnode 只存在于主存中**，而 inode 时从外存调入内存的），存储文件的各种信息，无论该文件来自于哪个文件系统（因为不同的文

Vnode 跟 inode 不一样！！
Vnode 是为了实现对不同文件系统操作的统一调用（无需上层关心具体实现）
YYK
2026/07/20 14:28

件系统的目录项千差万别）。并且 vnode 中还有函数功能指针指向函数列表，便于调用函数进行操作。

④下层文件系统需实现 VFS 规定的“接口函数集”（如打开、读写、关闭等操作的函数），且函数的参数格式、返回值规则必须严格符合 VFS 要求（否则无法被 VFS 识别）。例如：Ext4 和 FAT32 的“读文件”函数实现逻辑不同，但输入（文件标识、缓冲区地址、长度）和输出（实际读取字节数）必须统一。

VFS 会自动修改该函数参数”（错误，VFS 不修改底层函数，仅要求底层适配）；“该文件系统可直接被用户进程访问”（错误，用户进程必须通过 VFS 接口访问，底层不兼容则无法访问）
YYK
2026/07/20 14:28

⑤虚拟文件系统包含四种类型：超级块对象、索引节点对象、目录项对象、文件对象

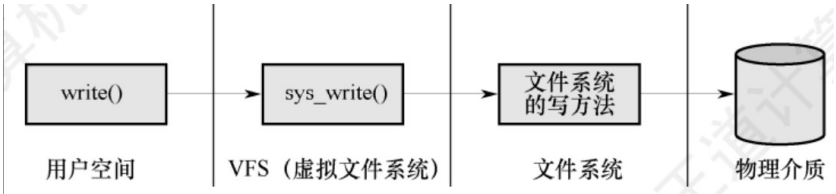


图 4.25　write()系统调用操作示意图

4.文件系统挂载(mounting)

如何将文件系统安装到虚拟的文件系统上：

文件系统挂载要做的事：

①在VFS中注册新挂载的文件系统。内存中的挂载表（mount table）包含每个文件系统的相关信息，包括文件系统类型、容量大小等。

②新挂载的文件系统，要向VFS提供一个函数地址列表

③将新文件系统加到挂载点（mount point），也就是将新文件系统挂载在某个父目录下

挂载点可以是一个未创建的目录”（错误，挂载点必须已存在）；“挂载后被挂载文件系统的根目录与原有目录树独立”（错误，已整合到原有目录树）。
正确：“挂载点是一个已存在的目录，挂载后访问该目录即访问被挂载文件系统”。

YYK
2026/07/20 14:38

挂载是 VFS 管理多文件系统的前提：被挂载的文件系统必须先符合 VFS 的接口要求（对应概念②），挂载后 VFS 会为其创建对应的管理结构（如记录挂载点与文件系统的映射），后续通过 VFS 接口访问时，VFS 可根据路径找到对应的文件系统并转发调用。

典型题目：“某文件系统能被成功挂载到系统中的
错误选项：“该文件系统必须是 Linux 原生文件系统”（错误，只要符合 VFS 接口，任何系统均可）；“该文件系统已存在于内存中”（错误，文件系统在外存，挂载是建立映射）。
正确选项：“该文件系统的接口符合 VFS 的规定和要求”（对应概念②，是挂载的必要条件）。

YYK
2026/07/20 14:39

3. 虚拟文件系统 VFS：向上为用户提供统一调用接口

① 超级块对象：表示已安装的特定文件系统，包含对inode的相关操作

② 索引节点对象：表示一个特定文件，vnode，只在主存中，即调入内存的inode+函数功能指针

③ 目录项对象：表示一个特定目录项

④ 文件对象：表示一个与进程相关的已打开文件，open，close

4. 文件系统挂载：即安装

① 在VFS中注册新挂载的文件系统，**内存中的挂载表**包含每个文件系统的相关信息（类型、容量....）

② 新挂载的文件系统，要向VFS提供一个函数地址列表（即open，close在哪）

③ 将新文件系统加到安装点/挂载点，即在某个父目录下（系统初始化时or用户自行，在已安装文件系统目录下）

五、输入输出(I/O)管理

(一)I/O 管理基础 （这一节的内容自己整理的不够到位！）

1.设备

设备的四种分类方式：

按信息交换单位			
块设备	以数据块为信息交换单位	磁盘、磁带	传输速率高、可寻址、随机读写
字符设备	以字符为信息交换单位	键盘、打印机	传输速率低、不可寻址、IO 中断方式

按传输速率		
高速设备	传输速率高	磁盘、光盘
中速设备	传输速率中	激光打印机
低速设备	传输速率低	键盘、鼠标

按使用特性		
人机交互	人和机器交互	键盘、显示器、打印机
存储设备	存储数据	磁盘、磁带

通信设备	计算机之间通信	网卡

按使用特性		
独占设备	同一时刻只能有一个进程占用 （关于为什么鼠标是独占设备而不是共享设备，我认为鼠标可以在一段时间内点击 QQ 和微信，符合共享设备的定义，但实际上，在点击 QQ 和微信时，这个鼠标是被进程独占的，反而磁盘在一段时间内轮流给不同进程传送数据，分时传送，这才叫共享设备） 独占设备采用静态分配方式	低速设备如打印机、鼠标和键盘、磁带机
共享设备	同一时间段内允许多个进程同时访问（分时方式共享） 共享设备必须是可寻址、可随机访问的设备。 共享设备采用动态分配方式	磁盘
虚拟设备	SPOOLing 技术将独占设备改造为共享设备，将一个物理设备改造成多个逻辑设备，可以同时分配给多个进程使用。 虚拟设备的引入是为了将一个物理设备变换成多个对应的逻辑设备，克服了独占设备速度慢、利用率低的特点。	比如打印机，虽然只有一台物理打印机，但是通过 SPOOLing 技术打造一个缓冲队列，将各个用户的打印需求装在缓冲队列里面，就好像在逻辑上全都独占打印机、有多台逻辑

分配共享设备是不可能引起死锁的，因为它们属于分时使用 CPU ，不会造成一个进程长期独占设备。

YYK
2026/07/21 14:56

例如磁盘，必须要指明两个进程各自写的范围，否则就会造成数据覆盖。可以随机访问才能让两个进程之间相互分时使用共享设备

YYK
2026/07/21 14:58

		上的打印机

IO 接口（也叫设备控制器）

①CPU 与设备之间控制的桥梁，实现设备与计算机之间进行数据交换。

②它在某种程度上替代了 CPU，即 CPU 的命令先存在 IO 接口里，然后 CPU 抽身去处理别的事情。

③组成结构：

a.CPU 与控制器的接口：负责实现 CPU 与设备控制器之间的通信。数据线传送读写数据、控制信息、状态信息。地址线传送的是访问 IO 接口中的寄存器编号。控制线传送读写控制信号。

b.设备控制器与设备接口：IO 接口可以和多个设备链接，与之链接的设备都有数据、状态、控制信号。

c.IO 逻辑：用于实现对设备的控制、通过一组控制线与 CPU 交互，对 CPU 发来的 IO 命令进行译码，接受操作命令和地址命令。

④主要功能：

a.接收命令、翻译命令

b.数据交换：CPU 与控制器、控制器与设备之间都可以进行数据交换

c.标识和报告设备的状态：存放某些设备的状态信息供 CPU 了解，例如只有达到设备就绪状态时，CPU 才能启动控制器从设备中读入数据

d.地址识别：识别是指向哪个设备的

e.数据缓冲：CPU 与 IO 设备数据传输差异，IO 接口中由缓存装置

f.差错控制：USB 接口、网卡保证数据传输正确（校验）

他还需要提供控制命令，我尼玛，这什么鬼王道
YYK
2026/07/21 15:19

可以控制一个或者多个设备
YYK
2026/07/20 15:36

最关键的作用就是解放 CPU
YYK
2026/07/20 15:39

所以译码器就藏在这个 IO 逻辑当中
YYK
2026/07/20 15:51

所以在设备控制器当中还会存在寄存器来接收命令参数、命令译码器来翻译命令
YYK
2026/07/20 15:39

通过数据总线，并行写入设备控制器
YYK
2026/07/20 15:40

控制器中需要接收数据，所以需要设置数据寄存器
YYK
2026/07/20 15:41

所以需要状态寄存器，用其中的每一位来反应设备的某种状态
YYK
2026/07/20 15:42

所以控制器当中应该设置地址译码器来翻译从哪个设备来的信息
YYK
2026/07/20 15:45

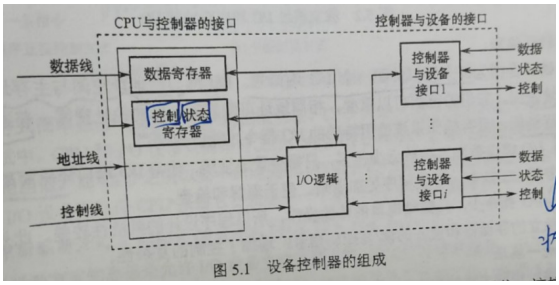


图 5.1　设备控制器的组成

IO 接口的类型：

①按数据传送方式：并行接口和串行接口

②按主机访问 IO 设备的控制方式：查询接口、中断接口、DMA 接口

③功能选择灵活性：可编程接口（可以通过变成改变接口功能）、不可编程接口

IO 端口（IO 接口内的寄存器）：

这个名字很有歧义，但要清楚的记得，这是寄存器，而不是别的东西
YYK
2026/07/20 15:52

①数据寄存器：缓存从设备/CPU 发来的数据

②状态寄存器：保存设备的执行结果或者状态信息，CPU 需要判断设备是否可用

③控制寄存器：由 CPU 写入，启动命令或者更改设备模式

每个端口都有一个端口地址，供 CPU 访问来选择对应的寄存器。两种编址方式：

	独立编址	统一编址（内存映射 IO）
特点：	存储器与寄存器编址范围分开，可能同一个地址 1 对应两个不同设备。用户程序不能对端口访问只能由操作系统通过特殊 IO 指令访问。	寄存器和内存按照统一编址。
	IO 端口译码简单，只需要少量	不需要专门的 IO 指令，CPU

优点：	地址线，寻址速度快。使用专用的 IO 指令，程序更清晰，便于理解和访问。	对 IO 访问更方面，IO 访问的保护机制可由虚拟存储管理系统实现，无需专门设置。
缺点：	IO 指令太少，只能简单传输操作，程序的设计灵活性差，CPU 需要提供两组独立的存储器和设备的读写控制信号。	主存容量变小，需要区分存储器和 IO 端口，译码电路复杂，译码速度变慢。

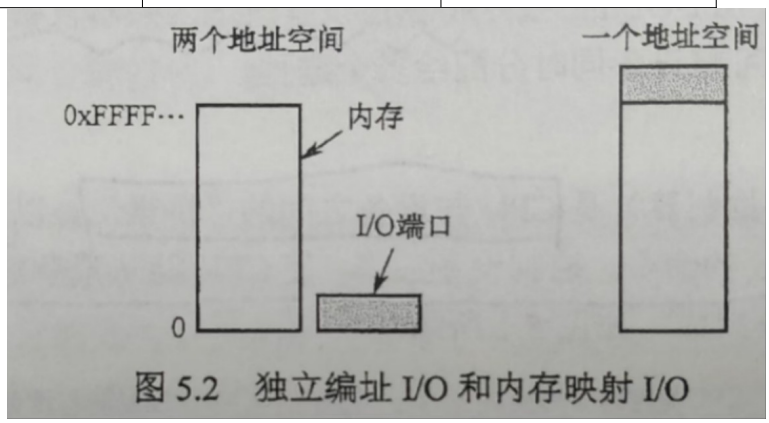
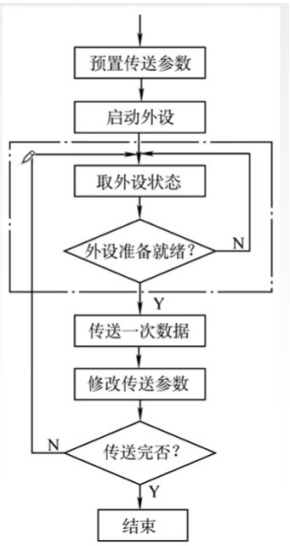
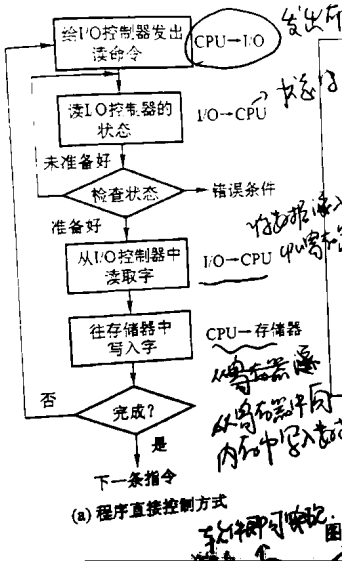


图 5.2　独立编址 I/O 和内存映射 I/O

2.I/O 控制方式

1.程序直接控制方式（论询方式）：





①工作流程：CPU 向 IO 接口发送一条 IO 指令，启动 IO 设备并读取一个字（这是读命令），然后从 IO 接口中取出该设备的状态，若准备完成，则从 IO 控制器中读取一个字（节）。

②独占查询和定时查询

独占查询	定时查询
一旦设备启动，CPU 就一直查询接口状态，外设和接口完全串行工作。	CPU 周期性地查询接口状态，每次等到条件满足才进行一个数据的传送，传送完成后返回用户程序，定时查询等待时间间隔和设备的数据传输速率有关。

③优点：软件即可实现，硬件消耗量少，比较简单

④缺点：CPU 被独占的话，消耗大量时间等待，CPU 的利用率很低。

2.中断驱动方式：

CPU 发出对设备的控制命令之后，转而返回继续执行任务（如后续任务不需要改数据的话，如果需要该数据，那么还需要切换进程），设备准备好数据之后给 CPU 发送中断信号，然后 CPU 执行中断处理程序，取出数据
YYK
2026/07/20 16:14

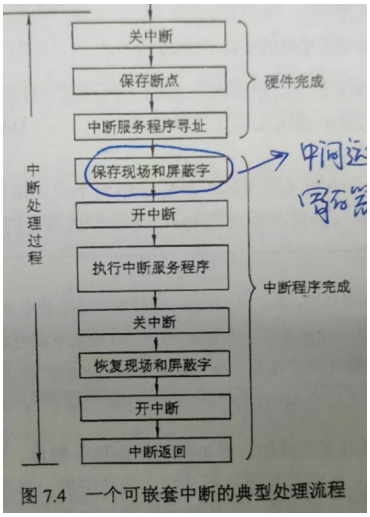


图 7.4　一个可嵌套中断的典型处理流程

①执行过程：对于 CPU 来说，它需要请求某个设备传送数据，那么就发出读指令给 IO 接口，然后该进程被阻塞，保存上下文，转而去运行其它程序。在每个指令周期的末尾，CPU 会检查是否有中断信号，若有来自于设备控制器的终端时，CPU 保存上下文，转去执行中断处理程序来处理该中断请求，CPU 从设备控制器读一个字的数据传送到 CPU 的寄存器，并存入主存。中断处理完成后，恢复该进程（或者其它进程）上下文，然后继续运行。

对于设备控制器来说，从设备中读入数据到 IO 端口的数据寄存器，读好之后便向 CPU 发送中断信号。设备控制器得到 CPU 的请求取出数据之后，将数据放在数据线上传送到 CPU 的寄存器。

②优点：主动向 CPU 报告 IO 准备完成，CPU 和设备并行工作，CPU 利用率得到提升。

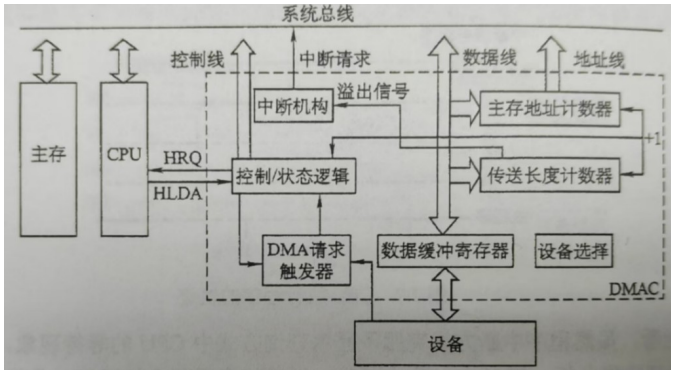
③缺点：每次传送数据都需要经过 CPU 寄存器，并且传送方式是以字（节）为单位，若对于块设备的 IO 操作，效率很低

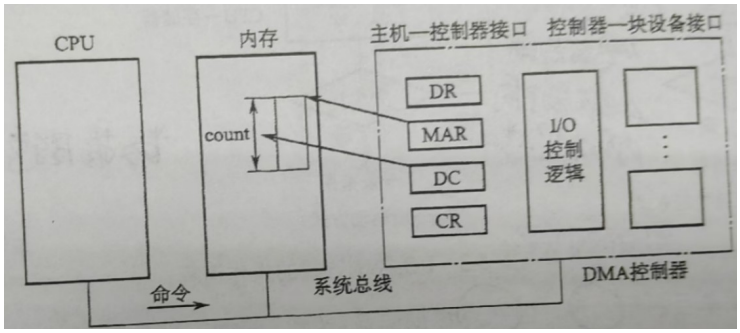
因为以字节为单位还是不方便，每个字节都需要进行一次中断，所以继而引入了直接存储方式，传输单位为数据块
YYK
2026/07/20 16:26

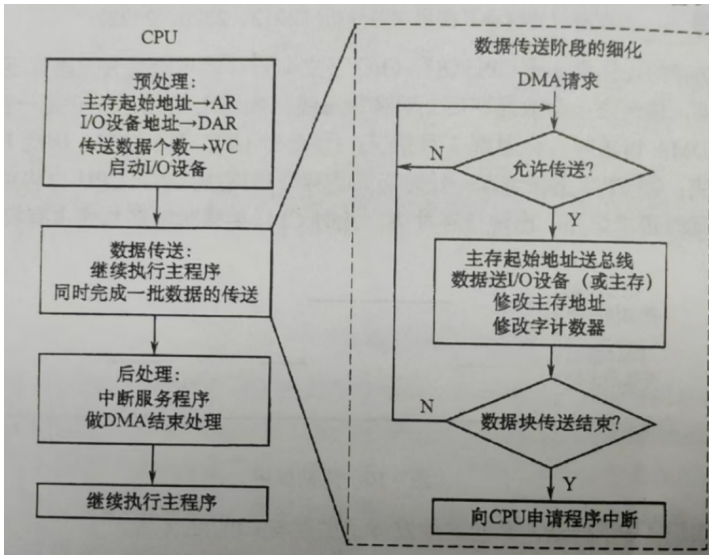
④中断处理程序是第一个获得字符型设备输入信息的（2010 真题）

3.DMA（直接存储器存取）方式：

DMA 是 I/O 接口的一部分（或依赖 I/O 接口工作），I/O 接口提供设备连接的基础功能，DMA 则是 I/O 接口中用于提升传输效率的高级硬件模块
YYK
2026/07/21 15:06


系统总线
控制线
中断请求
数据线
地址线
主存
CPU
中断机构
溢出信号
主存地址计数器
+1
HRQ
HLDA
控制/状态逻辑
传送长度计数器
DMA请求
触发器
数据缓冲寄存器
设备选择
DMAC
设备


CPU
内存
主机—控制器接口
控制器—块设备接口
DR
MAR
DC
CR
count
I/O
控制
逻辑
命令
系统总线
DMA控制器


CPU
预处理:
主存起始地址→AR
I/O设备地址→DAR
传送数据个数→WC
启动I/O设备
数据传送:
继续执行主程序
同时完成一批数据的传送
后处理:
中断服务程序
做DMA结束处理
继续执行主程序
数据传送阶段的细化
DMA请求
N
允许传送?
Y
主存起始地址送总线
数据送I/O设备（或主存）
修改主存地址
修改字计数器
N
数据块传送结束?
Y
向CPU申请程序中断

①执行过程：思想是在 IO 设备和内存之间开辟直接的数据交换通路，避开 CPU。所以采用数据块为单位，从设备直接到内存，不经过 CPU，在传送一个或多个数据块的开始和结束才需要 CPU 干预。CPU 收到设备的 DMA 请求，CPU 向 DMA 控制器发送一个命令，将 IO 控制权交给 DMA，然后给 DMA 设置 MAR 和 DC 初值，DMA 控制器直接与内存交换，每次传送一个字，直到全部传送完毕，向 CPU 发送中断信号。

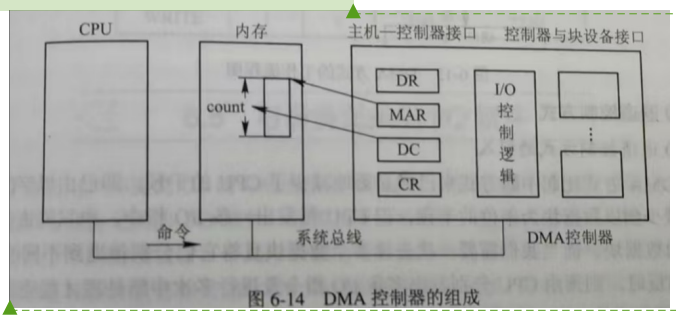


图 6-14　DMA 控制器的组成

这个过程是：每次从磁盘中取出一个字节的数据装入 MAR 中，然后再挪用一个存储器周期（读写操作才有的）将该字节数据传送到 MAR 所指示的内存单元中。
然后便对 MAR+1 ， DC-1 操作，若减 1 后不为 0 ，则继续传送下一个字节！
YYK
2026/07/20 16:45

看这图解释的多好，一眼就能看出来控制器中 MAR 、 DC 的作用，但一定要看清楚这个 MAR 存放的是内存中的地址，即要从磁盘中读入数据时，这个 MAR 指的是读入数据所需要放在内存中的地址。

而从磁盘中读出数据的地址，是通过 CR 中的命令直接交给 IO 逻辑。然后 CPU 就可以转头去执行其他工作了
YYK
2026/07/20 16:31

②四类寄存器：

命令/状态寄存器 CR	暂存从 CPU 发来的 IO 命令、或者设备的状态信息。
内存地址寄存器 MAR	输入内存时，存放目标内存的起始地址。输出时，存放内存到设备的内存源地址。
数据寄存器 DR	暂存从内存到设备、设备到内存的数据
数据计数器 DC	存放本次要传送的字节数

他们是不需要堆栈指针寄存器的

YYK
2026/07/21 15:21

③优点：以块为传送单位，CPU 介入频率进一步降低，CPU 数据传送不再经过 CPU 的寄存器，CPU 和设备的并行操作成都得到了进一步提升。

④缺点：硬件复杂度高、CPU 与 DMA 引发总线竞争、绕过 CPU 写数据可能引发内存与 Cache 数据不一致、外部绕过 CPU 破坏内存数据。

⑤一些注意点：

预处理程序和后处理程序都运行在内核态

预处理程序是请求 IO 的进程、负责后处理程序的是中断处理程序，但是中断处理程序不是一个单独的进程。

预处理程序需要让 CPU 参与设置相关调用参数，然后阻塞自身，直到系统调用完成返回，后处理阶段请求 IO 的进程仍然属于阻塞态。

⑥预处理和后处理

DMA（直接内存访问）的“预处理阶段”，指的是在数据传输正式开始前，CPU 和 DMA 控制器（DMAC）需要完成的一系列准备工作。这一阶段的核心目标是：明确“传输什么数据、从哪里传、传到哪里、传多少”，并建立 DMA 传输的基本规则，为后续“无需 CPU 干预的自主传输”奠定基础。

3.I/O 软件层次结构 （理解不够）☆☆☆☆

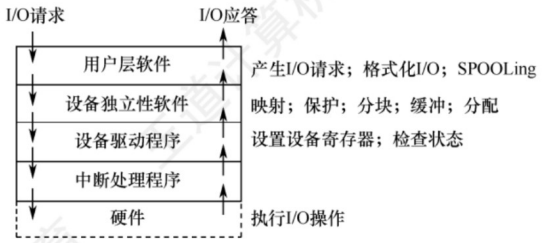


图 5.5 I/O 层次结构

（1）用户层软件：提供库函数接口供用户使用 IO 设备，这些库函数本质上底层就是一组系统调用。

（2）设备独立软件：逻辑设备名到物理设备名的映射、设备保护（防止非法访问设备）、设备分配与回收、数据块的分块与合块、缓冲区管理、设备命名、设备编号（这个号叫做绝对号）、向用户层提供统一接口等

这个阶段处于内核态，并且该进程处于运行状态。
而后处理阶段属于阻塞态、内核态
YYK
2026/07/21 15:12

产生 IO 请求就是用户的操作产生了 IO 请求；
I/O 操作的格式化是指对 I/O 数据的形式和结构进行规范处理。比如在进行文件读写时，文件中的数据可能有特定的格式要求，如文本文件中每行的结束符、二进制文件中数据的字节序等。再比如向打印机发送打印请求时，需要将文字、图像等内容按照打印机可识别的指令和格式进行组织，像设置字体、字号、页边距等打印格式信息，
YYK
2026/07/20 17:18

设备独立性是指：应用程序所用的设备不局限于某个物理设备，而是使用逻辑设备名来使用设备，但是在该层映射为具体的物理设备。而设备独立性软件就是实现该功能的
YYK
2026/07/20 17:41

设备无关软件是屏蔽了各个设备之间的差异，提供同一的执行接口，向下传递给具体的设备驱动程序来执行具体操作，例如向上屏蔽了 U 盘和磁盘之间的差异，向下交给磁盘 /U 盘驱动程序执行读写操作
YYK
2026/07/21 15:32

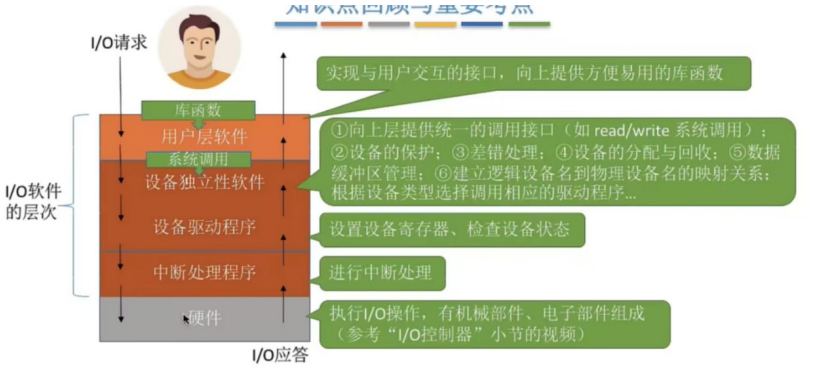
在 I/O 操作中，有时用户请求读写的数据量较大，而物理设备每次能处理的数据单元大小有限，需要将大的数据块划分成若干较小的数据块。比如磁盘存储时，文件数据可能会被分成多个磁盘块来存储，每个磁盘块大小通常是固定的（如 4KB）。合块是当多个较小的数据块需要连续存储或传输，且满足一定条件时，会将它们合并成一个较大的数据块。这可以减少存储和传输过程中的额外开销，比如减少磁盘寻道次数等。例如，在文件系统进行垃圾回收或碎片整理时，可能会把分散的小文件块合并成更大的连续块，以提高存储效率和后续的访问速度。

缓冲区是内存中开辟的一个空间，管理可缓解 CPU 与 I/O 设备速度不匹配的矛盾，减少 I/O 操作对 CPU 的中断频率，提高 CPU 和 I/O 设备的并行性，从而提升整个系统的性能和效率。
YYK
2026/07/20 17:22

（3）与硬件直接相关，负责将上层软件的抽象 I/O 请求转换为具体的设备操作。主要工作是设置设备寄存器，启动设备、初始化设备，检查设备状态等、检查设备操作合法性。

例如磁盘中转换为符合磁盘物理特性的机械运动（寻道、旋转）和电气操作（数据读写）
YYK
2026/07/20 17:21

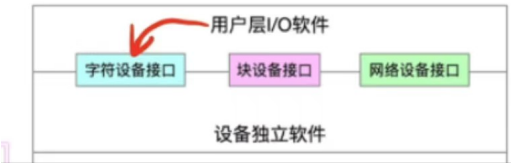
（4）当 I/O 操作完成或发生异常时，由硬件触发中断，中断处理程序进行相应处理，如保存现场、处理中断事件、恢复现场等。



理解并记住I/O软件各个层次之间的顺序，要能够推理判断某个处理应该是在哪个层次完成的（最常考的是设备独立性软件、设备驱动程序这两层。只需理解一个特点即可：直接涉及到硬件具体细节、且与中断无关的操作肯定是在设备驱动程序层完成的；没有涉及硬件的、对各种设备都需要进行的管理工作都是在设备独立性软件层完成的）

4.输入/输出应用程序接口 （理解不够）☆☆☆☆

应用程序 IO 接口的分类：



字符设备接口	块设备接口	网络设备接口
①传输以字符为单位 ②传输速率低、不可寻址 ③采用中断驱动方式	①传输以块为单位（例如磁盘） ②传输速率高、可寻址	

因为它常用于处理无结构的数据、没有明确的分隔符和边界、难以寻址
YYK
2026/07/20 17:43

④都属于独占设备，所以接口中需要提供打开和关闭操作（是指对互斥锁的访问），以实现互斥共享。	③通常采用 DMA 方式	
get 操作从字符缓冲区得到一个字符；put 写入一个字符。	read 操作读入多个字节、write 写入多个字节。seek 修改读写指针的位置	socket 系统调用产生套接字 connect 链接另一个 ip 的套接字行程端口之间互联

阻塞 IO/非阻塞 IO：

	阻塞 IO	非阻塞 IO
原理：	调用 IO 操作后，进程就被阻塞，并移动到阻塞队列，IO 操作完成后进程才被唤醒，移动到就绪队列。	调用 IO 操作后，进程不被阻塞，但进程需要不断询问 IO 操作是否完成，在 IO 执行阶段，进程还可以做其它事情。当 IO 操作完成后，系统将数据从内核复制到用户空间，今年继续处理数据。
优点：	操作简单、容易实现、并发量小的应用开发	进程在 IO 期间不会被阻塞，可以做其它事情，适合并发量大的应用开发。
缺点：	IO 执行阶段，进程一直被阻塞	轮询 IO 是否完成工作，占用 CPU 时间

(二)设备独立软件（设备无关软件）

1.缓冲区管理

一些基础知识点：

①设备的独立性是指：用户编程使用的设备与实际设备无关，只需要指明使用设备（可以通过文件名来当做逻辑名称）即可，不需要指明具体的物理设备

②设备独立性软件是指：不直接依赖特定硬件设备，而是通过抽象层或标准化接口与硬件交互的软件设计方式。其核心目标是提高软件的可移植性、兼容性和可维护性，使同一套代码能在不同硬件环境中运行而无需大量修改。

例如一些适合用设备独立性软件来管理的是：检查用户是否有权使用设备、缓冲区管理。

而向设备寄存器写命令是由设备驱动器完成的、将二进制整数转换成 ASCII 格式打印是由 IO 库函数完成。

磁盘高速缓存：

①把内存的一部分空间当做磁盘（开辟的空间大小固定）、或者把剩余的内存空间当做缓冲池供应分页系统和磁盘 IO 时共享，因为 CPU 访问内存速度更快。

②物理上属于驻留在内存的盘块、逻辑上属于磁盘。

③进程正在运行的程序既存储在内存、也存储在磁盘上。

缓冲区：

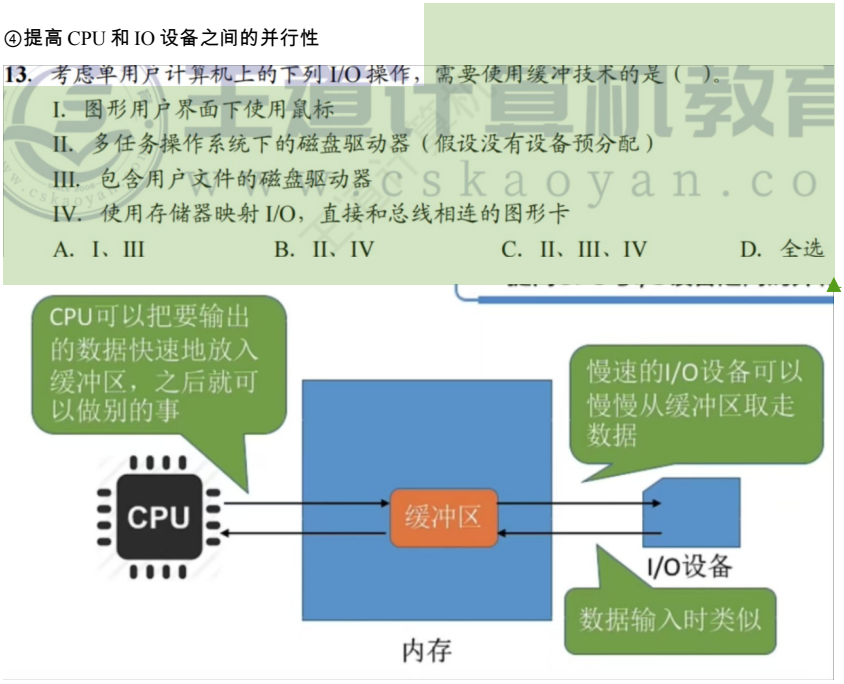
设立缓冲区的目的：

①缓和 CPU 和 IO 设备速度不匹配的问题

②减少对 CPU 的中断频率（字符型设备每输出一个字符就要请求一次中断，所以引入缓冲区可以填满缓冲区后才请求一次中断），放宽对 CPU 中断响应时间的限制。

③解决基本数据单元大小（数据粒度）不匹配的问题（例如输出进程每次输出一块的数据，但是 IO 设备每次只能输入一个字符）

④提高 CPU 和 IO 设备之间的并行性



需要使用缓冲技术的是：
：鼠标产生数据速度快且不规则，CPU 处理速度快但不能随时响应，用缓冲技术可暂存鼠标数据，使 CPU 能在合适时间读取处理，减少数据丢失，提高交互流畅度。
：多任务下多个进程可能同时访问磁盘，磁盘读写速度慢，缓冲技术能在内存设缓冲区，先将数据写入缓冲区，再批量写入磁盘，减少磁盘 I/O 次数，提高整体性能。
：磁盘与内存速度差异大，读写用户文件时，缓冲技术可缓存文件数据，减少磁盘访问，如读文件时先读入缓冲区，后续访问直接从缓冲区读取，提高数据读取效率。
：图形卡生成图像数据速度快，显示器刷新速度相对慢，缓冲技术可在内存设帧缓冲区，暂存图形卡生成的数据，按显示器刷新节奏输出，保证图像显示稳定流畅。
YYK
2026/07/21 16:16

缓冲区实现方法：

①硬件实现，成本太高

②内存作为缓冲区，下列根据缓冲区的数量，分为不同的缓冲技术

③需要缓冲技术的是：

（1）单缓冲

①用户进程每次发出一个 IO 请求，操作系统便在内存中为之分配一个缓冲区，通常一个缓冲区的大小就是一个块。

②只有缓冲区空时，才能向缓冲区输入信息，并且只有充满之后，才能取走信息。

③单缓冲区处理每块数据的平均时间为 Max(C,T)+M

Movement time

Computation time

Total time

指的是从缓冲区传输到 CPU 的时间

YYK
2026/07/21 15:38

指的是用户工作区处理数据的时间

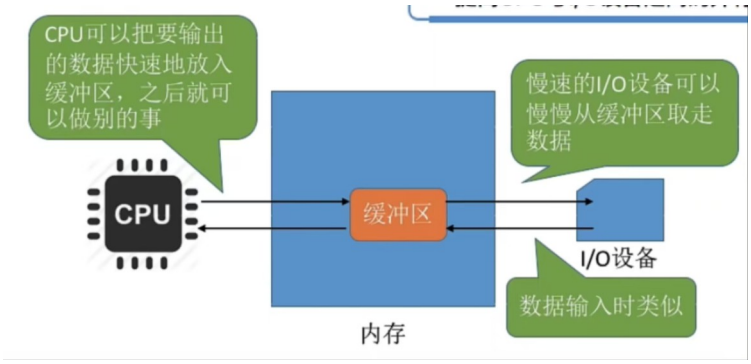
YYK
2026/07/21 15:38

这里指的并不是总时间，而是从开始传输数据到传输结束、准备好下一次传输的时间 T

YYK
2026/07/21 15:37

④缓冲区是共享资源，使用时必须互斥

⑤缓冲区的进程访问同步问题才是缓冲区管理者应该考虑的重要问题，要考虑进程对缓冲区的访问顺序。



（2）双缓冲

①设立两个缓冲区，轮流输入数据和取出数据。

②双缓冲区每块数据处理的平均时间是 Max(C+M,T)，比较送到用户区时间+CPU 处理时间和磁盘送到缓冲区时间的对比。

③设立双缓冲区还可以实现两个主机之间同时互相发送数据

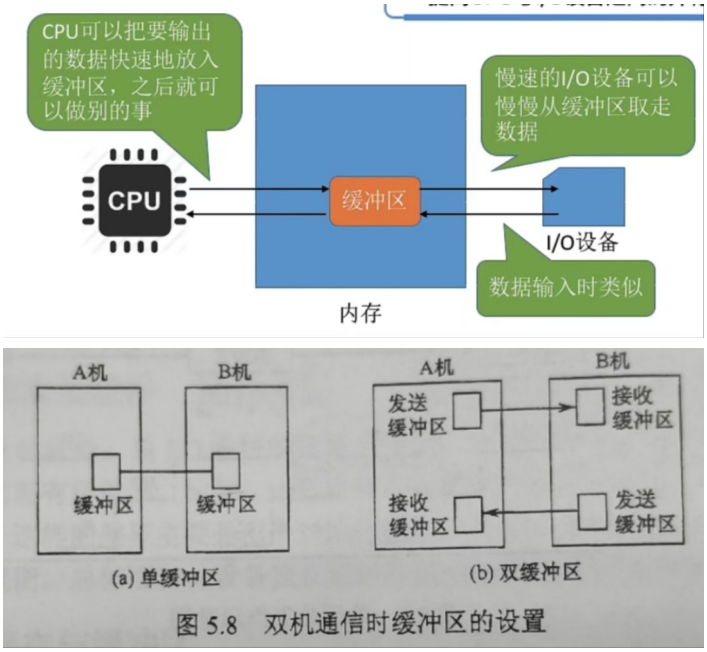


图 5.8　双机通信时缓冲区的设置

（3）循环缓冲

①双缓冲区，如果输入和输出的速度基本相同，那么效果会很好，但是速度差异很大，效果就极差。

②因此引入多缓冲区循环队列，一个指针 in 指向输入缓冲区块，另一个指针 out 指向输出缓冲区块，缓冲区块大小相等。

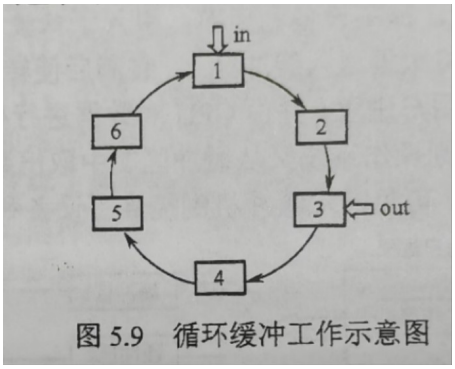


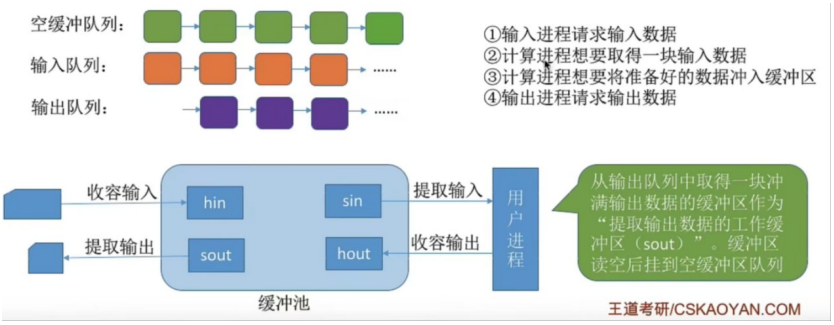
图 5.9　循环缓冲工作示意图

（4）缓冲池

①缓冲池=管理自身的数据结构+一组操作函数的管理机制

②缓冲池可供多个进程共享使用

③缓冲池由多个系统公用的缓冲区组成。



④输入进程请求输入数据，则从空缓冲队列队头取出一个空缓冲区，用户收容输入，输入后的缓冲区被挂到输入队列队尾等待其它进程取走数据。其余三个过程不再介绍，上述同理。

⑤缓冲池是在**内存**中建立的

高速缓存与缓冲区的对比：

表 5.1 高速缓存和缓冲区的对比

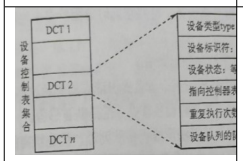
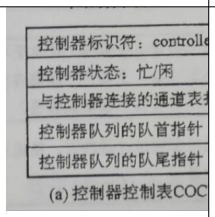
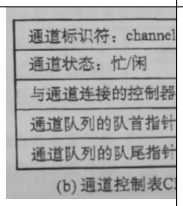
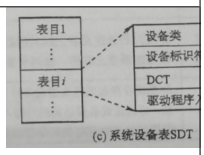
		高速缓存	缓冲区
相同点		都介于高速设备和低速设备之间	
区别	存放数据	存放的是低速设备上的某些数据的复制数据，即高速缓存上有的，低速设备上面必然有	存放的是低速设备传递给高速设备的数据（或相反），而这些数据在低速设备（或高速设备）上却不一定有备份，这些数据再从缓冲区传送到高速设备（或低速设备）
	目的	高速缓存存放的是高速设备经常要访问的数据，若高速设备要访问的数据不在高速缓存中，则高速设备就需要访问低速设备	高速设备和低速设备的通信都要经过缓冲区，高速设备永远不会直接去访问低速设备

①高速缓存技术例如快表、Cache 等，**是为缓解高速设备和低速设备不匹配的问题**

②缓冲区是为了解决高速设备和低速设备速度不匹配的问题

2.设备分配与回收

（1）系统中设备分配的数据结构

设备控制表 DCT	控制器控制表 COCT	通道控制表 CHCT	系统设备表 SDT
系统为每个设备分配一张 DCT 表	每个设备控制器对应一张 COCT	每个通道对应一张 CHCT	整个系统只有一张 SDT
			
①设备类型：打印机、扫描仪 ②设备标识符：设备的物理名称，系统中唯一。 ③设备状态：忙碌/空闲（改为空闲即可实现设备回收） ④指向控制器表的指针：该指针指向对应的控制器表 COCT ⑤重复执行次数或时			记录已连接到系统的所有物理设备的情况，每个物理设备对应一个表目

间：达到一定次数或时间认定次次 IO 失败 ⑥设备队列的队首指针：指向正在等待该设备的进程队列（PCB 组成）的队首。			

设备分配的步骤：

①进程提供所请求的物理设备的设备名，然后操作系统在 SDT 系统设备表中，逐个查询相关表目，找到对应设备的 DCT 设备控制表。

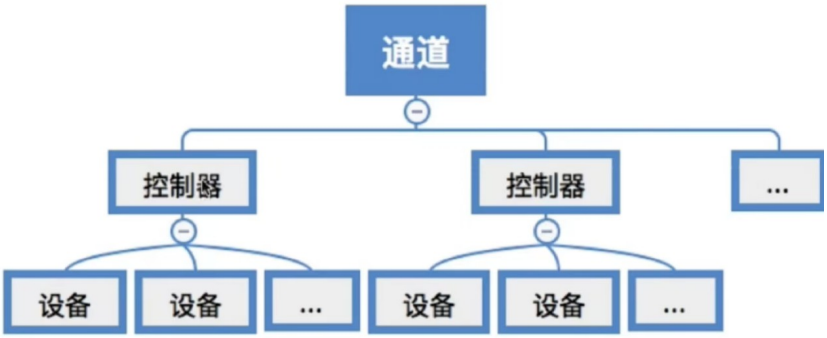
②根据设备控制表 DCT 中的“设备状态”判断该设备是否可用，若不可用则把该进程的 PCB 挂到该设备的等待队列中；若不忙碌则根据一定策略讲设备分配给进程。

③不仅要把该设备分配给进程，还要把该设备的设备控制器分配给该进程。该控制器控制表存储在 DCT 的指针指向 COCT。

④查询 COCT 状态，若忙碌，则将该进程挂到该控制器的等待队列中，若不忙碌，则将该控制器分配给该进程。

⑤根据 COCT 的与控制器连接的通道表指针指向 CHCT，查询 CHCT 对应通道的状态，若忙碌，则挂到该通道的等待队列中；若不忙，则将该通道分配给进程。

⑥查询 System Device Table----查询 Device Control Table----查询 Controller Control Table ----查询 Channel Control Table



设备分配需要考虑的因素：

①固有属性：独占、虚拟、共享设备

②设备分配算法：FCFS 先来先服务、最高优先级优先算法（优先级仙相同的采用先来先服务）

③设备分配中的安全性：安全分配（IO 请求后，该进程被阻塞，CPU 与 IO 设备串行）、不安全分配（IO 请求后，CPU 继续执行该进程，需要时可以发出第二个、第三个 IO，只有需要的设备被其它进程占用后才阻塞该进程，一个进程可同时操作多个设备）

④静态分配（运行前将所需资源分配给进程）还是动态分配（进程运行过程中动态的给其分配）

⑤逻辑设备与物理设备的映射关系（这个技术使得所有物理设备都被抽象成逻辑设备，运行代码时不会因为物理设备的改变而崩溃，2023 真题）

由于物理设备名用户编程不方便、更换了相同的设备但是原有的设备名无法启动新设备、即使有空闲的相同的设备但是由于物理设备名不匹配导致进程被阻塞，所以引入了：

逻辑设备名到物理设备名的映射：

把打印机抽象成“特殊文件”，通过该文件的文件名来当做逻辑名

①通过逻辑设备名“打印机”在 SDT 系统设备表中找到对应类型设备的表目，只有所有该类型的设备全部忙碌，才阻塞进程。

②若某个设备空闲，则分配给该进程，并且在逻辑设备表 LUT（Logical Unit Table）中新增表目，记录逻辑设备名和物理设备名的映射。并且再次通过逻辑设备名访问该设备时，可以映射到物理设备名，也能找到该设备的驱动程序入口地址。

逻辑设备名	物理设备名	驱动程序入口地址
/dev/printer	3	1024
/dev/tty	5	2046
…	…	…

④可以通过两种方式来设置逻辑设备表：整个系统设置一张（要求逻辑设备名不能重复，适用于但用户操作系统）、每个用户设置一张（不同用户的逻辑设备名可以重复，适用于多用户操作系统）

逻辑设备名	系统设备指针
/dev/printer	3
/dev/tty	5
…	…

3.假脱机技术(SPOOLing)

①用软件方式实现、需要多道程序设计技术（该技术可以提高单机资源利用率）支持、需要外存磁盘开辟“井”、将独占设备改造成共享设备、由系统建立输入输出进程控制设备与井之间数

据传送、提高 IO 速度、实现虚拟设备功能让进程误以为独占设备、以空间换时间、提高独占设备资源利用率。

②引入假脱机技术是为了缓解 CPU 和 IO 设备速度的矛盾，将 CPU 产生的数据先放到一个存储区域“磁盘内”，然后让 IO 设备慢慢的从磁盘取走数据输出。

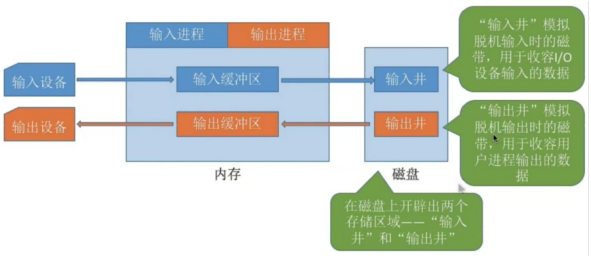
记着一定是磁盘内开辟一段空间

YYK
2026/07/21 16:41

③输入井和输出井：

他们本质是磁盘开辟的一段空间，将输入输出数据保存为文件，各个文件之间排成链接队列。

用户的产出结果首先被送到磁盘固定区域，尽快中间要经过内存固定区域的缓冲区，但在逻辑上，磁盘固定区域才是逻辑终点。

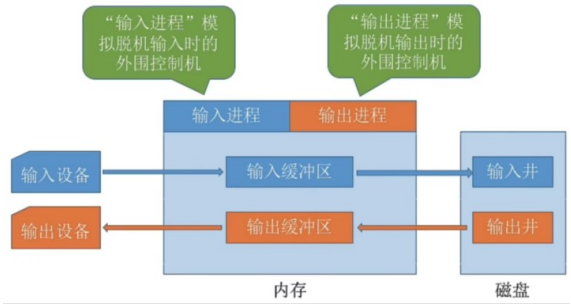


④输入缓冲区和输出缓冲区：

在内存中开辟空间，目的是将磁盘的数据引入内存并交给 IO 设备输出。

⑤输入进程和输出进程：

要求数据必须经过输入输出缓冲区（内存），才能交付给输入输出设备，因为没办法之间建立设备与磁盘之间的联系！！！并且这个过程是操作系统控制实现的，用户作业根本不知道这个过程，对它来说是透明的。



⑥井管理程序：

用于控制作业与磁盘井之间的信息交换。

⑦打印机举例：来自不同用户的打印文件先暂存在磁盘空间，然后将用户打印请求表挂在假脱机文件队列上，这样好像每个用户都独占了打印机，但实际是共享了打印机。所以用户进程实际分配到的不是物理设备，而是磁盘空间。整个过程利用到了文件系统（将文件放到磁盘），设备管理程序（控制打印机输出）。

⑧SPOOLing 系统由三大部分组成：

预输入程序	输入井和输出井
井管理程序	输入输出缓冲区
缓输出程序	输入进程和输出进程

4.设备驱动程序接口（理解不够）☆☆☆☆☆

①意思就是：下层的驱动程序要根据上层的设备独立性软件提供的驱动程序接口的规范，来编写自己的驱动程序。上层规定了函数名、参数名、参数个数等，厂商就要根据不同操作系统提供的信息来编写对应的驱动程序。



②设备驱动程序的功能：

a.接收上层应用的抽象 IO 请求，转化成具体的要求来交给设备控制器，让其执行操作，还要从设备控制器中取出它的设备状态信息和 IO 操作的完成情况反馈给进程；

例如接收到 write（ file，1024 ）等，翻译成就是把相关的内容写到 xx 磁道 xx 扇区
YYK
2026/07/21 15:58

b.检查用户 IO 请求合法性、了解设备工作状态、传递与设备操作有关参数、设置设备工作方式；

发出 IO 命令，若设备空闲则执行操作，若不空闲则挂到等待队列；

及时响应由设备控制器发来的中断请求，根据终端类型调用响应的中断处理程序。

c.负责处理磁盘调度（磁盘驱动程序）

d.驱动程序不对 IO 传回来的数据进行分析和缓冲，这个功能是由操作系统完成的。

f.初始化设备（如初始化设备控制器的寄存器值）

g.读写设备的操作是由驱动程序控制完成的（2022 真题）

③设备驱动程序的特点：

a.要根据设备的 IO 控制方式来编写设备驱动程序，常用的是中断驱动和 DMA（2022 真题）

所以设备的驱动程序与 IO 控制方式息息相关！
YYK
2026/07/21 17:13

b.要根据不同的设备硬件来编写驱动程序，所以不同类型的设备拥有不同的驱动程序

c.目前很多设备驱动程序固化在 ROM 中

d.设备驱动程序允许**同时多次调用**

e.不同操作系统的相关设备的驱动程序是不同的，要根据不同情况进行定制。

f.驱动程序可以用汇编、部分也可以用高级语言编写。

g.执行驱动程序时可能会发生阻塞（2022 真题：若采用直接控制方式，不会发生阻塞，因为一直占用 CPU 进行轮询操作；但中断控制方式，会把 CPU 让给其它进程，阻塞自身让其只完成 IO 过程；驱动程序对 DMA 控制器初始化后，发出启动 DMA 传送命令，外设与内存之间传送数据，CPU 被让出。）

④设备驱动程序的处理过程：

将抽象要求转化成具体要求（例如计算出具体存储的磁盘面、扇区号）-----对具体要求结合设备功能进行校验（如果该设备不具备完成该要求的任务，就终止了，例如检查该磁盘面、扇区号是不是越界了）-----检查设备状态是否空闲（例如检查磁盘是否空闲）-----空闲后则传递必要参数-----启动 IO 设备

用户程序发出磁盘 IO请求后的处理完整流程（2011、2012 真题）：

磁盘 IO 的性能，可以通过重排 IO 请求的顺序，减少平均等待时间
YYK
2026/10/26 09:02

1. 用户程序发起 I/O 请求（用户级 IO 软件）

用户程序提供参数包括：文件描述符 fd（指向内核中打开的文件对象）、目标缓冲区地址 buffer（位于用户态内存空间）、读取字节数。

2. 系统调用处理（与设备无关的系统调用层软件）

①陷入内核态：read()触发软中断，CPU 切换到内核态。

read 要求用户提供三个输入参数：①文件描述符 fd；②buf 缓冲区首址；③传送的字节数 n。read 的功能是试图从 fd 所指示的文件中读入 n 个字节的数据，并将它们送至由指针 buf 所指示的缓冲区中。

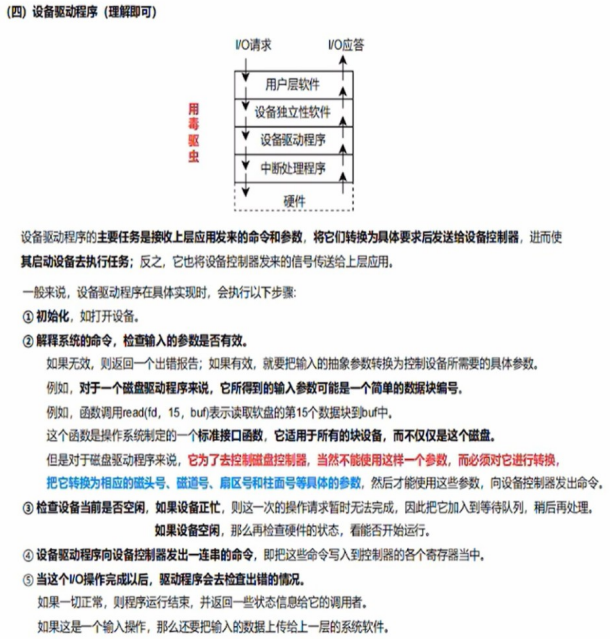
read 里面填写的并不是文件地址或者文件名，而是先通过 open 打开文件获得文件描述符 fd，随后通过 fd 来进行操作；同时如果读取的文件没有在内存中，则进程休眠（阻塞）；
YYK
2026/10/26 08:59

②内核检查 fd 文件描述符是否有效、buffer 地址是否合法。

③根据 fd 找到对应的文件对象（如 struct file）。

3. 设备驱动程序处理（设备驱动程序）

将抽象要求转化成具体要求（例如计算出具体存储的磁盘面、扇区号）-----对具体要求结合设备功能进行校验（如果该设备不具备完成该要求的任务，就终止了，例如检查该磁盘面、扇区号是不是越界了）-----检查设备状态是否空闲（例如检查磁盘是否空闲）-----空闲后则传递必要参数-----启动 IO 设备



4. 磁盘硬件执行

①磁头移动到目标扇区（寻道时间 + 旋转延迟）。

②数据从磁盘扇区读取到控制器缓存（或通过 DMA 直接写入内存）。

③完成后，磁盘控制器触发硬件中断。

5. 中断处理程序（中断处理程序软件）

①暂停当前任务，执行磁盘控制器注册的中断服务程序。

②读取控制器状态寄存器，确认操作成功或失败。

③若为 DMA 传输，无需额外数据拷贝；否则从控制器缓存拷贝数据到内存。

④标记 I/O 请求完成，唤醒等待该 I/O 的进程

中间需要把进程 P 插入到就绪队列！

6. 返回到用户程序

①进程从内核态返回到用户态，read()系统调用结束。

②用户程序从 buffer 内存缓冲区中获取数据，继续执行。

数据流转全貌

1. 用户态 → 内核态：read()参数通过寄存器/栈传递，缓冲区地址被验证。
2. 内核态 → 磁盘控制器：
3. 磁盘 → 内存：数据通过 DMA 或中断拷贝到内核缓冲区。
4. 内核态 → 用户态：数据最终拷贝到用户空间。

进程执行键盘 scanf 系统调用的完整处理流程（2017 真题）：

流程如下：

①进程 P 处于运行状态，然后进程 P 在用户态执行代码，调用 scanf() 函数，需要等待用户输入（即缓冲区的内容为空时，触发 read 系统调用），所以进程 P 进入阻塞态。

它自身是一个函数，底层有相关的逻辑，例如如果用户缓冲区不为空，则直接读走用户缓冲区的数据，如果用户缓冲区为空，则触发 read（）系统调用，并且传入三个参数： fd 文件描述符指明输入的设备、buf 用户缓冲区、 count 读取的最大字节数

YYK
2026/07/21 16:02

②进入阻塞状态（Blocked）：由于 I/O 设备（键盘）的速度远慢于 CPU，且采用中断控制方式，操作系统会将进程 P 置为阻塞状态（Blocked），等待输入完成。此时，进程 P 不再占用 CPU，操作系统会调度其他就绪进程运行（如果有）。

这个过程是执行 trap 指令陷入到内核态之后的过程

YYK
2026/07/21 16:05

③状态恢复（Ready）：当键盘输入完成（用户输入了值并按回车键），键盘控制器会触发一个硬件中断。中断处理程序会读取输入数据，并将其传递给进程 P 的缓冲区。然后返回系统调用之后，操作系统随后将进程 P 的状态从阻塞状态（Blocked）改为就绪状态（Ready），等待 CPU 调度。

④重新运行（Running）：当进程 P 被操作系统调度器选中时，它从就绪状态（Ready）进入运行状态（Running），并继续执行后续代码（如调用 f1()）。

因此，进程 P 的状态变化序列为：

Running → Blocked → Ready → Running

CPU 是否会进入内核态？

是的，CPU 会在以下两个关键点进入内核态：

①系统调用执行时：

当 scanf() 触发底层系统调用（如 read()）时，CPU 通过软中断（例如，int 0x80 或 syscall 指令）从用户态切换到内核态。在内核态中（此时 CPU 执行的还是进程 P），操作系统处理 I/O 请求：检查输入缓冲区，发现数据未就绪时，才把进程 P 阻塞，并调度其他进程。

②硬件中断处理时：当键盘输入完成时，硬件（键盘控制器）会发送中断信号给 CPU。CPU 响应中断，立即从当前状态（无论是用户态还是内核态）切换到内核态（此时是“中断态”），执行中断处理程序（此时执行的不是进程 P 了，因为它被阻塞了，）。中断处理程序在内核态中读取输入数据，唤醒阻塞的进程 P（将其状态改为就绪），并完成 I/O 操作。

```
plaintext

用户态：进程P执行 -> 调用scanf() -> 触发系统调用
          |                              |
          V                              V
内核态：          系统调用处理：进程P阻塞          <- CPU进入内核态（1）
          |                              |
          |                              V
          |                  等待键盘输入（进程P Blocked）
          |                              |
          V                              |
内核态：          中断处理：输入完成唤醒进程P      <- CPU进入内核态（2）
          |                              |
          V                              V
用户态：进程P就绪 -> 被调度 -> 继续执行
```

进程执行键盘 scanf 系统调用的数据流向（2024 真题）：

①首先键盘输入的数据先被保存到键盘设备控制器的数据寄存器中，键盘控制器通过中断线向 CPU 发送中断信号，然后触发中断服务程序。

②CPU 阻塞当前进程，转为内核态执行中断服务程序，该过程将键盘控制器内的数据传到位于内存的内核态的缓冲区，中断服务程序结束。

③操作系统然后唤醒用户进程，若执行 read 系统调用，让其将数据从内核态缓冲区传到用户态缓冲区。

④键盘输入的数据只会被保存到用户态的缓冲区，只有当出现下一条指令例如 mov R1, 28393012H，才将用户态缓冲区的键盘数据传送到 CPU 的通用寄存器组。

这个过程是由设备驱动程序执行的，与底层硬件之间相关

YYK
2026/07/21 16:07

中断服务程序结束后，这个数据还是停留在内核的缓冲区当中

YYK
2026/07/21 17:12

这个过程是普通内核程序执行的，属于设备无关软件层

YYK
2026/07/21 16:07

(三)外存管理

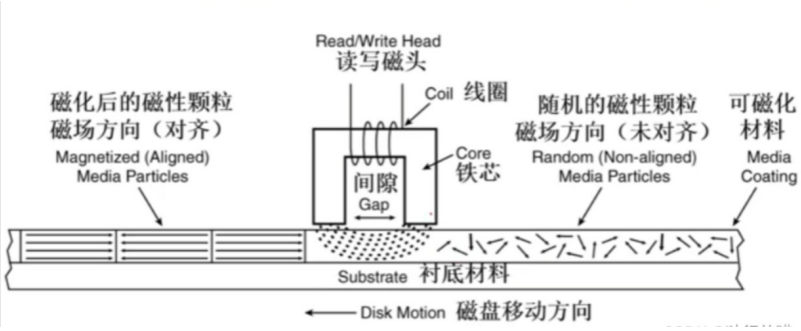
1.磁盘

磁盘结构，格式化，分区，磁盘调度方法

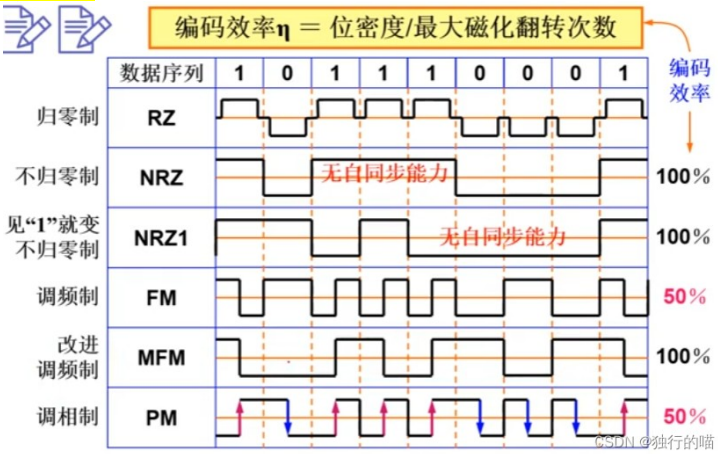
磁盘的基本概念：

1.定义

磁盘是涂有磁性介质的物理盘片，通过对磁性材料的设置磁场方向来写入 0 和 1。



磁记录方式如下：



2.磁盘存储器的优缺点

优点：

①记录介质可重复使用（只要改变磁场方向就可以轻松改变 01 状态，进而改变数据）

②记录信息可以长期保存不丢失、甚至可以脱机存档（就是脱离主机仍然保存着数据）

③非破坏性读出，不需要再生（磁场方向又不会衰减，故不需要再生）

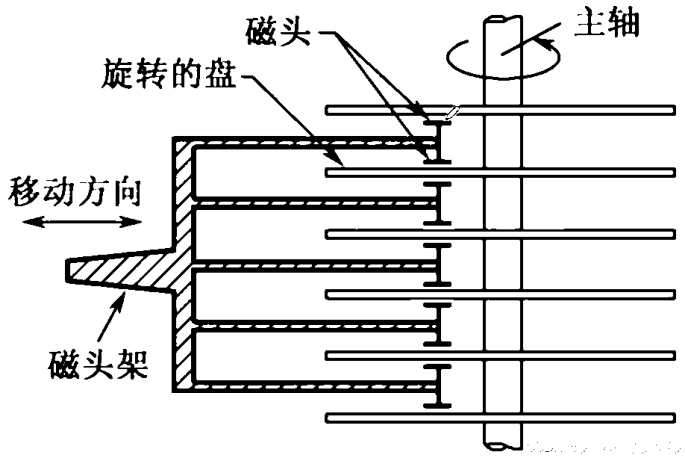
缺点：

①存取速度慢（寻道、旋转时间慢，固态硬盘只需要电子信号控制即可）

②机械结构复杂（盘片、磁头、马达、机械臂，摔一下，零件就容易坏）

③对工作环境要求较高（不能在高磁场的环境当中、湿度温度对磁头、盘片都有影响）

3.磁盘存储器的组成结构



（1）结构=磁盘驱动器（通过磁头进行读写操作的装置）+磁盘控制器（驱动器与主机的接口，解释 CPU 发来的命令并向磁盘驱动器发出控制命令，检测磁盘驱动器的状态）+盘片

（2）磁盘基本组成的解释

①盘面（磁盘一般具有多个盘面，题目会给出）

②磁道（每个盘面都有会 n 个圈圈，这就是磁道，一般来说，相邻磁道和扇区会有中间空隙隔开，以免精度错误。最重要的是，一个磁道存储的信息是一样的，所以内磁道存储的位密度更大）

③柱面（类似于一个圆柱体，是指从上到下所有盘面相同位置的磁道组成的圆柱面）

④扇区（扇区只是一个磁道被分成了多个圆弧，每个圆弧被叫做一个扇区，扇区是可寻址的最小单位）

（3）☆这里对磁盘驱动器做一个格外的解释：

普通情况下，每个盘面都有一个磁头、根据柱面号，让磁头找到对应的柱面，所有磁头“共同进退”。

但是这个世界上磁盘也分为很多种类：

①固定头磁盘：杆子不是伸缩的，每个磁道都有一个磁头

②活动头磁盘：杆子可伸缩，一个盘面只有一个磁头

③固定盘磁盘：盘片永久固定在磁盘控制器当中

④活动盘磁盘：盘片可以更换

⑤温彻斯特磁盘：最早的磁盘、磁头活动盘片固定

4. 磁盘的性能指标

（1）记录密度：道密度、位密度、面密度

（2）磁盘容量：分为格式化容量和非格式化容量。非格式化容量就是面数 x（磁道数/面）x 内圆周长 x 最大位密度。格式化容量就是面数 x（磁道数/面）x（扇区数/道）x（字节数/扇区）。由于格式化需要将磁盘分成扇区，每个扇区内部前面写上地址标志、地址、校验码、同步码等，以便存取时寻址使用。同时，为了防止转速变化引起首尾重叠，在扇区尾部还留 有一定字节的空隙。它们均需占用一些存储空间。 一般格式化容量要比未格式化容量小 10%一 20%。

（3）存取时间：寻道时间（最外道到最内道磁头移动时间的一半）+旋转延迟时间（旋转半周的时间）+传输时间

①寻道时间=启动磁头臂时间 s+磁头移动一道所需时间 m*需要移动的磁道数目 n

寻找时间（寻道时间）T_S：在读/写数据前，将磁头移动到指定磁道所花的时间。
①启动磁头臂是需要时间的。假设耗时为 s；
②移动磁头也是需要时间的。假设磁头匀速移动，每跨越一个磁道耗时为 m，总共需要跨越 n 条磁道。则：
寻道时间 $T_S = s + m*n$

②延迟时间

延迟时间T_R：通过旋转磁盘，使磁头定位到目标扇区所需要的时间。设磁盘转速为 r（单位：转/秒，或 转/分），则
平均所需的延迟时间 $T_R = (1/2)*(1/r) = 1/2r$

1/r 就是转一圈需要的时间。找到目标扇区平均需要转半圈，因此再乘以 1/2

硬盘的典型转速为 5400 转/分，或 7200 转/分

③传输时间

传输时间T_t：从磁盘读出或向磁盘写入数据所经历的时间，假设磁盘转速为 r，此次读/写的字节数为 b，每个磁道上的字节数为 N。则：
传输时间 $T_t = (1/r) * (b/N) = b/(rN)$

每个磁道要可存 N 字节的数据，因此 b 字节的数据需要 b/N 个磁道才能存储。而读/写一个磁道所需的时间刚好又是转一圈所需要的时间 1/r

磁盘转速是固有属性，无法改变，只能通过寻道来优化时间，故而引出磁盘调度算法

平均寻道长度=寻道数量/进程数

（4）数据传输速率：就是磁盘存储器在单位时间内向主机传输数据的字节数（计算传输时间时，要减去经过的间隙的时间，因为这段时间内不传送数据，所以要丢弃），磁盘转速为 r 转/s，每条磁道容量为 N 字节，则数据传输速率为 D=rN。

5. 磁盘地址

柱面（磁道）号	盘面（磁头）号	扇区号

另外还有簇号的编码方式：

簇号的编排遵循“柱面优先，磁道次之，扇区最后”，即内层的柱面簇号最小，外层的柱面簇号最大。磁道越靠上，簇号越小。（2019 真题）

物理地址《柱面号，磁道号，扇区号》，柱面号、磁道号、扇区号默认从 0 开始编号。

将簇号转换成磁盘物理地址的过程由磁盘驱动程序完成。

6. 磁盘的工作过程

寻址、读盘、写盘
每个操作都有一个对应的控制字，取出控制字然后执行这个控制字。磁盘不可能在同一时刻读写同时操作、也不可能同时进行两处位置的读操作或写操作。

磁盘的管理：

1.磁盘初始化（物理格式化、低级格式化）：

①主要目标就是划分扇区，建立磁盘的物理结构，包括磁道、扇区和柱面等。 这个过程将磁盘表面划分成许多小的、可寻址的单元。同时操作系统自动用后备扇区顶替坏扇区，这个过程对用户透明。

②这个过程会写入一些低级别的信息到磁盘，例如引导扇区（包含主引导程序、分区表、结束标志）信息

③每个扇区由头部、数据区域、尾部组成，头部和尾部包含了一些对磁盘控制器的使用信息。

④利用 CRC 还可以对扇区进行校验。

对磁盘进行分区 C、D 区：

在逻辑格式化和物理格式化之间还夹杂一个步骤是对磁盘进行分区（C 区、D 区）（2021 真题），每个分区由一个柱面或者多个柱面组成，每个分区的起始扇区和大小都记录在磁盘主引导记录的分区表中。

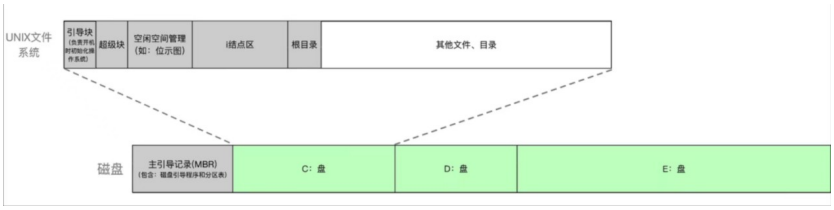
逻辑格式化：

①逻辑格式化是在物理格式化之后，在磁盘上建立文件系统。它创建文件系统所需的元数据结构，例如文件分配表 (FAT)、目录结构、空闲空间、已分配空间、空目录、根目录以及其他用于管理文件和目录的控制信息，并对这些信息进行初始化，记录在磁盘上！！！。

建立文件系统时，由文件系统确定盘块大小，无论什么类型的磁盘硬盘，都是由文件系统确定盘块大小的
YYK
2026/07/21 17:56

②这允许操作系统理解和管理磁盘上的数据。具体操作: 创建根目录、文件系统结构，并设置文件系统的参数，例如簇大小，文件系统类型 (NTFS, FAT32, ext4 等)。

③特点: 是高层操作，可以反复进行，并且不会破坏磁盘的物理结构，只会删除文件系统中的数据。



所以流程应该是磁盘物理格式化、对磁盘进行分区、磁盘逻辑格式化、安装操作系统（2021 真题）

引导块（王道编写的什么垃圾）：

①计算机启动时需要运行一个初始化程序（自举程序），初始化 CPU、寄存器、设备控制器和内存，然后启动操作系统。所以它需要找到磁盘内存储的操作系统，将它加载到内存，并转到起始地址，开始运行操作系统。

②初始化程序保存在 ROM 中，但是为了避免初始化程序改动而需要修改 ROM 硬件，所以在 ROM 中存储的是自举装入程序，即把完整的自举程序放在磁盘的启动块上，把安装自举程序的程序放在 rom 中。

③启动分区位于磁盘固定位置，具有启动分区的磁盘称为启动磁盘或者系统磁盘。

④

坏块：

利用备用磁盘块顶替掉坏的磁盘块，这个过程对操作系统透明

磁盘调度算法

（1）先来先服务（First Come First Served，FCFS）：按照请求的先后顺序进行磁盘调度。

如果后面来的请求需要等待前面的请求完成后才能执行，会导致平均等待时间较长。优点是公平，只有少量进程，且都访问簇聚的磁盘块是，性能最优；在大量进程访问磁盘是，性能接近于随机算法。

（2）最短寻道时间优先（Shortest Seek Time First，SSTF）：选择当前距离磁头最近的请求先执行。这种算法可以减少等待时间，每次寻道时间最短（但不能保证平均寻道时间最小），但是可能会出现饥饿现象。（半路被打劫，导致一部分磁道永远得不到磁头，因此产生饥饿现象）

（3）扫描算法（SCAN，也叫电梯）：磁头按一个方向移动，直到磁盘一侧末端，哪怕这里没有请求，我也要来，然后返回磁盘的另一端。这种算法可以平均分配磁盘访问的时间，并且会

导致磁道访问频率不平均，对扫描过的区域不公平，会导致一些请求长时间等待，偏向于处理最里或者最外的磁道。（比如正在从 2 号磁道向大磁道行进，但是 1 号磁道有需求，必须等到一来一回才轮到它）

（4）循环扫描算法（Circular-SCAN）：类似于扫描算法，但是磁头到达磁盘的一端时，会立即返回另一端（这个过程不执行任何操作，即规定了访问磁盘只能从一个方向），正向处理请求，反向回家不工作。这种算法可以避免饥饿现象，并且磁盘的访问频率也比较均匀。

（5）SCAN+LOOK 调度：扫描 SCAN 算法+边移动边观察，即磁盘只需要移动到最远端的一个请求即可返回，不需要到磁盘的端点。

（6）Cirlular_SCAN+LOOK 调度：相当于是 C-SCAN 的优化。只移动到最后一个请求的位置，然后再返回到第一个请求的位置。这种算法可以减少磁头的移动次数，提高磁盘访问效率。

表 5.2　四种磁盘调度算法的优缺点

算　法	优　点	缺　点
FCFS 算法	公平、简单	平均寻道距离大，仅应用在磁盘 I/O 较少的场合
SSTF 算法	性能比“先来先服务”好	不能保证平均寻道时间最短，可能出现“饥饿”现象
SCAN 算法	寻道性能较好，可避免“饥饿”现象	不利于远离磁头一端的访问请求
C-SCAN 算法	消除了对两端磁道请求的不公平	—

减少延迟的方法：

①由于磁头读取完一个扇区之后需要一定的处理时间，该时间内不允许进行读操作，所以在这个时间内磁盘仍在旋转，就错过了即将要读的下一个扇区，所以采用交替编号，把逻辑上读相邻的扇区物理上分开，这样可以避免多绕一圈的延迟时间。

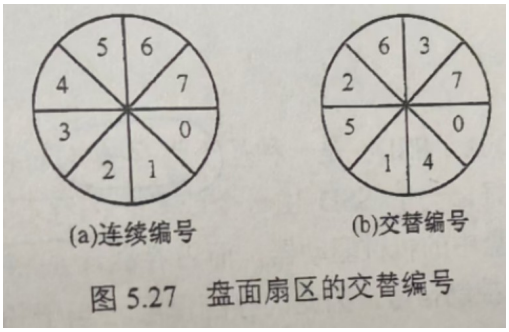


图 5.27　盘面扇区的交替编号

②同时也要对不同盘面的相同位置的扇区错位编号

文件访问速度取决于三个方面：

①目录结构和目录检索方式

②文件的存储结构

③磁盘 IO 速度

提高磁盘 IO 速度的方法：

①磁盘高速缓存：它在磁盘和主存之间提供一个高效的数据缓存层，使得常用数据可以直接从缓存中读取，而不需要实际访问磁盘，从而显著提高了数据访问速度。（由于借用的是主存的内存空间，所以逻辑上属于磁盘，但物理上属于内存）

②调整磁盘请求顺序（磁盘调度算法）

③提前读：在读磁盘当前块时，提前把下一个磁盘块也读入内存缓冲区。

④延迟写：在缓冲区首部加入延迟写标志，释放此缓冲区，等到其它进程申请该缓冲区时，才把该缓冲区内容写入磁盘，腾空该缓冲区。

⑤优化物理块分布：把一个文件放到一个磁道或者相邻磁道，减少寻道时间

⑥虚拟盘：用内存仿真磁盘，存放临时文件

⑦磁盘 RAID 阵列：并行交叉存取

2.固态硬盘读写性能特性，磨损均衡

固态硬盘 SSD

1.读写特性

①以页为单位读写

②以块为单位擦除，擦干净的块，里面的页可写 1 次，读∞次

③随机访问，通过逻辑地址—>物理地址，电路信号迅速定位

④读快，写慢（需要先擦除）

⑤若一页中有数据，先复制到别的新块，再擦整块，再写

2.对比机械硬盘特性

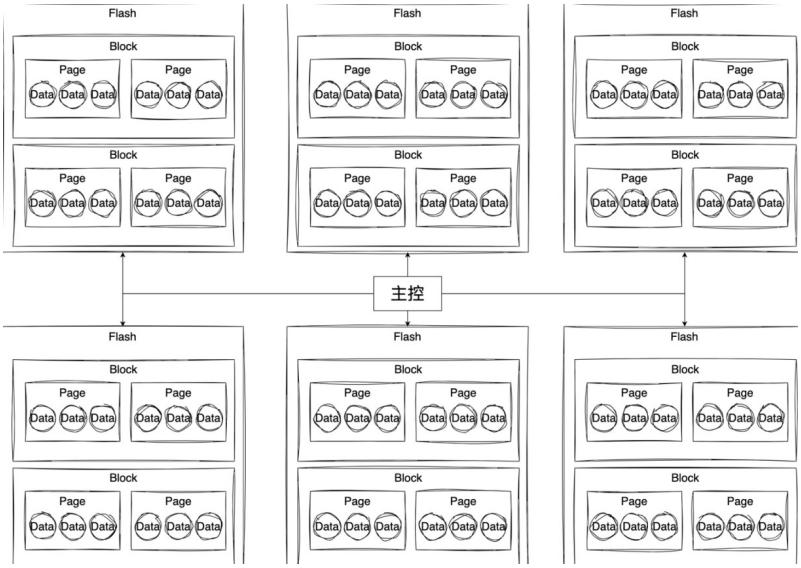
（1）基于闪存（闪存本质上是只读存储器）技术（FLASH 芯片、DRAM 芯片），电可擦除 ROM（EEPROM），与 U 盘无差异，但存取性能更好

（2）半导体存储器，比机械磁盘随机访问速度快（因为它是用电子信号控制，机械硬盘有延迟）

（3）SSD 安静、耐造、能耗低、贵、安全性更好

（4）SSD 的一块，被擦多了会坏，机械硬盘的扇区不会坏

3.组成结构



4.磨擦均衡技术

①擦除平均分布到各个块，提升寿命

② 动态磨损均衡，写时，选累计擦除次数少的块

③ 静态磨损均衡，SSD 监测，让老块读，让新块写